

6120110054

分类号_____

密级_____

UDC_____

学号_____



江西理工大学

硕士学位论文

Thesis for Master's Degree

论文题目 基于 PostgreSQL 与 PostGIS 的空间

数据库设计及应用研究

申请学位类别 理学硕士

专业名称 地图学与地理信息系统

研究生姓名 林媛媛

导师姓名、职称 兰小机 教授

二〇一四年五月

学位论文独创性声明

本人声明所呈交的论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含已获得江西理工大学或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中做了明确的说明并表示谢意。

申请学位论文与资料若有不实之处，本人承担一切相关责任。

研究生签名：林媛媛

时间：2014年5月26日

学位论文版权使用授权书

本人完全了解江西理工大学关于收集、保存、使用学位论文的规定：即学校有权保留按要求提交的学位论文印刷本和电子版本，学校有权将学位论文的全部或者部分内容编入有关数据库进行检索，并采用影印、缩印或扫描等复制手段保存、汇编以供查阅和借阅；学校有权按有关规定向国家有关部门或者机构送交论文的复印件和电子版。本人允许本学位论文被查阅和借阅，同意学校向国家有关部门或机构送交论文的复印件和电子版，并通过网络向社会公众提供信息服务。

保密的学位论文在解密后适用本授权书

学位论文作者签名（手写）：林媛媛

导师签名（手写）：李永红

签字日期：2014年5月26日

签字日期：2014年5月26日

分类号: _____

密 级: _____

U D C: _____

学 号: _____

江西理工大学

硕 士 学 位 论 文

基于 PostgreSQL 与 PostGIS 的空间数据库设计及应用研究

**Based on PostgreSQL and PostGIS spatial database design
and application research**

学 位 类 别: _____ 理学硕士

作 者 姓 名: _____ 林媛媛

学 科、专 业: _____ 地图学与地理信息系统

研 究 方 向: _____ GIS 应用研发

指 导 老 师: _____ 兰小机 教授

2014 年 05 月 20 日

摘 要

流通企业及相关服务是国民经济的重要组成部分，它们是否能够持续、健康、快速、协调的发展将直接制约着经济与社会发展的稳定性。当前，由于流通企业的数据信息量急速的扩张，采用传统的信息服务将很难再达到用户们的需求。所以，采用空间信息技术来解决大数据量、数据种类繁多等多种问题已刻不容缓。但由于企业级软件大多都需要付出高昂的成本代价，并且还存在着诸多有形无形的限制，这就给用户带来极大的不便。而与之相比，开源型软件则大大的不同。开源软件以其源代码开放、版本自由化、软件自身免费等特征得到了人们的关注。就目前而言，Open-source 在 GIS 的发展中占据了一席之地。

本文从开源软件对 GIS 发展的影响出发，针对企业级 DBMS 产品的高代价、多限制等问题，以空间数据库以及空间数据模型的理论内涵为基础，采用开源对象-关系型数据库 PostgreSQL 以及其空间数据引擎 PostGIS 进行重点流通企业的空间数据设计。再以开源 GIS 桌面软件 QGIS 进行了重点流通企业应用系统的研发。具体的工作如下：

(1) 分析介绍了 Open-source 在国内外的研究现状，并着重阐述了开源 DBMS 应用现状以及企业级 DBMS 所存在的问题。还进行了空间数据模型的理论内涵进行了较为深入的挖掘。

(2) 根据空间数据模型以及空间数据库的基础理论知识，应用对象-关系型数据库 PostgreSQL 以及它的空间扩展模块 PostGIS，针对具体的 GIS 应用项目进行了空间数据库的具体设计。

(3) 将 shp、tab、tif 等格式的矢量数据与栅格数据进行了入库的管理工作。为了保证数据在空间数据库中的一致性，笔者对本课题中的数据进行了处理、统一编码以及坐标规范化等工作。

(4) 在充分分析了北京市石景山区的基础地理数据以及重点流通企业数据信息的特点以及北京市石景山区商委的具体需求分析后，采用了开源 GIS 桌面软件 QGIS 进行了重点流通企业信息管理系统的研发工作。

通过上述的研究，本文针对政府部门及企业的中小型 GIS 项目，提出了一种基于全开源架构的解决方案。综合使用了多种当前主流的开源技术，完成了北京市石景山区重点流通企业信息管理系统的空间数据库设计、数据入库管理、PostGIS 数据与 QGIS 链接、QGIS 二次开发配置以及系统的具体研发工作。如此，不但缩短了项目的完成周期，还大大的降低了项目中的成本代价。为构建中小型 GIS 项目提供了新的思路与依据。

关键字：空间数据库；开源软件；GIS；PostgreSQL；PostGIS；QGIS

Abstract

Circulation enterprises and related services is an important part of national economy, they can be sustained, healthy, rapid and coordinated development of will directly restricts the stability of the economic and social development. At present, because of the expansion of rapidly circulation enterprise data information, the traditional information service will be hard to achieve the demand of users. So, the space information technology is to solve the large amount of data, such as different kinds of data in respect of a variety of problems. But as a result of enterprise software mostly needs to pay a high cost price, and also has the limitation of tangible and intangible, but it is inconvenient for users. And, by contrast, open source software is greatly different. With its source code open source software, version, liberalization and free software itself got the attention of people. For now, the Opening -source occupies a place in the development of GIS.

In this paper, starting from the influence of open source software development of GIS, in view of the enterprise DBMS products such as high costing problems, many restrictions, in spatial database and spatial data model based on the theoretical connotation of the source object - relational database PostgreSQL and its spatial data engine PostGIS on spatial data design of circulation enterprises. Is used to the open source GIS desktop software QGIS key circulation enterprise application system development. Specific work is as follows:

(1) The Open - source is discussed in the research status at home and abroad, and emphatically expounds the open-source DBMS application status and existing problems in the enterprise DBMS. Also on the theoretical connotation of spatial data model is discussed in deep mining.

(2) According to the spatial data model and spatial database basic theory knowledge, application objects to relational database PostgreSQL and its spatial extension module PostGIS, is used for specific GIS spatial database application program design.

(3) The SHP, TAB, tif format of vector data and raster data warehouse management work. In order to guarantee the data consistency in the spatial database, the author this project the data in the processing, unified coding and coordinates of standardization work.

(4) On the basis of fully analyzing the Beijing city geography data and the characteristics of key circulation enterprise data information as well as the Beijing city ShangWei specific requirements analysis, focuses on using the open source GIS desktop software QGIS

circulation development of the enterprise information management system.

Through the above research, this article in view of the government departments and enterprises of small and medium-sized GIS projects, this paper proposes a solution based on the open architecture. Comprehensive use of a variety of the current mainstream of open source technology, completed the Beijing city key circulation enterprise information management system of spatial database design, data warehousing management, PostGIS data with QGIS links, QGIS secondary development configuration and system research and development of the specific work. For building small and medium-sized GIS project provides a new train of thought and basis.

Key words: Spatial database;open source software;GIS;PostGIS;QGIS;

目 录

摘 要.....	I
Abstract.....	II
第一章 绪论.....	1
1.1 研究背景与意义	1
1.2 国内外研究现状	4
1.3 研究内容	6
1.4 论文的组织结构	6
第二章 空间数据模型概述.....	8
2.1 空间数据模型的概念	8
2.2 空间数据类型及空间操作	9
2.2.1 OGC 地理信息实现标准——简单要素访问	9
2.2.2 ISO/IEC SQL/MM 空间数据标准	13
2.2.3 两个标准的分析比较.....	15
2.3 空间数据的完整性约束	20
2.3.1 空间数据完整性约束概述.....	20
2.3.2 实体完整性约束.....	20
2.3.3 参照完整性约束.....	21
2.3.4 域完整性约束.....	22
2.3.4 专题语义完整性约束.....	26
2.3.5 时态语义完整性约束.....	27
2.3.6 空间语义完整性约束.....	27
2.3.7 关系类完整性约束.....	30
2.3.8 组合语义完整性约束.....	31
2.3.9 空间数据多版本约束.....	31
2.3.10 触发器约束.....	31
第三章 基于 PostGIS 的重点流通企业空间数据库设计	34
3.1 平台概述	34
3.1.1 PostgreSQL 的概述	34

3.1.2	PostGIS 概述	36
3.2	PostGIS 数据模型	36
3.2.1	矢量数据模型	36
3.2.2	栅格数据模型	39
3.2.3	拓扑数据模型	40
3.2.4	长事务管理及线性参考	42
3.3	重点流通企业空间数据库设计	43
3.3.1	需求分析设计	44
3.3.2	分层设计	44
3.3.3	概念结构设计	45
3.3.4	逻辑结构设计	46
3.3.5	完整性约束设计	49
3.3.6	触发器设计	52
第四章	重点流通企业信息管理系统的设计及实现	53
4.1	系统设计原则	53
4.2	系统的架构	54
4.3	系统应用逻辑设计	54
4.3.1	客户逻辑设计	54
4.3.2	数据逻辑设计	54
4.3.3	系统功能设计	55
4.4	空间数据库的环境配置与建立	56
4.4.1	环境的搭建与配置	56
4.4.2	空间数据入库	60
4.4.3	空间数据的浏览	61
4.5	重点流通企业信息管理系统的实现	62
4.5.1	北京市石景山区概况	62
4.5.2	重点流通企业信息管理系统	63
第五章	总结与展望	73
5.1	总结	73
5.2	展望	73
参考文献		75
致谢		77

攻读学位期间的研究成果.....	78
------------------	----

第一章 绪论

1.1 研究背景与意义

在我国，由于信息化建设起步相对较晚，目前大多数的国内企业仍处于信息化建设的第一阶段，即简单应用阶段，而这一比例更是达到了 75%。而在这 75%的企业中，绝大多数的又是中小型企业，它们根本没有建设信息化重要性的意识，或者是因为自身存在的限制等，而导致这些企业不能建设比较完善的信息化系统，从而也制约着它们在管理能力以及其他方面上的发展。昂贵的建设成本是造就这种状况出现的主要原因之一。于是能够为中小型企业或是中小型项目构建系统平台的开源软件和开源 DBMS 就显得尤为的重要。当然，这高昂的成本代价对于大型企业或是大型项目来说也并不十分的乐观。即使已付出了巨额的成本，其在信息化的建设程度也还是普遍较低，大多数也只是应用在了一些很底层的需求上，并没有真正的把信息化建设与企业的管理进行相结合。

总之，因信息化起步较晚，我国的信息化水平与欧美发达国家相比仍处于相对低的水平之上。所以，想要提高我国的企业运作、政府建设以及产业化发展等工作的效率，基础信息化的建设将为他们提供动力。但是随着政府推动经济体制的改革，信息化建设在中小型企业或是中小型项目当中的发展，开源软件平台将对它们起到不可置否的推动与促进。

在社会经济的迅猛发展下，在当前空间信息技术的高速发展下，应用空间技术实现政府部门基于 GIS 的电子政务办公已成为时下的普遍趋势。而在这些大大小小的项目中，采用开源软件平台不但会为企业或是政府部门获取竞争的优势，还能提高管理的效率、降低企业或是项目的成本。并且针对有些政府部门的数据信息需要保密的特性，采用开源的数据库对保密的数据信息进行存储自然是不二的选择。

目前市场上比较主流的开源 DBMS 大体上有三类：由科研项目转变而来的代码开放型数据库，如 PostgreSQL。它是将自己的源代码进行了开放性的处理；由个人或是几个人组成的研发小组进行自主研发的数据库，如 MySQL；由一些企业级数据库厂商，将自己的源代码进行公开的数据库，如 FireBird^[1](李建峰，2006)。

据统计显示，目前在应用了数据库产品的企业中，已有约四成的企业，在业务中选择使用开源 DBMS 产品^[2]。另有约三成的企业正逐步的向开源 DBMS 产品进发。随着开源 DBMS 产品的日趋稳定与成熟，开源 DBMS 的迅猛发展，也将会给企业级数据库带来巨大的冲击。而这样的发展也势必会使数据库产品更加的多元化，并且也使得用户有了更多更好地选择空间。

目前，为人所熟知的开源 DBMS 的种类有上百种，而被人们经常选择使用的，则有 60 多种。而在这些数据库产品当中，应用最为广泛的就是 MySQL 和 PostgreSQL。表 1.1 是目前较为主流的开源 DBMS 的信息比较表。

表 1.1 开源 DBMS 信息比较表

		Open Source DBMS			
		PostgreSQL	FireBird	MySQL	Ingres
ACID		Support	Support	Support(InnoDBtable)	Support
Associated Integrity		Support	Support	Support(InnoDBtable)	Support
DB transactions		Support	Support	Support(InnoDBtable)	Support
Unicode Index	R-/R+	Support	Support	Support	Support
	Hash	Support	NoSupport	Only MyISAM	Support
	GiST	Support	NoSupport	Only InnoDB	Support
			NoSupport	NoSupport	NoSupport
Table partition	Range	Support	NoSupport	Support	Support
	Hash	Support	NoSupport	Support	Support
	List	Support	NoSupport	Support	Support
Cluster		Support by add-on	NoSupport	Support	Support
Spatial extension		Support	NoSupport	Support	Support

经过多年发展的商业型数据库，其性能是十分优越的，而且在行业的解决方案上也日趋完美。现在，被用户们所信赖的主要是甲骨文的 Oracle 和微软的 SqlServer 以及 IBM 的 DB2。但正因其优越的性能以及成熟的行业解决方案，也使得使用这些商业型数据库的代价变得越来越高昂。而在此时，开源 DBMS 便应运而生了，与商业型数据库相比较，虽然开源 DBMS 的发展历史比较短，但其的发展速度却是相当快的。就目前用户的反应来看，被用户熟识的开源 DBMS 在性能上是可以与企业级数据库一争高下的。而且与商业型数据库比，开源 DBMS 需要的存储空间小，但却能够满足用户的绝大多数的需求。例如，PostgreSQL 和 MySQL 等。表 1.2 为开源 DBMS 与商业数据库在性能上的信息比较。

表 1.2 开源 DBMS、商业数据库性能比较信息表

	PostgreSQL	MySQL	SQLServer	Qracle
数据库类型	对象-关系型	关系型	关系型	关系型 (Oracle Spatial 支持几何对象)
平台支持	UNIX、Windows/NT、Mac 平台	UNIX、Windows/NT、Mac 平台	Windows/NT、Mac 平台	UNIX、VMS、Windows/NT、OS/2、Mac 平台
数据类型	传统类型、几何类型、自定义数据类型	传统类型、Open GIS 几何类型 (仅限于 MyISAM 表)	传统类型、自定义数据类型	传统类型
SQL 支持	全面支持 SQL89 并支持 SQL92 大部分特性	支持 ANSI/ISO SQL 标准	支持 Entry 级 SQL-92 标准	与 ANSI/ISO SQL89 二级完全兼容
数据文件最大容量	32TB	64TB(应用 InnoDB 存储引擎)	32TB	4194394(与操作系统有关)*数据块大小
索引机制	B-Tree、R-Tree、Hash、聚集	B-Tree 磁盘表 (MyISAM) 和索引压缩	B-Tree	B+Tree、位图索引、聚集、分区表与分区索引
用户权限管理	Pg-user 系统视图	登陆、用户、角色和组	授予权限、用户和角色	用户认证、用户口令管理、权限和角色管理
前端命令操作	Psql	Mysql	查询分析器	SQL*plus
并发控制	多版本并发控制	多版本并发控制	允许大容量数据操作、创建索引、文件增长、文件修改	并行服务器并行 SQL 语句处理

从上表来看, 开源 DBMS 不但价格十分的低廉, 而且其在性能上与企业级数据库相比, 也毫不逊色。而在空间数据的处理上以及效率上, 开源 DBMS 也存在着非常大的潜能。例如 PostGIS, 它是对象-关系型数据库 PostgreSQL 的一个扩展模块。PostGIS 的主要效用就是针对空间数据信息进行存储与管理。并且它在处理空间数据的性能上比很多关系型数据库都更胜一筹。PostGIS 不但支持多种类型的空间数据, 而且它还被众多软件产品所支持(如, Mapguide, Autodesk Map 等)。在中小型项目中, PostgreSQL 与 PostGIS 以其低廉的成本优势以及那可以满足用户绝大部分需求的强大功能, 势必将成为中小微型企业的首选。特别是在一些政府部门的项目中, 由于出于对安全、资金等的考量, 采用 PostgreSQL 与 PostGIS 以及其他的开源性 GIS 软件产品, 不仅能够大大的缩减财政预算, 既节省了资金很好的完成了工作, 又能够很好的保证安全性, 这也正是

政府部门喜闻乐见的。

本文将以北京市石景山区商务委的项目为载体,提出一种基于全开源体系结构下针对重点流通企业信息进行管理的解决方案。之所以会对这个课题进行研究,主要有以下几个原因:首先是基于成本的考量,所以选择抛弃功能上强大成本也十分高昂的商业型数据库,转而选择价格低廉且没有诸多限制的开源 DBMS 产品;其次,在进行了企业级与开源 DBMS 比较之后,因开源 DBMS 体积小,且日渐成熟,所以选择了它。而且,选择全开源的架构,也可以为中小企业或是中小型项目提供一个代价相对较小,又能够完全满足需求的新思路。最后,则是出于针对某些政府部门对数据信息的保密问题,应用开源的数据库以及开源的软件,能够完成上述任务。因为,开源软件它没有过多的限制存在,也不属于任何国家、任何企业以及个人,这样无形为它增添了相对的安全性

与开源的数据库产品一样,开源的 GIS 软件的数量也是相当众多,而当中也有不少性能优越、稳定,功能较为强大的 GIS 软件如 QGIS、ShrpMap、Google Maps Navigation、OpenLayers 以及 GIS 的开源服务器 GeoServer 等。如今,在国外也已经出现了一些较为成功的案例。在笔者大量的阅读国内外的文献以及网上资料之后,针对各个社区用户进行调研以及充分的了解各个产品的功能和特点下,进行比较系统的对比后,最后选择了性能优越、相对稳定的 PostgreSQL 以及它的空间扩展模块 PostGIS 和 QGIS 作为构建石景山区的重点流通企业信息管理系统的平台。

1.2 国内外研究现状

目前对于面向对象型数据库的研究仍处在不断摸索的阶段。所以,对于空间数据库的研究方向就出现了两个分支。第一,则是对对象型数据库更深入的挖掘与研究。第二,即是对商业型的关系数据库进行空间功能上的扩展,使之有能力对空间数据信息进行存储与管理^[3]。对象-关系型数据库是传统的关系型数据库的延伸,即在关系型数据库功能上的一种延伸。这种关系型的数据库有能力为空间数据对象提供支持,进行几何要素类型、空间参考等类型的存储与定义。目前,普遍被人们所熟识的对象-关系型数据库有 Oracle、Informix、DB2 等。这些原关系型的数据库,纷纷推出了针对空间数据进行管理的扩展模块,还提供了空间对象的 API 函数,例如对点、线、面等空间对象的定义。这些 API 函数,可以进行各种空间对象数据结构的预定义。但用户需按着它所定义的数据结构进行使用,不能根据具体的 GIS 要求(即使是 GIS 软件商)进行重定义。这就使得用户在使用它们时,所需要付出的代价变得十分高昂,并且这些商业型数据库软件在面对海量数据时,其的存取速度也将随数据量的增多而逐渐降低。

正因商业巨头们为着自身的利益而对用户做出了诸多限制与高额的使用费用,才使得人们对于自由的开源技术不戢的追求。开源技术正以其自由、资源共享以及低费用甚

至是完全免费的理念逐渐斩获了广大用户的青睐。开源技术能够提供一个可以替代专业软件的平台，与专业软件相比，开源技术将更加的可靠、灵活和具有竞争力。目前，对于开源技术在地理信息系统(GIS)上的探索与拓展也成了 GIS 研究领域的一个热门方向。

在国外，许多国外的知名专家们也都着手针对开源 GIS 软件与开源 DBMS 进行了深入与细致的研究。像 Song^[4]等，基于 MapServer 的柬埔寨环境地图系统；Jaroslav^[5]等采用 GRASS 实现一种太阳辐射模型的集成应用；C.George^[6]等采用开源的 MapWindow 开发的联合国土壤和水资源评价工具；Bas Van-meulebrouk^[7]等在南非 Cell-Life 非政府组织的支持下，研究了利用开源 GIS 软件开展了 HIV/AIDS 管理信息系统的研究；Alessandro Bezzi^[8]等采用了开源的 GRASS 在荷兰的 ITC 支持下开展了考古方面的研究，并实现了模型建模与管理；Lars Gunnar 和 Trond Andresen^[9]进行了基于开源的 MapServer 软件在地区健康管理方面上的研究与开发实践；Andrew J^[10]则是利用了已出版的卡特琳娜飓风地图结合了开源 GIS 软件研究了此次灾害下的死亡率与位置的关系。而在目前，许多国外的企业也正走向开源 GIS 的研究，例如：Autodesk^[11]公司支持很多研究机构开展基于开源 MapGuide 的网络空间信息服务方面的研究；以及 NASA^[12]也在进行着这方面的研究，比如支持开源影像发布的技术研究---NASA WorldWind。

将目光在转向国内的学术界，开源 GIS 软件与开源 DBMS 的研究也悄然而至。在国内利用开源 GIS 软件的主要是一些行业部门，他们主要是利用开源的 GIS 进行地图制图与 Web 发布等功能。例如：熊静^[13]开发了一个基于 MapServer 的遥感影响发布的信息系统；郑斌^[14]等采用了开源的 GeoTools 平台设计与实现了一个城市基准地价信息的发布系统；圣荣^[15]等则是研究了基于 MapServer 的网络空间信息共享系统；张大鹏^[16]等是应用了 GeoServer 研发了一个 110 指挥中心警情分析系统共；杨朝辉^[17]等是利用开源的 GeoServer 与 PostGIS 实现了网络房地产估价系统的设计；朱俊峰^[18]则是开展了基于 SharpMap 和 NTS 构建的 WebGIS 的研究；黄冲^[19]等则是研究了基于开源的 WebGIS 的最短路径算法；冯宇^[20]等做了基于开源 WebGIS 的干线公路网用地控制系统；宋现锋^[21]则是利用开源的 MapServer 进一步的开展了 Flash 地图的研究；吕德奎^[22]等研究了开源版 MapGuide 的应用模式；胡庆武^[23]等人采用了开源 Nasa WorldWind 的 G-S 空间信息服务模式，并将这个研究应用到了九寨沟旅游信息系统的研发中。

以上的研究体系大多数都是将开源 GIS 软件与开源的数据库应用到了基于 WebGIS 的 B/S 架构的研究中，很少有人会在 C/S 架构下应用开源技术针对某一项目提出适宜的解决方案。本文的研究重点则是一个基于 C/S 架构下针对重点流通企业数据信息应用 PostGIS 进行空间数据库的设计及应用研究。

1.3 研究内容

在开源技术迅猛发展的大背景下，开源软件产品也已越见成熟、稳定，并且社区等机构能够为其提供良好的技术支持。目前，成熟的开源 DBMS 产品比较多，但是针对中小型项目来说，选择一款功能强大，成本低廉，又能完全满足项目需求的方案，正是本文众多工作当中的一项。

首先，笔者深入的研究了空间数据库的基础与理论等知识，继而再理清空间数据库设计的基本逻辑及思路。

其次，笔者在大量的阅读国内外的文献之后，根据项目的具体要求，将着重研究 PostgreSQL 以及其空间扩展模块 PostGIS 的理论基础，并在 PostGIS 的空间数据模型等理论基础的指导下，进行石景山区重点流通企业信息管理系统的空间数据库的具体设计工作。

最后，本文将应用 PostgreSQL 以及它的空间扩展模块 PostGIS 作为石景山区重点流通企业信息管理系统的后台数据库，应用 PostGIS 来存储业态的分布数据，限上/限下企业数量的数据，流通产业发展以及消费变化趋势等空间数据以及属性数据。再应用开源的 GIS 软件 QGIS 作为前端客户端，以 QGIS 二次开发来实现用户在功能上的需求，完成北京市石景山区商委重点流通企业信息管理系统。

在系统开发的过程当中，以 VS2010 为开发环境，采用 C++ 语言，应用 QT 4.7.3 和 QGIS2.0 链接库对 QGIS 源码进行编译后，再开发基于 QGIS 平台的北京市石景山商委重点流通企业信息管理系统。本系统包含了 GIS 的一般功能、针对石景山区的经济中心地带一轴三心四街的查询、统计及分析功能以及具有特色的蔬菜服务网点分布的统计与分析功能。此系统统共有五部分组成：分析管理模块、统计管理模块、编辑管理模块、查询管理模块以及规划模块。它实现了针对电子地图的基本操作，对商业业态、重点限上/限下企业、蔬菜服务网点分布等的查询、统计分析等功能。还将不同年份的规划图进行叠加，以此来查看区内的规划情况。

1.4 论文的组织结构

本文共有五个部分组成：

第一部分为绪论，对地理信息系统以及其的发展进行阐述，并对数据库的发展历程以及国内外的研究现状进行了详细的阐述。还针对商业数据库与开源 DBMS 进行了比较系统的比较，并通过二者的比较引出在针对管理重点流通企业的解决方案中，选择全开源系列软件产品的具体原因。并在此部分进行了课题来源，研究背景及意义等说明。

第二部分为空间数据模型概述。分别空间数据模型的概念、空间数据的类型及空间

操作进行了系统剖析。还介绍了 ISO/IEC 与 SQL/MM 空间数据标准，并对这两种数据的标准进行了比较与分析。最后则是对空间数据库的完整性约束进行了比较详细的阐述。并以上述的各个理论作为本研究的理论基础。

第三部分详尽的叙述空间数据库设计的理论基础与基于 PostgreSQL 与 PostGIS 下的北京市石景山区的重点流通企业信息管理系统的空间数据库进行了具体设计，将分别从需求、分层、概念、逻辑以及完整性约束等方面介绍信息管理系统的空间数据库的设计思路。

第四部分为北京市石景山区重点流通企业信息管理系统的设计与实现。在本章中将着重介绍重点流通企业信息管理系统的整体设计原则与思路。然后，进行了具体的系统研发以及系统基本测试的工作。最后则以截图来对系统的功能进行了展示与说明。

第五部分是对本次研究的总结和展望，总结了研究中所完成的工作，研究中取得成果，总结了本课题中的不足、需要改进与进一步完善的地方，并提出了一些展望。

第二章 空间数据模型概述

空间数据模型是 GIS 与空间数据库的理论基础。空间数据模型是人们对现实世界地理实体、现象以及它们之间相互关系的认识和理解，是现实世界在计算机中的抽象与表达，对海量空间数据的描述、组织、管理以及共享具有重要的作用。

2.1 空间数据模型的概念

空间数据模型是地理信息系统和空间数据库的理论基础，它的研究一直是是地理信息科学的基本问题。空间数据模型是人们对现实世界地理现象、实体以及它们之间相互关系的认识和理解，用一定的方案建立起数据组织方式实现计算机对现实世界进行抽象与表达，对于描述、组织以及共享海量空间数据具有重要的作用。

空间数据模型是关于现实世界中空间实体及其相互间联系的组织形式，它为描述空间数据的组织和设计空间数据库模式提供基本方法^[24]。空间数据模型的三要素是：空间数据结构、空间数据操作和空间完整性约束。

① 空间数据结构：空间数据结构是对系统静态特性的描述。它主要用来描述空间数据的类型、内容、性质以及纾解见的关系等。由于不同的空间数据结构有着不同的操作与约束，所以得出空间数据结构是空间数据模型、空间数据操作以及约束的基础。

OGC 简单要素访问规范定义了常用的空间数据类型，即几何类型，如 Point、MultiPoint、LineString、MultiLineString、Polygon、MultiPolygon 等；

② 空间数据操作：空间数据操作是对系统动态特性的描述。它主要用来描述空间数据库中的空间对象所能够执行的操作与有关的操作规则。空间数据模型定义的操作，包括操作的确切含义，运算符号，操作规则（如优先级）和语言操作的实现。

在 OGC 简单要素访问规范，定义空间操作可以分为三类：a) 用于所有几何类型的基本操作；b) 用于空间对象进行拓扑关系测试的操作；c) 用于空间分析的一般操作。

③ 空间数据约束：空间数据约束主要描述空间数据间的语法、词义联系、它们之间的制约和依存关系，以及空间数据动态变化的规则。完整性约束的模型有：1)在本数据模型中必须遵守完整性约束条件的规则，如实体完整性与参照完整性。2)对于所反映的具体应用所涉及的数据必须提供定义完整性约束条件的机制并遵守特定的语义约束条件。

空间数据约束包括常规的数据库完整性约束、几何完整性约束和空间关系完整性约束等。几何完整性约束主要包括线不能自相交、多边形不能自重叠等；空间关系完整性

约束主要包括拓扑完整性约束(如相邻、重叠等)、语义完整性约束(如水域中不能存在陆地)和自定义完整性约束(如湖岸 500m 范围内的树木不能采伐)。

2.2 空间数据类型及空间操作

空间数据类型及空间操作是空间数据模型的主要组成部分。空间数据模型的设计、空间数据库系统的性能与查询语言的效率都与他们有着密切的关系。空间数据类型的定义和实现是开发 SDBMS 要解决的最基本问题,同时对空间数据类型的了解,也有助于我们深入理会某个特定的 SDBMS 本质特征,从而进一步帮助我们设计和维护好实际 GIS 工程项目中的空间数据库。

为了规范空间数据类型及空间操作算子的设计与实现,OGC 和 ISO/IEC 国际标准化组织制定了空间数据类型标准以及每一种空间数据类型拥有的空间操作算子标准。以下重点介绍这些标准。

2.2.1 OGC 地理信息实现标准--简单要素访问

OGC 地理信息实现标准--简单要素访问(Simple feature access, 简称为 SFA, 目前最新版本为 1.2.1)包括通用体系架构(Common architecture)和 SQL 选项(SQL option)两部分。

Common architecture 的内容包括:

- ① Geometry 对象模型
- ② 文本标注(ANNOTATION TEXT)
- ③ Geometry 的 WKT 表达
- ④ Geometry 的 WKB 表达
- ⑤ 空间参考系的 WKT 表达

SQL 选项(简称 SFASQL)的主要内容包括:

- ① 使用预定义数据类型的 SQL 实现
- ② 使用扩展 GEOMETRY 类型的 SQL 实现

(1) SFA 几何对象模型概述

简单要素几何对象模型中(Geometry Object Model)定义了 17 个几何对象类型: Geometry、Point、Curve、LineString、Line、LinearRing、Surface、Polygon、PolyhedralSurface、Triangle、TIN、GeometryCollection、MultiPoint、MultiCurve、MultiLineString、MultiSurface 和 MultiPolygon, 它们之间的层次关系如图 2.1 所示。

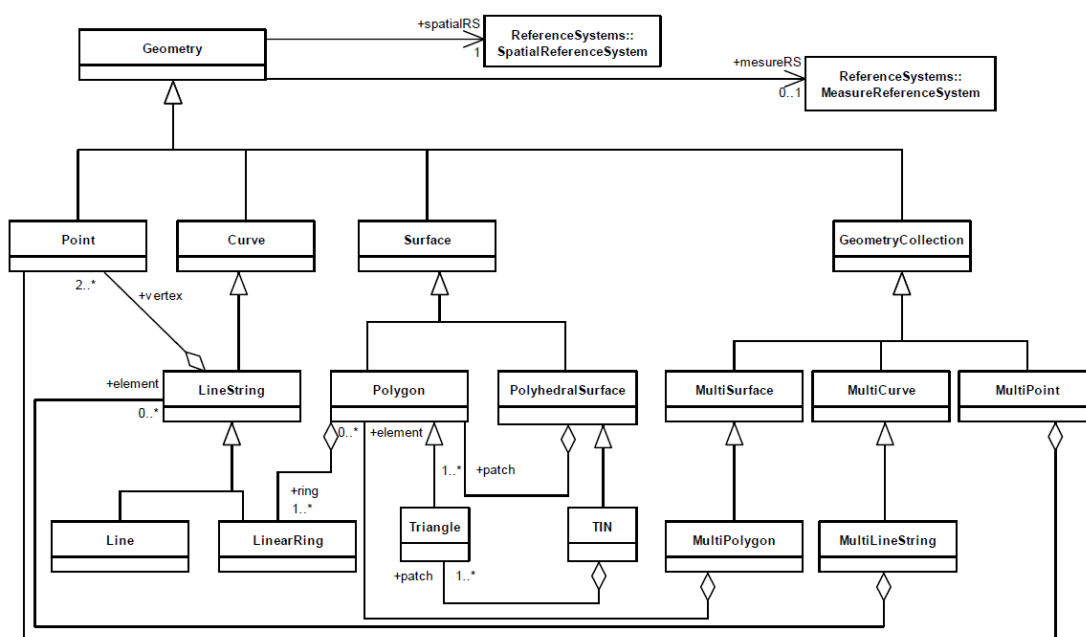


图 2.1 OGC 简单要素访问(版本 1.2.1)几何对象模型图

(2) SFASQL 的主要内容

OGC 的 SFASQL 标准已被 ISO TC211 吸纳成为 ISO 19125 标准。在 SQL 实现机制中，地理要素集被存放于数据表中，地而非空间属性则用 SQL 数据类型的字段来记录。空间属性的存储存在着两种方式：一个是 SQL 中预定义的数据类型，另一个则是 SQL 中扩展的 GEOMETRY 类型。

1) 预定义数据类型的 SQL 实现

SQL 预定义数据类型中对要素表、几何对象以及空间参考信息进行了定义。如图 2.2 所示。该模式包括以下 4 个表：

- ① GEOMETRY_COLUMNS 表：描述要素表及相关的几何属性；
- ② SPATIAL_REF_SYS 表：描述坐标系和几何变换；
- ③ FEATURE TABLE：存储要素集合，要素表中的 Geometry Column(GID)作为外关键字与 GEOMETRY TABLE 相关联；
- ④ GEOMETRY TABLE：使用 SQL 数值类型或 SQL 二进制类型存储要素的几何对象。

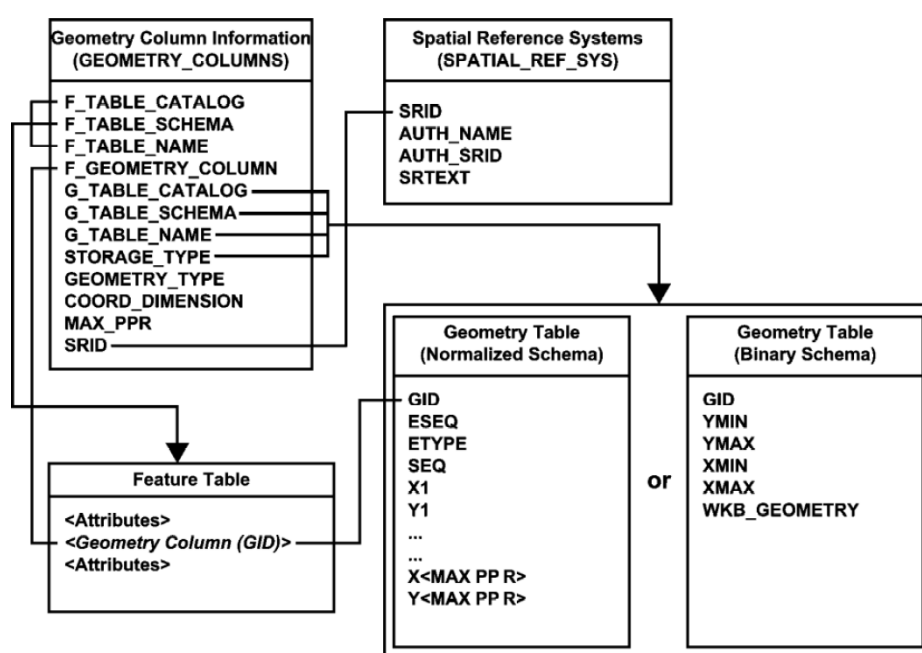


图 2.2 使用预定义数据类型的要素表模式

SQL 预定义数据类型中具有常规数值类型和二进制类型这两种实现模式。用常规数值类型实现方式存储地理要素时，要素的几何形状存于几何表中，而几何对象的 GIS 则存在了几何的列字段中。通过一些列的坐标进行每个地理要素的描述，并用 SQL 预定义的数值类型进行坐标的存储。二进制方式与常规数值方式在存储对象上是基本相似的，只有一处不同的地方。即通过二进制来表达要素的几何形状。几何表中的字段行与要素要一一相应。

使用 SQL 常规数值类型的几何存储模式将几何对象的坐标以预定义 SQL 数值类型存储。在几何表中，一个或多个坐标点(X, Y, 可选的 Z、M 值)可以用数值对表达(如图 2.3 所示)。每个几何对象由关键字(GID)标识并且由一个或多个按顺序(ESEQ)排列的基本元素组成。几何对象中的每个基本元素可能被分配到几何表的一行或多个行中，跨多行时按序列号(SEQ)排序，元素的几何类型由 ETYPE 标识。

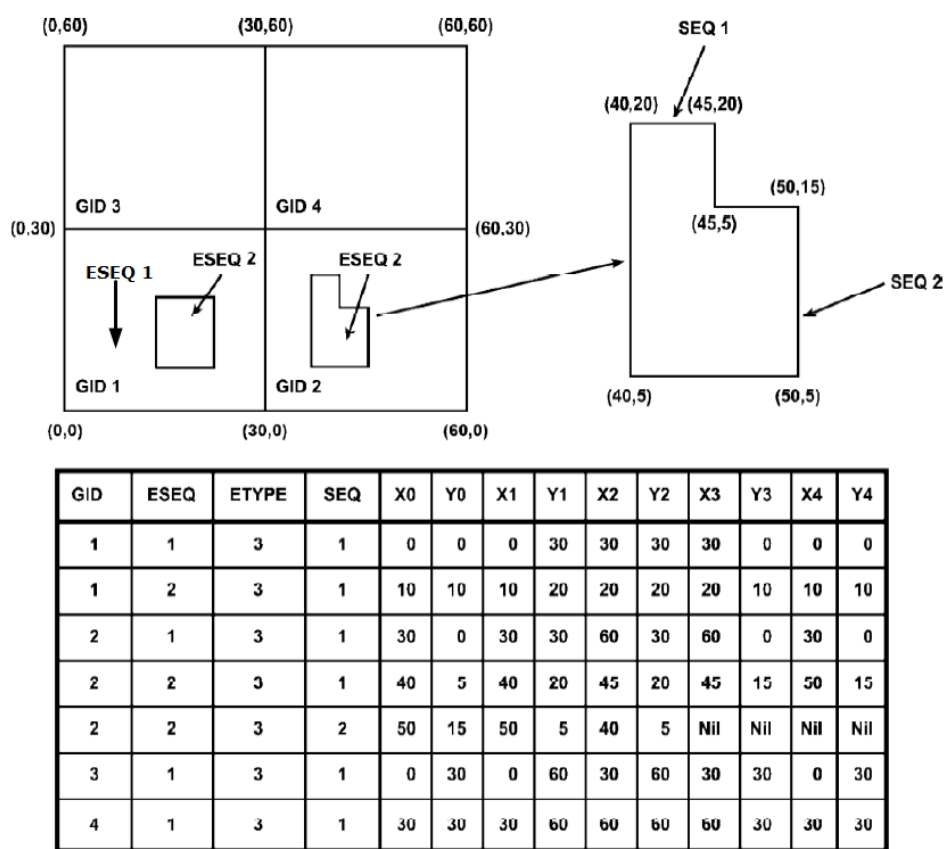


图 2.3 使用 SQL 数值类型时多边形在几何表中存储样例

几点说明：

- (a) ETYPE 指明元素的几何类型，如 Point、LineString、Polygon、MultiPoint、MultiLineString、MultiPolygon 等；
- (b) 几何对象由多个元素组成时，由 ESEQ 序号标识不同的元素；
- (c) 一个几何元素可能跨多行，由 SEQ 序号标识不同的行；
- (d) 多边形的边界坐标首位要一致，即最后一个点的坐标与第一个点的坐标相同；多边形可以包含空洞；当多边形的环由多个部分按顺序组装而成时，每部分由 SEQ 来标识其顺序；
- (e) 当一个几何对象存储跨多行时，下一行的第一个点要重复上一行的最后一个点，即它们的坐标相同。
- (f) 对几何对象的组成元素的数量没有限制，对一个元素占用多少行也没有限制。

2) 使用 SQL 二进制类型的几何存储模式

使用 SQL 二进制类型的几何存储模式用 GID 作关键字，并且使用 WKBGeometry 的方式存储几何对象。几何表包含了几何对象的最小矩形框以及 WKBGeometry，如表 2.1 所示。

表 2.1 使用 WKBGeometry 时多边形的几何存储模式

GID	XMIN	YMIN	XMAX	YMAX	Geometry
1	0	0	30	30	<WKBGeometry>
2	30	0	60	30	<WKBGeometry>
3	0	30	30	60	<WKBGeometry>
4	30	30	60	60	<WKBGeometry>

3) 使用扩展 GEOMETRY 类型的 SQL 实现

使用扩展 GEOMETRY 类型的实现模式定义了管理要素表、几何对象和空间参考系信息的模式，如图 2.4 所示。该模式包括以下 3 个表：

- ① GEOMETRY_COLUMNS 表：描述可用的要素表及相关的几何属性；
- ② SPATIAL_REF_SYS 表：描述坐标系和几何变换；
- ③ FEATURE TABLE：存储要素集合，要素表的 Geometry Column 为 SQL 的 GEOMETRY 扩展类型；

Geometry 扩展类型的实现模式使用 UDT(User Defined Type)扩展 SQL 的类型系统，通过 UDT 提供扩展的 Geometry 类型和相关函数(方法)，从而为 SQL 提供空间数据访问功能。这种实现模式下的几何对象模型同 SFA 的 Common architecture 中的几何对象模型一致，如图 2.1 所示。

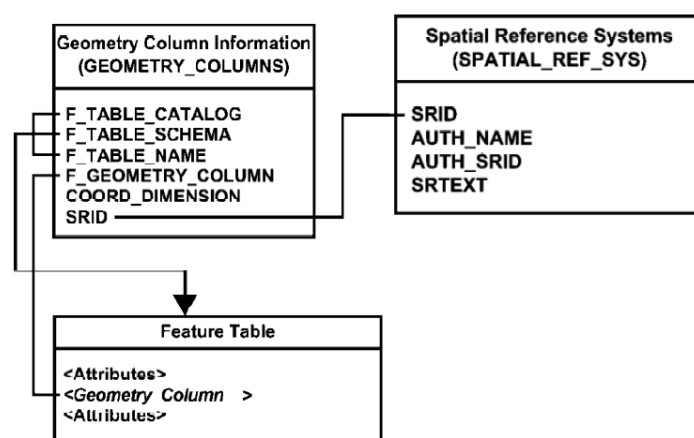


图 2.4 使用 SQL 的 Geometry 扩展类型的要素表模式

2.2.2 ISO/IEC SQL/MM 空间数据标准

国际标准化组织(the International Organization for Standardization: ISO)与国际电工委员会(the International Electrotechnical Commission: IEC)的第一联合技术委员会(ISO/IEC JTC1)与数据管理和交换分技术委员会(SC32)联合负责制定 SQL/MM 标准(全称为: SQL Multimedia and Application Packages, ISO/IEC 13249)，该标准分为以下五部分：

Part 1 Framework: 列出所有的公共概念, 并说明了它们在其它部分中的应用方法。

Part 2 Full-Text: 定义了支持文本数据存储的结构化用户定义类。

Part 3 Spatial: 定义了矢量数据存储与检索标准。

Part 4 General Purpose Facilities: 已撤销, 目前尚无内容。

Part 5 Still Image: 定义了栅格数据存储与检索标准。

Part 6 Data Mining: 定义了数据挖掘相关标准。

以下简要介绍 SQL/MM: Part 3 Spatial(ISO/IEC 13249-3)相关的主要内容。

(1) SQL/MM 几何对象模型概述

SQL/MM 几何对象模型中共定义了 18 个几何对象类型: ST_Geometry、ST_Point、ST_Curve、ST_LineString、ST_CircularString、ST_CompoundCurve、ST_Surface、ST_CurvePolygon、ST_Polygon、ST_PolyhedralSurface、ST_Triangle、ST_TIN、ST_GeomCollection、ST_MultiPoint、ST_MultiCurve、ST_MultiLineString、ST_MultiSurface 和 ST_MultiPolygon, 所有的对象名称都以 ST_为前缀。它们之间的层次关系如图 2.5 所示。

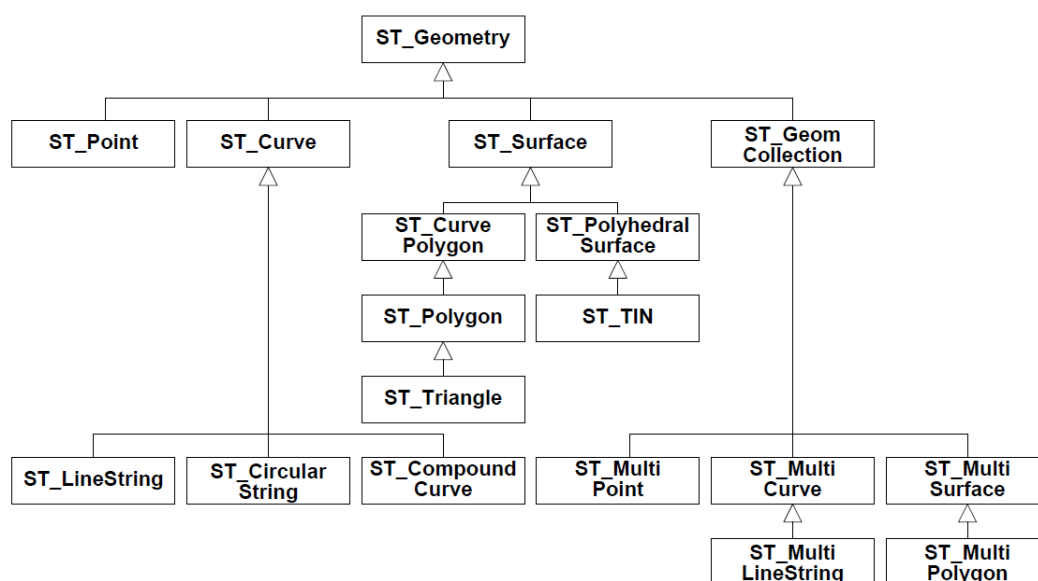


图 2.5 SQL/MM 几何对象模型图

曲线(Curve, 类型 ST_Curve)的子类包括直线串(ST_LineString)、圆曲线串(ST_CircularString)和复合曲线串(ST_CompoundCurve)。ST_LineString 是一种以线性内插方式在两点间生成的直线串。ST_CircularString 则是一种以圆弧插值方式在三点间生成的弧段。ST_CompoundCurve 则是一个以直线与弧段进行插值而生成的直线串组合, 并且弧线与直线可以连接成曲线。二维 ST_Surface(几何面)的子类是 ST_CurvePolygon(曲线多边形)。ST_CurvePolygon 则是由曲线构成的含洞的多边形, 其子类 ST_Polygon(多边形)则是由直线串构成的, 如图 2.6 所示。

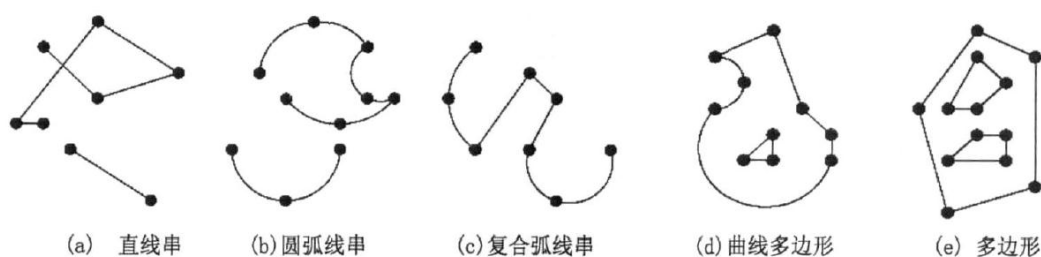


图 2.6 曲线和多边形的例子

(2) 要素表实现模式

SQL99 是 SQL/MM Spatial 标准的基础。它的扩展环境描述和 SFA SQL 所支持的 UDT 扩展环境是基本一致。并且它在实现方式上与 SFASQL 中的 Geometry 类型很相似。图 2.7 表述了 SQL/MM Spatial 中要素表的实现模式。ST_GEOMETRY_COLUMNS 表、ST_SPATIAL_REF_SYSTEMS 表和 FEATURE TABLE 与 SFA SQL 的扩展 Geometry 类型对应的表也类似。

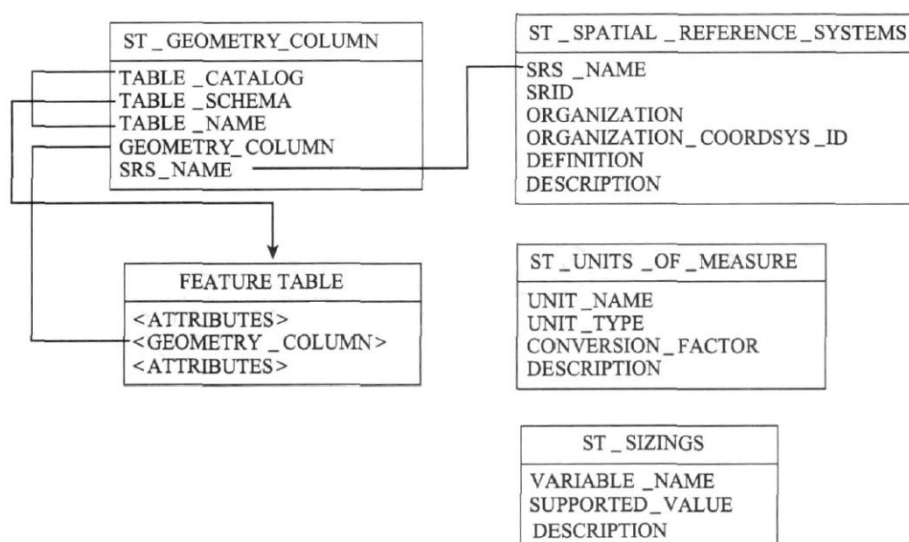


图 2.7 SQL/MM 基于 Geometry 扩展类型的实现模型

2.2.3 两个标准的分析比较

(1) 几何对象对比分析

两个标准定义的几何对象模型基本一致，但也有各自的特点，它们的对比如表 2.2 所示。

表 2.2 两个标准几何对象对比表

序号	SFASQL	SQL/MM	SFASQL	SQL/MM
1	Geometry	ST_Geometry	GeometryCollection	ST_GeomCollection
2	Point	ST_Point	MultiPoint	ST_MultiPoint
3	Curve	ST_Curve	MultiCurve	ST_MultiCurve
4	LineString	ST_LineString	MultiLineString	ST_MultiLineString
5	Line	-	MultiSurface	ST_MultiSurface
6	LinearRing	-	MultiPolygon	ST_MultiPolygon
7	-	<i>ST_CircularString</i>	-	<i>ST_Angle</i>
8	-	<i>ST_CompoundCurve</i>	-	<i>ST_Direction</i>
9	Surface	ST_Surface	-	<i>Topology-Geometry</i>
10	-	<i>ST_CurvePolygon</i>	-	<i>Topology-Network</i>
11	Polygon	ST_Polygon	Annotation Text	-
12	PolyhedralSurface	ST_PolyhedralSurface		
13	Triangle	ST_Triangle		
14	TIN	ST_TIN		

SFASQL 规范中既对几何对象类进行了定义，又对注记文本类型进行了定义。而 SQL/MM 标准承袭了 SFASQL 中的部分对象模型，同时也对继承的这部分进行了相应的扩展。定义了方向(ST_Direction)和角度(ST_Angle)类型，同时还定义了拓扑几何(Topology-Geometry)和拓扑网络(Topology-Network)的实现模型。SFASQL 的几何对象只对线性插值方式进行了定义，而不对圆弧插值方式下的几何对象进行定义，而且也没有进行拓扑的定义。SQL/MM 标准则对圆弧插值下的几何对象进行了定义，并且还能够支持复合型的几何对象。如，SQL/MM 标准中已不再支持 Line 与 LinearRing 这两个几何对象。取而代之的则是增加了 ST_Circularstring、ST_CompoundCurve 和 ST_CurvePolygon 类型，并用他们来准确表达圆弧形成的几何对象。

(2) 空间操作对比分析

两个标准为各类几何对象定义的空间操作基本相同，但 SQL/MM 明显丰富于 SFASQL，两个标准中 Geometry 与 ST_Geometry 定义的空间操作对比如表 2.3 所示，限于论文篇幅，略去每种几何类型定义的空间操作对比。

表 2.3 两个标准中 Geometry 与 ST_Geometry 定义的空间操作对比表

分类	SFASQL Geometry	SQL/MM ST_Geometry	功能概述
基本操作	Dimension(): Integer	ST_Dimension(): SMALLINT	返回几何对象的维数, 小于或等于坐标维数。
		ST_CoordDim(): SMALLINT	
	GeometryType(): String	ST_GeometryType(): String	返回几何对象的类型, 如 'Point'、'ST_Point'
	SRID(): Integer	ST_SRID(): Integer	返回几何对象所属的空间参考系 ID
		ST_SRID(anSRid): ST_Geometry	
		ST_Transform(anSRid): ST_Geometry	返回转换到指定空间参考的几何对象。
	Envelope (): Geometry	ST_Envelope(): ST_Polygon	返回几何对象的最小外包矩形
	IsEmpty (): Integer	ST_IsEmpty(): Integer	如果几何对象为空集, 则返回 1。
	IsSimple (): Integer	ST_IsSimple(): Integer	如果几何对象是简单的 (不自交), 则返回 1。
		ST_IsValid(): Integer	Test if an ST_Geometry value is well formed.
	Is3D (): Integer	ST_Is3D(): Integer	如果几何对象含有 Z 值, 则返回 1。
	IsMeasured (): Integer	ST_IsMeasured(): Integer	如果几何对象含有 M 值, 则返回 1。
	Boundary (): Geometry	ST_Boundary(): ST_Geometry	返回几何对象的边界
空间关系检测运算	Equals(aGeom): Integer	ST_Equals(aGeom): Integer	如果两个几何对象的内部和边界在空间上都相等, 则返回 1。
	Disjoint (aGeom): Integer	ST_Disjoint (aGeom): Integer	如果两个几何对象的内部和边界在空间上都不相交, 则返回 1。
	Intersects(aGeom): Integer	ST_Intersects(aGeom): Integer	如果两个几何对象在空间上相交, 则返回 1。
	Touches(aGeom): Integer	ST_Touches(aGeom): Integer	如果两个几何对象边界相交但内部不相交, 则返回 1。
	Crosses(aGeom): Integer	ST_Crosses(aGeom): Integer	如果一条线和面的内部相交, 则返回 1。
	Within (aGeom): Integer	ST_Within (aGeom): Integer	如果这个几何对象空间上位于另一个几何对象内部, 则返回 1。
	Contains(aGeom): Integer	ST_Contains(aGeom): Integer	如果这个几何对象空间上包含另一个几何对象, 则返回 1。
	Overlaps(aGeom): Integer	ST_Overlaps(aGeom): Integer	如果几何对象内部有非空交集, 则返回 1。

续表 2.3 两个标准中 Geometry 与 ST_Geometry 定义的空间操作对比表

	Relate(aGeom, DE-9IMMatrix: String): Integer	ST_Relate(aGeom, DE-9IMMatrix: String): Integer	如果该几何对象与另一个几何对象的 DE-9IM 与 DE-9IMMatrix 相匹配, 则返回 1
	LocateAlong(mValue): Geometry	ST_LocateAlong(mValue): ST_Geometry	返回与给定的 M 值相匹配的几何集。
	LocateBetween(mStart, mEnd): Geometry	ST_LocateBetween(mStart, mEnd): ST_Geometry	返回与给定的 M 值范围相匹配的几何集。
分类	SFASQL Geometry	SQL/MM ST_Geometry	功能概述
空间分析运算	Distance (aGeom): Double	ST_Distance (aGeom): Double ST_Distance (aGeom, aunit): Double	返回两个几何对象间的最短距离。
	Buffer (adistance): Geometry	ST_Buffer(adistance): ST_Geometry	根据给定距离创建缓冲区多边形。
	-	ST_Buffer(adistance, aunit): ST_Geometry	根据给定距离及单位创建缓冲区多边形。
	ConvexHull(): Geometry	ST_ConvexHull(): ST_Geometry	返回几何对象的最小凸多边形。
	Intersection(aGeom): Geometry	ST_Intersection(aGeom): ST_Geometry	返回由两个几何对象的交集构成的几何对象。
	Union(aGeom): Geometry	ST_Union (aGeom): ST_Geometry	返回由两个几何对象的并集构成的几何对象。
	Difference (aGeom): Geometry	ST_Difference (aGeom): ST_Geometry	返回这个几何对象与另一个几何对象不相交的部分。
	SymDifference (aGeom): Geometry	ST_SymDifference (aGeom): ST_Geometry	返回两个几何对象与对方互不相交的部分。
	AsText(): String	ST_AsText(): String	将几何对象以 WKT 格式输出
	AsBinary(): Binary	ST_AsBinary()	将几何对象以 WKB 格式输出
		ST_AsGML()	将几何对象以 GML 格式输出
		ST_ToPoint(): ST_Point	将几何对象转换为 ST_Point
		ST_ToLineString(): ST_LineString	将几何对象转换为 ST_LineString
		ST_ToCircularString(): ST_CircularString	将几何对象转换为 ST_CircularString
		ST_ToCompoundCurve(): ST_CompoundCurve	将几何对象转换为 ST_CompoundCurve
		ST_ToCurvePolygon(): ST_CurvePolygon	将几何对象转换为 ST_CurvePolygon
		ST_ToPolygon(): ST_Polygon	将几何对象转换为 ST_Polygon
		ST_ToGeomCollection(): ST_GeomCollection	将几何对象转换为 ST_GeomCollection

续表 2.3 两个标准中 Geometry 与 ST_Geometry 定义的空间操作对比表

		ST_ToMultiPoint(): ST_MultiPoint	将几何对象转换为 ST_MultiPoint
		ST_ToMultiCurve(): ST_MultiCurve	将几何对象转换为 ST_MultiCurve
		ST_ToMultiLine(): ST_MultiLineString	将几何对象转换为 ST_MultiLineString
		ST_ToMultiSurface(): ST_MultiSurface	将几何对象转换为 ST_MultiSurface
		ST_ToMultiPolygon(): ST_MultiPolygon	将几何对象转换为 ST_MultiPolygon
		ST_WKTToSQL(awkt): ST_Geometry	将 WKT 表达的几何对象转换为相应的 ST_Geometry 类型
		ST_WKBToSQL(awkb): ST_Geometry	将 WKB 表达的几何对象转换为相应的 ST_Geometry 类型
		ST_GMLToSQL(agml): ST_Geometry	将 GML 表达的几何元素(如 LineString)转换为相应的 ST_Geometry 类型(如 ST_LineString)
		ST_GeomFromText(awkt): ST_Geometry ST_GeomFromText(awkt, ansrid): ST_Geometry	根据 WKT 表达创建几何对象 (Function)
		ST_GeomFromWKB(awkb): ST_Geometry ST_GeomFromWKB(awkt, ansrid): ST_Geometry	根据 WKB 表达创建几何对象 (Function)
		ST_GeomFromGML(agml): ST_Geometry ST_GeomFromGML(agml, ansrid): ST_Geometry	根据 GML 表达创建几何对象 (Function)

(3) 空间数据存储类型分析

空间数据库中的物理存储结构在对空间数据进行存储设计时,应当考虑到其与数据库扩展环境之间的关系。SFASQL 规范不但对预定义数据类型的扩展环境给予支持,还对 UDT 的扩展环境进行支持^[25]。SFASQL 规范中二进制类型、常规数值类型和被扩展的几何对象类型。而二进制类型与常规数值类型属于预定义数据类型,扩展的几何对象类型则属于 UDT。

大型关系型的数据库厂商在进行空间数据库的扩展时更倾向于选择对象类这种方式来存储空间数据,而 GIS 厂商却不同。它们有两种方式来对空间数据进行存储,一是预定义数据类型,一是对象类。选用预定义这种常规的方法来存储空间数据会出现效率低下的问题。因为这种方式在存储空间数据时,因为要对额外的信息进行维护,这样就

要涉及到关系运算、表操作以及可变长的空间数据管理等这些操作。而二进制的存储方式，虽然简化了可变数据的长度，却有出现了读写效率慢的问题。对象类目前主流的存储方式，它充分发挥了面向对象的思想，不但解决了可变长的记录管理问题，同时还简化了关系表之间的结构，并且使数据库的维护变得更加的容易，效率也提升了。

2.3 空间数据的完整性约束

2.3.1 空间数据完整性约束概述

空间数据完整性约束指为了保护数据的正确性、有效性、相容性，并防止错误数据入库的一组规则。它不但定义了数据模型的语义约束同时还定义了数据之间关系所需满足的语义约束。数据库系统是必须要遵守完整性约束规则的，因为它是用来保证数据的正确性与相容性的。

SDBMS 必须提供的完整性约束规则：

- (1) 定义完整性约束条件的机制；一般由 SQL 的 DDL 语句来实现。
- (2) 检查完整性约束条件的方法；一般在 INSERT、UPDATE、DELETE、语句执行后开始检查，也可以在事务提交时检查。
- (3) 用户违反完整性约束条件的解决措施；如应拒绝该用户继续操作，来保证数据的完整性。

SDBMS 应该提供的完整性类型包括：

- (1) 实体完整性(Entity Integrity)
- (2) 参照完整性(Referential Integrity)
- (3) 域完整性(Domain Integrity)
- (4) 专题语义完整性(Thematic Semantic Integrity)
- (5) 时态语义完整性(Temporal Semantic Integrity)
- (6) 空间语义完整性(Spatial Semantic Integrity)
- (7) 关系类完整性约束(Relationship Integrity)
- (8) 组合语义完整性约束(Combination Semantic Integrity)
- (9) 空间数据多版本约束(Multiversions Constraint)
- (10) 触发器(Trigger)

2.3.2 实体完整性约束

空间数据库中，基本关系(要素类)与现实世界的实体(要素)集是一一对应的关系。一个基本关系(如一个要素类)通常对应现实世界的一个实体(要素)集。例如地籍管理系

统中宗地要素类(Parcels)是宗地要素的集合。现实世界中的实体能够应用唯一性标识来进行区分。与此相对应,在关系模型中则以主键作为唯一性标识来进行区分。当然,作为唯一性标识的主键(PRIMARY KEY)的值不能为空,也不可重复。只有这样才能确保实体的完整性约束。要素类中的 ObjectID 字段的取值可用于区分地理要素,可以作为主键,当然也可以定义其它字段或字段组合作为主键。

关系模型中的实体完整性采用唯一标识 PRIMARY KEY 进行定义如:

```
CREATE TABLE ADMIN.BLOCKS
(
    OBJECTID NUMBER NOT NULL ,
    RES NUMBER(5),
    BLOCK_ID NUMBER,
    SHAPE SDE.ST_GEOMETRY,
    PRIMARY KEY (BLOCK_ID)
);
```

以上 SQL 语句创建一要素类 BLOCKS(街区、区块),并通过 PRIMARY KEY 指定 BLOCK_ID 为主关键字,ADMIN 为一用户名。

```
CREATE TABLE ADMIN.PARCELS
(
    OBJECTID NUMBER NOT NULL ,
    PROPERTY_I NUMBER(38, 8),
    LANDUSE_CO NVARCHAR2(3),
    ZONING NVARCHAR2(6),
    PARCEL_ID NUMBER(10),
    RES NUMBER(5),
    ZONING_S NVARCHAR2(4),
    BLOCKID NUMBER,
    SHAPE SDE.ST_GEOMETRY,
    PRIMARY KEY (PARCEL_ID),
    FOREIGN KEY (BLOCKID) REFERENCES ADMIN.BLOCKS (BLOCK_ID)
);
```

以上 SQL 语句创建一要素类 Parcels(宗地),并通过 PRIMARY KEY 指定 "PARCEL_ID"为主关键字。

实体完整性的检查与违约的处理,包括:主键值是否唯一,若不唯一则拒绝执行更新等操作;若主键的各属性为空,则不会进行下面的更新操作。

2.3.3 参照完整性约束

参照完整性约束是被用来保证两关系间引用的一致性。在 CREATE TABLE 中一般用 FOREIGN KEY 来指明哪一个或是那几个为外键。与此同时,还应用 REFERENCES 来表明这些外键对那几个表的主键进行了参照。如上面 CREATE TABLE

"ADMIN"."PARCELS"中的:

FOREIGN KEY ("BLOCKID") REFERENCES "ADMIN"."BLOCKS" ("BLOCK_ID")
指明 BLOCKID 字段是要素类 PARCELS 的外键,它引用要素类 BLOCKS 中的主关键字 BLOCK_ID;此时的 PARCELS 表被称为从表(子表),而 BLOCKS 表则被称为主表(父表)。

要素类 Parcels 中的一条记录表示一块宗地,其中 BLOCKID 字段的值为宗地所在的街区编号,它引用 BLOCKS 中的 BLOCK_ID。

因为参照完整性会把两个表相关联,所以在对这两个表进行更改操作时,是十分有可能造成参照完整性的破坏的。这时如遇到参照完整性被破坏的情况,可以采用以下三种方法来进行处理:

(1) 拒绝(reject)

不允许执行该操作,该策略一般设置为默认策略。

(2) 级联删除或更新(cascade)

当对父表的一个元组进行了更新操作而造成两表之间参照完整性不一致时,应当对子表中不一致的元组进行删除或是更新的操作,来保证两表间的参照完整性的一致。

(3) 设置为空值(set-null)

当父表的一个元组与子表对应的元组不一致造成了参照完整性的不一致时,可以将子表中相应的元组的属性设置为空。这样就能够保证两表间参照完整的一致性了。具体的处理策略如下所示:

```
CREATE TABLE ADMIN.PARCELS
(
    OBJECTID NUMBER NOT NULL ,
    PROPERTY_I NUMBER(38, 8),
    LANDUSE_CO NVARCHAR2(3),
    ZONING NVARCHAR2(6),
    PARCEL_ID NUMBER(10),
    RES NUMBER(5),
    ZONING_S NVARCHAR2(4),
    BLOCKID NUMBER,
    SHAPE SDE.ST_GEOMETRY,
    PRIMARY KEY (PARCEL_ID),
    FOREIGN KEY (BLOCKID) REFERENCES ADMIN.BLOCKS (BLOCK_ID)
        ON DELETE CASCADE /*级联删除 Parcels 表中相应的元组*/
        ON UPDATE CASCADE /*级联更新 Parcels 表中相应的元组*/
);
```

2.3.4 域完整性约束

空间数据库中的表由一系列字段定义,每个字段都有相关联的数据类型。域完整性是值字段的表约束,它包含了对字段类型、字段值域、有效规则以及是否可为空等约束。

表中字段取值的约束，它包括字段的类型、字段的值域、字段的有效规则、是否允许空值等约束。

(1) 数据类型约束

a) 常规数据类型约束

在 ArcGIS 中创建表或要素类时，提供了相应的数据类型：Short Integer、Long Integer、Float、Double、Text、Date、Blob、Guid、Raster；各 DBMS 也提供了常用的数据类型。在创建数据库模式时，通过数据类型来限制相应字段上的取值。此外，不同的数据类型具有不同的运算法则，如常规的加、减、乘、除运算只对数值型数据有效，对其它数据类型一般是无效的。

b) 空间数据类型约束

为了存储管理空间数据，ArcGIS、各 DBMS 都提供相应的空间数据类型。几何对象模型及相应的空间操作、函数的相关内容详见本章上一节。如果空间数据库中建立要素类，应当指明它的几何类型，并指明几何字段的完整性约束。这样做的原因是为了保证几何数据的几何完整性约束。

几何完整性约束有两种，一是对几何类型进行的约束；而是对几何形状进行的约束。几何类型约束与常规数据类型约束类似，点要素只能存储 Point 几何形状，而不能存储 Polyline 或 Polygon，要素类型应该与几何类型匹配。

OGC SFA 对几何形状的约束做了如下的规定：

Point 的几何形状没有任何约束，其坐标构成可以是(X, Y)、(X, Y, Z)、(X, Y, M)或(X, Y, Z, M)。

MultiPoint 的几何形状约束是不能有重复的点(即不能存在坐标相同的点)。

LineString 的几何形状约束是必须由直线段构成，且不能存在自相交的现象。

MultiLineString 的几何形状约束是必须由直线段构成，且每一部分及部分各之间不能存在自相交的现象。

ArcGIS 中的 Polyline 涵盖了 OGC SFA 中的 LineString 和 MultiLineString，并且线段类型更丰富。

Polygon 的几何形状约束：Polygon 首先是平面且一定是闭合的，它由 1 个外环(如图 2.9 d)不满足)和 0 到多个内环组成，环自身不能存在自相交，环与环之间不能相交(但允许相切交于一点，不能多点相切相交，如图 2.9 a)、b)不满足)或重叠，多边形的内环不能相连；多边形不能有悬挂的线(如图 2.9 c)不满足)；每个内环定义则代表了一个面内的洞；外环特征点坐标以逆时针的顺序进行排列，内环特征点坐标以顺时针的顺序进行排列。满足、不满足几何形状约束的多边形示例分别如图 2.8、2.9 所示。

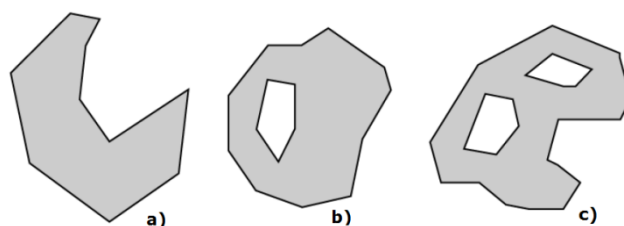


图 2.8 多边形示例，a)、b)、c)分别由 1、2、3 个环组成

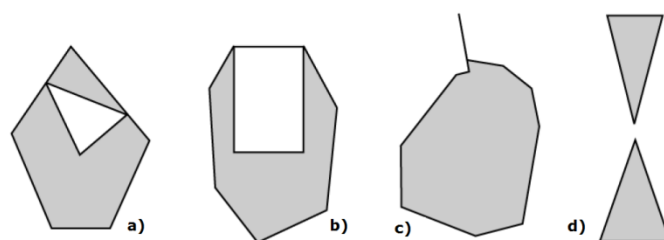


图 2.9 不满足几何形状约束的多边形示例

PolyhedralSurface 的几何形状约束：PolyhedralSurface 是连续的多边形的集合，多边形共享公共边界，一条边界最多被两个多边形所共享，多边形之间不能存在缝隙或交叉重叠。而在对相邻多边形的公共边进行遍历时，应当采取方向相反的方式，如图 2.10 所示。

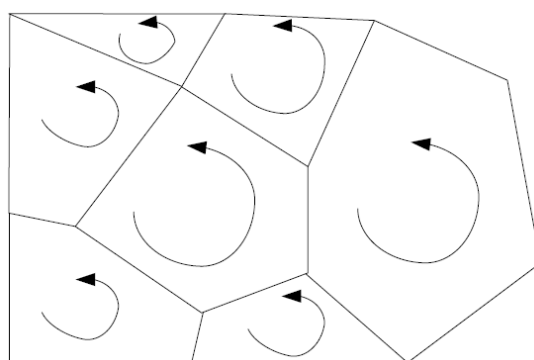


图 2.10 满足几何约束的 PolyhedralSurface 示例

MultiPolygon 的几何形状约束：首先应该是拓扑闭合的；任意两个多边形内部之间不可以相交，如图 2.12 c)不满足；任意两个多边形的边界不能穿越(cross，如图 2.12 c)不满足)、但可以在有限点上相接(touch，如图 2.11 c)满足，图 2.12 a)不满足)；不可以有悬挂的线段，如图 2.12 b)不满足；多边形的内环不能再镶嵌内环，如图 2.11 d)应该理解为由 2 个多边形组成的 MultiPolygon。满足、不满足几何形状约束的多边形示例分别如图 2.11、2.12 所示。

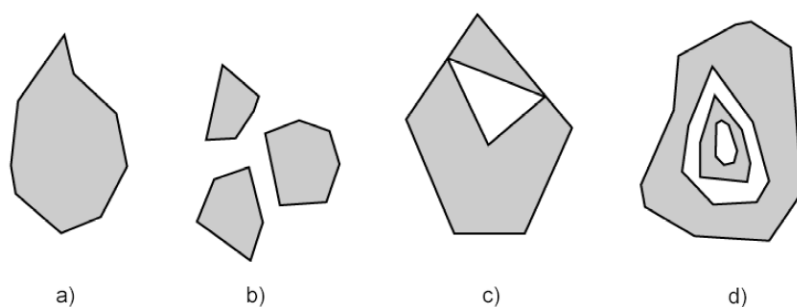


图 2.11 MultiPolygon 示例，a)、b)、c)、d)分别由 1、3、2、2 个多边形元素组成

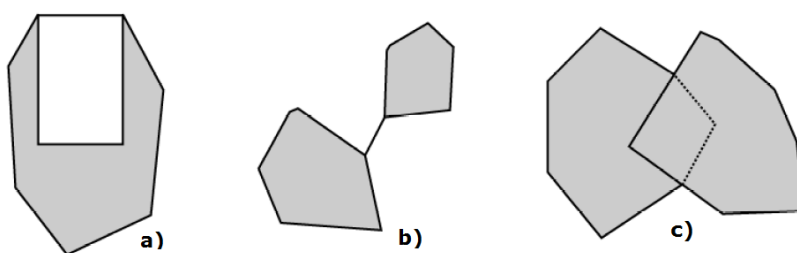


图 2.12 不满足几何约束的 MultiPolygon 示例

ArcGIS 中的 Polygon 涵盖了 OGC SFA 中的 Polygon 和 MultiPolygon，并且可以环内套环，镶嵌的深度也无限制。

(2) 字段取值约束

(a) 非空约束(NOT NULL)

非空约束是指在某一字段后面标注了 NOT NULL 这一关键字就表明这个字段的该属性不可以为空值。具体如下：

```
CREATE TABLE ADMIN.Owners
(
    OBJECTID NUMBER NOT NULL ,    /*要求 OBJECTID 列值不能为空*/
    PROPERTY_ID NUMBER(38, 8),
    OWNER_NAME NVARCHAR2(30),
    OWNER_PERCENT NUMBER(38, 8),
    DEED_DATE NVARCHAR2(20),
    OWN_ID NUMBER(38, 8) NOT NULL , UNIQUE (OWN_ID) VALIDATE
    /*要求 OWN_ID 列值不能为空，且必须唯一*/
)
```

(b) 列值唯一约束(UNIQUE)

唯一约束是指在某一字段后面标注了 UNIQUE 这一关键字就表明这个字段的该属性只能取唯一的一个值，并且这个值不能是空值。如上面 CREATE TABLE ADMIN.Owners 中的“UNIQUE (OWN_ID)”指明 Owners 表中 OWN_ID 字段取值必须唯一。

(a) 用 CHECK 短语指定列值应该满足的条件

CHECK 子句是确保某些属性值能够满足约束的先决条件。

其一般格式为：

CHECK(<条件>)

如：

```
CREATE TABLE Student
(
    Sno CHAR(9) PRIMARY KEY,
    Sname CHAR(8) NOT NULL,
    Ssex CHAR(2) CHECK (Ssex IN ('男', '女')), /*性别 Ssex 只允许取'男'或'女'*/
    Sage SMALLINT,
    Sdept CHAR(20)
);
```

(c) 完整性约束命名子句

完整性约束命名子句是指用 CONSTRAINT 关键字对完整性约束的条件进行命名。

有了这样的管理方式就能够非常便利的对完整性约束的条件进行添加或是删除了。

格式：

CONSTRAINT <完整性约束条件名>

[PRIMARY KEY 短语 |FOREIGN KEY 短语 |CHECK 短语]

如：

```
CREATE TABLE Student
(Sno NUMERIC(6) CONSTRAINT C1 CHECK (Sno BETWEEN 90000 AND 99999),
Sname CHAR(20) CONSTRAINT C2 NOT NULL,
Sage NUMERIC(3) CONSTRAINT C3 CHECK (Sage < 30),
Ssex CHAR(2) CONSTRAINT C4 CHECK (Ssex IN ('男', '女')),
CONSTRAINT StudentKey PRIMARY KEY(Sno) );
```

在 Student 表上共创建了 5 个约束条件，包括主键约束以及四个列级约束。

(d) 属性域

属性域(Domains)用来定义字段类型的合法值。是对表、要素类、子类型等的属性字段的值进行限制。Geodatabase 中的要素类与表可以共享属性域中不同的属性域子类型。Geodatabase 有两种不同的属性域：范围域(Range Domains)和编码值域(Coded Value Domains)。范围域是数值属性指定值的有效范围，通过最小值、最大值定义；编码值域给一个属性指定有效的取值集合。

2.3.5 专题语义完整性约束

专题语义完整性约束(Thematic Semantic Integrity Constraints)用于确保专题属性的一致性。一方面，它通过指定两个或多个字段值之间的关系、或一个字段值与一个已定义的其他值之间的关系来限制单个实体的属性范围，例如，假设道路有道路类型(road

type)和车道数(number of driving lanes)两个属性,那么道路类型为高速公路的道路至少有 4 条车道。又如:能通航的河流水深一定要达到规定的最小深度;楼房的层数一定大于 1;地块的面积必须大于 0 等。另一方面,专题语义完整性约束还用于确保地理实体之间属性一致性,例如,赣州、吉安、九江等属于江西省的地级市,不能出现“赣州是广东省的地级市”,否则属性语义不一致;工业用地上不能出现居民住宅,陆地地块不能位于水体区域,等等。

2.3.6 时态语义完整性约束

时态语义完整性约束(Temporal Semantic Integrity Constraints)用于确保时态数据的逻辑一致性,使时态数据符合现实世界的语义。时态数据中有以下几种基本数据类型:时刻(Time Instant)、时间间隔(Time Interval)与时长(Time Period)。复杂的时态语义完整性约束涉及象事件、过程、状态、动作等特定时间点或时间间隔之间的关系。例如:一座桥梁修建时间、通车时间 2 个时态属性,其隐含的时态语义:桥梁修建时间必须在通车时间之前。时态语义完整性的约束是用来检查是否存在不合理的数据,如“桥梁修建时间在通车时间之后”。

时间间隔通过两个有序的时间点来描述现实世界中的事件,在时间坐标轴上第一个时间点位于第二个时间点之前。对于一个给定的时间间隔 I_n ,用 I_{n-} 、 I_{n+} 分别表示该时间间隔的开始时间点、结束时间点,那么任意两个时间间隔 I_1 、 I_2 之间存在以下关系:

$$I_1 \text{ before } I_2 : I_{1+} < I_{2-}$$

$$I_1 \text{ meets } I_2 : I_{1+} = I_{2-}$$

$$I_1 \text{ overlaps } I_2 : (I_{2-} > I_{1-}) \wedge (I_{1+} < I_{2+}) \wedge (I_{1+} > I_{2-})$$

$$I_1 \text{ starts } I_2 : (I_{1-} = I_{2-}) \wedge (I_{1+} < I_{2+})$$

$$I_1 \text{ finishes } I_2 : (I_{1+} = I_{2+}) \wedge (I_{1-} > I_{2-})$$

$$I_1 \text{ during } I_2 : (I_{1-} > I_{2-}) \wedge (I_{1+} < I_{2+})$$

$$I_1 \text{ equal } I_2 : (I_{1-} = I_{2-}) \wedge (I_{1+} = I_{2+})$$

以上时态关系在不同领域有着广泛应用,且已成为 ISO/TC211 标准的一部分。在时态语义完整性约束实现中,可以根据实际问题的现实语义,利用上述关系检测时态数据是否符合实际语义。

2.3.7 空间语义完整性约束

空间语义完整性约束(Spatial Semantic Integrity Constraints)是指用于限制地理实体

的空间布局、地理实体的空间属性及地理实体之间的空间关系。空间实体之间的空间关系一般是指下列关系：拓扑关系、方位关系和度量关系。

(1) 拓扑语义完整性约束

拓扑属性是指几何对象在拓扑变换时能够保证空间属性不发生变化的特征。拓扑是一个定性的概念，它不依赖于任何定量度量值(如坐标、长度、面积等)，拓扑只关心地理实体之间的连接或非连接特性。

OGC SFA 规范中定义了 DE-9IM 中的八种空间拓扑关系：Equal(相等)、Disjoint(相离)、Intersect(相交)、Touch(相接)、Cross(穿越)、Within(在里面)、Contains(包含)和 Overlap(有重叠)。基于维数扩展的九交模型，通过计算两个几何对象之间的维数来判断两个几何对象之间的拓扑关系。交集的维数可以 0(点)、1(线)、2(多边形)或-1(空集)。

不是所有的拓扑关系对所有的几何类型都有效，OGC SFA 规范对拓扑语义约束作了规定。简述如下：

设 P 为点对象，L 为线对象，A 为面对象，mP 为多点，mL 为多线，mA 为多面。给定两个几何对象 a，b，I(a)，B(a)和 E(a)分别表示 a 的内部、边界和外部，dim(a)表示几何对象的维数。常用拓扑关系定义及约束如下：

Equals: $a.Equals(b) \Leftrightarrow (a \subseteq b) \wedge (b \subseteq a)$ ，a、b 只能为相同类型的几何对象。

Disjoint: $a.Disjoint(b) \Leftrightarrow a \cap b = \emptyset$ ，a、b 可以是任何类型的几何对象。

Intersects: $a.Intersects(b) \Leftrightarrow !a.Disjoint(b)$

Touches: $a.Touch(b) \Leftrightarrow (I(a) \cap I(b) = \emptyset) \wedge (a \cap b \neq \emptyset)$ ，a、b 的几何类型为 A/A、L/L、L/A、P/A 及 P/L，如图 2.13 所示。

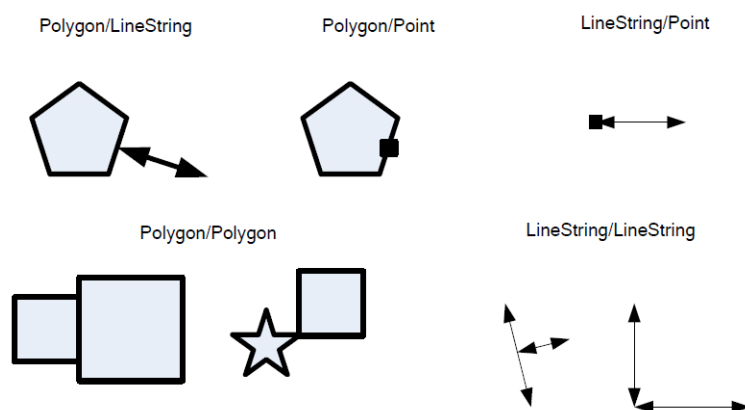


图 2.13 Touches 关系示例

Crosses: $a.Cross(b) \Leftrightarrow (I(a) \cap I(b) \neq \emptyset) \wedge (a \cap b \neq a) \wedge (a \cap b \neq b)$ ，a、b 的几何类型为 P/L、P/A、L/L 和 L/A。

Within: $a.Within(b) \Leftrightarrow (a \cap b = a) \wedge (I(a) \cap E(b) = \emptyset)$ ，a、b 的几何类型为 A/A、L/A、P/A、和 L/L，图 2.14 所示。

Contains: $a.Contains(b) \Leftrightarrow b.Within(a)$

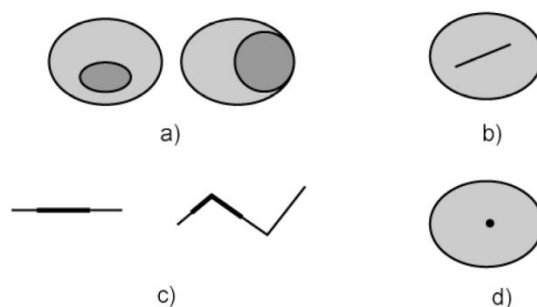


图 2.14 Within 关系示例

Overlaps: $a.Overlaps(b) \Leftrightarrow (dim(I(a))=dim(I(b))=dim(I(a) \cap I(b))) \wedge (a \cap b \neq a) \wedge (a \cap b \neq b)$,
a、b 的几何类型为 A/A、L/L 和 P/P.

此外，还有一个通用的关系检测操作 Relate: $a.Relate(b, DE-9IM\ matrix)$ ，该操作直接使用某种拓扑关系对应的九交矩阵来检测两个几何对象之间是否满足该种类型的拓扑关系，如 Within 对应的九交矩阵“T*F**F***”，Overlaps 对应的九交矩阵“T*T***T**”(A/A、P/P)或“1*T***T**”(L/L)。

拓扑语义完整性约束(Topological Semantic Integrity Constraints)用于约束两个几何形状之间应该满足某种空间拓扑关系，以确保空间数据符合拓扑语义。例如，多边形公共边界的一致性、边线公共结点的一致性；对多边形边界的修改，与边界相邻的多边形必须同步更新；等高线不能相交；宗地之间不能有重叠、也不能有缝隙(出现飞地)；河流不能与道路相交，除非通过桥梁或隧道；道路不能穿越房屋，住宅类型的房屋必须位于住宅类型的宗地上，等等。

拓扑语义完整性约束还包括地理网络完整性约束，地理网络有定向和非定向两种类型；例如，排水网络、供电网络等就是定向网络(几何网络)；道路、交通网络就是非定向网络。网络由一系列的结点(Nodes)或交汇点(Junctions)和边线(Edges)两种元素连接而成，结点、边线之间的相互连接应满足一定的连通规则。

Geodatabase 通过一系列拓扑规则(如：must not overlap、must be covered by 等)和拓扑工具来确保空间数据的拓扑完整性。在设计 Geodatabase 时，用户能够指出空间数据应该遵循的拓扑规则，如：相邻关系、连接关系、覆盖关系、相交关系、重叠关系等。所有这些关系都对应相应的规则。在数据编辑时，使用拓扑编辑工具，确保要素几何的完整性。

(2) 方位语义完整性约束

方位语义完整性约束(Directional Semantic Integrity Constraints)是指地理实体之间的方位关系约束。通常来说，是根据能否使用固定坐标系，方位关系有两种类型：基于地理坐标系的东南西北方位关系(如图 2.15 a)、b))和前后左右方位关系(如图 2.15 c))。

基于地理坐标系的东南西北方位关系又分为基于锥形划分的方位关系(如图 2.15 a))和基于投影的方位关系(如图 2.15 b))，如房屋的正面朝南，房屋的北面有一颗树，房屋的西南角有一个池塘。

在方位关系判断中，几何对象通常用它的最小外包矩形 MBR 近似表达，MBR 用其左下角 P_{LL} 和右上角 P_{UR} 表达，那么可以通过比较两个几何对象 A、B 的 MBR 的 P_{LL} 、 P_{UR} 的 X、Y 的大小来确定它们之间的方位关系。例如，如果 $Y_{P_{LL}}^A > Y_{P_{UR}}^B$ ，则几何对象 A 位于几何对象 B 的强北(Strong North)方向；如果 $(Y_{P_{UR}}^A > Y_{P_{UR}}^B) \text{ and } (Y_{P_{LL}}^B < Y_{P_{LL}}^A < Y_{P_{UR}}^B)$ ，则几何对象 A 是在几何对象 B 的弱北(Weak North)方向上；其它方位关系可类似判断。

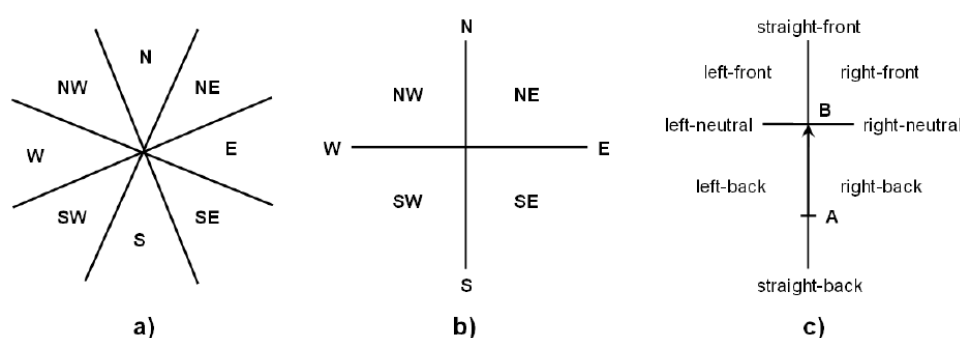


图 2.15 方位关系示例：a)、b)分别为锥形、基于投影的主方位关系，c)为前后左右方位关系

前后左右方位关系不需要固定的坐标系，而是针对某个特定的地理实体，描述其前后、左右的地理实体的分布，如房屋的后面是花园，前面是运动场，左边是菜地，右边是商店。方位关系还有平行、垂直关系，例如，道路指示牌应该朝向行车方向，城市地下管线应该沿街道布设，天桥应该与道路垂直。

(3) 度量语义完整性约束

度量属性是基于地理实体之间的距离的，相应的度量语义完整性约束(Metric Semantic Integrity Constraints)用于限制距离和大小。线几何对象有长度度量属性，多边形几何对象有周长、面积等度量属性，相应的几何对象上有计算长度、面积的方法，这些方法可以用来限制线要素的最小长度或面要素的最大面积，且长度、面积不能为负值。还有，空间数据编辑过程中分割、合并操作，应确保相应的度量一致性；如对一地块进行分割，分割后的地块面积之和应该等于原始地块的面积。

度量语义完整性约束还用于限制地理实体之间的距离，例如，加油站离学校的距离不能小于 500 米；核电站距居住区的距离不得小于 10 千米；为了交通便利，高速公路离城市距离不能超过 5 千米；排水管线的埋设需要满足一定的坡度。

2.3.8 关系类完整性约束

空间数据库中的实体之间可以相互关联，ArcGIS 提供了多种方法来将 Geodatabase

中的记录关联在一起，如关系类(Relationshipclass)、关联(Relate)、连接(Join)、拓扑(Topology)、网络(Network)。建立地理要素之间的关系时，首先考虑的是对要素之间的空间关系进行建模，如拓扑模型、网络模型等。

关系类是指管理一个类中的对象与另一个类中对象的关系。在关系的任何一端的对象可以是地理要素，或是表中的记录。对关系类进行完整性的约束，为了确保对象间的引用一致性。这样就能够在修改某一对象的同时进行其相关的对象的修改。例如，铁支杆支持 A 类变压器，木支杆支持 B 变压器，此外，还可以指定关系基数范围，如一个铁支杆能支持 0—3 个 A 类变压器，而一个木支杆只能支持 0—2 个 B 类变压器。

在几何网络中，网络连通性规则的功能是规定相连的网络要素类型及数量。定义这些规则就是为了保证空间库中网络数据的完整性。通过这些规则就能够随时进行无效要素的检查与处理。

2.3.9 组合语义完整性约束

组合语义完整性约束是由以上多个语义完整性约束组合而成的约束，在一个约束中限制多个语义领域的关系或属性。例如，如果管道直径大于 40cm，蝶形阀不能与该管道相连；这一约束组合了拓扑约束和属性约束，拓扑约束为拓扑连接(Connectivity)或拓扑相交(Intersects)，即蝶形阀与管道连接或相交；属性约束是管道直径小于等于 40cm，才能够与蝶形阀连接或相交。

组合语义完整性约束还可以由部分(parts)-整体(whole)关系和属性关联组合而成，例如，一个省的居住人口数是其所有行政管辖区居住人口的总和，这个例子涉及行政管辖区和省两种类型的地理实体，它们之间的关系是 partOf 的关系；这个例子还显示了部分整体关系与拓扑关系有一定的联系，行政管辖区与省之间存在包含与被包含的拓扑关系。

2.3.10 空间数据多版本约束

空间数据库提供了对空间数据的长事务进行处理以及控制多用户并发访问的控制机制，即版本机制。这个机制能够使多用户对同一数据进行编辑等操作。当两个编辑用户在同一版本或不同版本中对同一数据进行操作可能会产生冲突。空间数据库将以已制定的版本约束为蓝本，进行冲突的检测并给出适宜的解决方案。

2.3.11 触发器约束

目前很多商用 DBMS 都支持触发器。触发器又叫做事件--条件--动作规则，是当特定的系统事件(对一个表的增、删、改操作)发生时，首先进行规则条件的检测，若符合条件，则进行下面的操作；若不符合条件，将不再进行下面的操作。具体实例如下代码所示：

```
create table account          /*银行账户表 */
(
    customerName varchar2(30) primary key,
    cardID varchar2(8),
    currentMoney number
);
insert into account values('兰天','10010001',5000);
insert into account values('兰德','10010002',3000);

create table trans            /*银行交易(存款取款)表 */
(
    transDate date,
    cardID varchar2(8),
    transType varchar2(10),    /*存款或取款 */
    transMoney number
);
insert into trans values(sysdate,'10010001','取款',1000);

create or replace trigger trans_trigger
before insert
on trans
for each row
declare
    v_currentMoney account.currentMoney%type;
begin
    --判断类型
    if :new.transType='取款' then
        --取款
        select currentMoney into v_currentMoney
        from account
        where cardID=:new.cardID;

        if v_currentMoney < :new.transMoney then
            raise_application_error(-20001,'余额不足');
        end if;

        update account
        set currentMoney=currentMoney-:new.transMoney
        where cardID=:new.cardID;
    else
        --存款
        update account
        set currentMoney=currentMoney+:new.transMoney
        where cardID=:new.cardID;
    end if;
exception
    when no_data_found then
```

```

        raise_application_error(-20002,'无效的帐户');
    end;

```

在空间数据管理中，可以应用触发器来保证多表之间空间数据的一致性。如 SDE 模式中的 TG_GCOL_NAME 触发器：

```

CREATE OR REPLACE TRIGGER "SDE"."TG_GCOL_NAME"
BEFORE INSERT OR UPDATE OF
"COLUMN_NAME", "GEOMETRY_TYPE", "OWNER", "TABLE_NAME"
ON "SDE"."ST_GEOMETRY_COLUMNS"
REFERENCING OLD AS OLD NEW AS NEW
FOR EACH ROW
DECLARE
    CURSOR select_seq IS SELECT ST_GEOM_ID_SEQ.nextval FROM dual;
    SEQ_O NUMBER;
    INVALID_SEQ EXCEPTION;
BEGIN
    :NEW.OWNER := UPPER(:NEW.OWNER);
    :NEW.TABLE_NAME := UPPER(:NEW.TABLE_NAME);
    :NEW.COLUMN_NAME := UPPER(:NEW.COLUMN_NAME);
    :NEW.GEOMETRY_TYPE := UPPER(:NEW.GEOMETRY_TYPE);

    IF INSERTING THEN
        OPEN select_seq;
        FETCH select_seq INTO SEQ_O;
        IF select_seq%NOTFOUND THEN
            RAISE INVALID_SEQ;
        ELSE
            CLOSE select_seq;
        END IF;
        :NEW.GEOM_ID := SEQ_O;
    END IF;
    EXCEPTION
    WHEN          INVALID_SEQ          THEN          raise_application_error
(-20005,'ST_GEOMETRY_COLUMNS sequence ST_GEOM_ID_SEQ not found. ');
    CLOSE select_seq;
END;

```

该触发器的作用是：当用户创建或更新一个存储类型为 ST_Geometry 类型的要素类时，自动调用该触发器在 ST_GEOMETRY_COLUMNS 系统表中记录 OWNER、TABLE_NAME、COLUMN_NAME、GEOMETRY_TYPE、GEOM_ID 等相关信息，创建时要获得 GEOM_ID 的值。

当然，用户也可以根据 GIS 工程项目的实际需要，创建自己的用于空间数据管理和维护的触发器。

第三章 基于 PostGIS 的重点流通企业空间数据库设计

3.1 平台概述

3.1.1 PostgreSQL 的概述

PostgreSQL 是一个开源的、社区驱动的、符合标准的对象-关系型数据库管理系统，它不仅支持关系数据库的各种功能，而且还具备类、继承等对象数据库的特征。它具有强大的功能、复杂的结构以及丰富的特性。有些功能特性甚至连商业 DBMS 都没有。它曾是加州大学伯克利分校的一个数据库研究计划，而今却以成为数据库产品中的领导者，不但被人们所熟识，还拥有一些忠实的用户群。

PostgreSQL 在性能上丝毫不逊于任何的大型商业数据库产品，并且它还提供了非常丰富的接口库，用于满足用户不同需求的开发。它能够完美的支持 SQL 标准、并拥有如异步复制、预写日志容错技术以及多版本等众多的功能。而且，在大数据背景下的今天，PostgreSQL 对于大数据的管理已表现出了自己的特点。同时，PostgreSQL 支持地理空间数据的管理。PostgreSQL 中已经定义了一些基本的几何类型，如：点(POINT)、线(LINE)、线段(LSEG)、方形(BOX)、路径(Path)、多边形(POLYGON)和圆(CIRCLE)；另外，PostgreSQL 定义了一系列的函数和操作符来实现几何类型的操作和运算；同时，PostgreSQL 引入空间数据索引 R-tree。

(1) PostgreSQL 的物理结构

默认情况下，PostgreSQL 中的数据文件将被存放于指定 PGDATA 的 data 目录里。Data 文件夹里所保存的是数据集簇的配置文件与一些其他的子目录：

PG-VERSION：是用于存储 PostgreSQL 版本号的文件目录。如，安装的为 PostgreSQL9.2 版本，则文件中所记录的即为 9.2。

postmaster.opts：指服务器前一次启动时使用过的参数文件，如
E:/PostgreSQL/9.2/bin/postgres.exe "-D" "E:/PostgreSQL/9.2/data。

postmaster.pid：它是一个锁文件，是被用来记录着当前守护进程 Postmaster 的进程号与共享内存段 id 的^[24]。但它并不是一个永久性文件，而是一个临时性的文件。如，当关闭服务器以后，这个文件将被删除。

postgresql.conf：主要配置文件，除基于主机的访问控制和用户名映射之外的其他用户可设置参数都保存在这个文件中。

pg_hba.conf：是对于主机访问的控制文件；当用户访问主机时，它将把用户的认证

信息保存起来。

pg_ident.conf: 是用户名的映射文件; 这个映射文件, 定义了 OS 的用户名与 PostgreSQL 用户名之间的对应关系, 而这些用户名之间的对应关系将被 **pg_hba.conf** 用到。

base (目录): 它是一个包含了所有数据库的目录; 数据库目录是以数据库的 OID 进行编号命名的, 其中名为 1 的目录对应模板数据库 **template1**, 同时它还对应着 **pg_default** 的这个表空间。

Global (目录): 它是一个全局表。用来保存所有集簇的共享信息。例如, **pg_database** 它就对应于 **pg_global** 这个表空间。

pg_clog (目录): 它是一个子目录; 被用来保存事务提交以后的所有状态数据。

pg_log (目录): 包含 PostgreSQL 的日志文件, 这个日志一般是记录服务器与数据库的状态, 比如各种 **Error** 信息, 定位慢查询 **SQL**, 数据库的启动关闭信息, 发生 **checkpoint** 过于频繁等的告警信息等。

pg_stat_tmp (目录): 它是一个用于统计子系统需要的所有临时文件的子目录。

pg_subtrans (目录): 它是一个子目录; 用来储存子事务的所有状态的数据信息。

pg_tblspc (目录): 它是一个子目录; 用于描述表空间符号的链接关系。这里的表空间是指用户自己创建的表空间。

pg_twophase (目录): 它是一个子目录; 用于存储预备事务状态的文件信息。

pg_xlog (目录): 它是一个子目录; 用于存储 **WAL** 的文件。

pg_notify (目录): 它是一个子目录; 用于存储发往信息通道的异步消息。

pg_serial (目录): 此文件夹为空。

(2) PostgreSQL 的系统结构与内存结构

PostgreSQL 中有两个进程, **Postmaster** 与 **Postgres**。**Postmaster** 是守护进程而 **Postgres** 是服务进程。守护进程 **Postmaster** 是用于接收客户端信息的, 并且再接收了客户端的信息之后, 又为客户端与服务进程 **Postgres** 建立了链接。一旦客户连接一个服务进程 **Postgres** 后, 客户将直接与服务进程交互不在通过守护进程。当 **Postmaster** 进程为用户的请求建立了一个监听的端口后, 它将开启以下的辅助进程。系统日志进程 **SysLogger**、统计数据收集进程 **PgStat**、系统自动清理进程 **AutoVacuum**。当用户的请求需要进入循环模式时, 守护进程 **Postmaster** 将会启动循环监听进程, 并启动以下几个辅助进程。后台写进程 **BgWriter**、预写式日志进程 **WalWriter**、预写式日志归档进程 **PgArch**^[26]。服务进程 **Postgres** 才是真正接收用户请求的服务模块。它将直接接受用户的命令并对命令进行编译后执行, 然后再返回结果给用户。用户的命令分为查询命令和非查询命令。查询命令, 即插入、删除、更新和选择等命令; 非查询命令, 则是如创建或删除表、索引或视图等命令; **Postgres** 还可以根据用户的不同命令来进行处理策略的选择。

SysLogger: 系统日志进程; 它从守护进程 **Postmaster** 或是其他的后台进程及其子进程中的 **stderr** 进行输出, 并将这些输出的数据信息写入到日志文件中^[27]。

PgStat: 在 **SysLogger** 进程启动后, **PgStat** 进程也将跟着开始启动。统计数据收集进程是用于创建与测试 **UDP** 端口的进程。它专门负责将数据库运行中的统计信息进行汇总, 例如在一个表或索引中进行了几次插入与更新的操作, 磁盘块的数量和元组的数量, 最近清理和分析表的时间以及用户自定义函数调用执行的时间等。

AutoVacuum: 它是一个进程初始化的过程, 它只完成了 **AutoVacuum** 启动选项和 **track_counts** 启动项的检查工作。

BgWriter: 后台写进程; 它一个将 **PostgreSQL** 后台中的脏页写到磁盘上的进程。为了减少查询处理时的堵塞, 并增大缓冲区的空间, 可以应用 **BgWriter** 进程来完成。它能定期的将缓冲区里的脏页写入磁盘中, 这样基于可以减少堵塞的可能性, 并为缓冲区开辟新的空间。

WalWriter: 预写式日志 **WAL** (**Writer Ahead Log**, 也称为 **xlog**) 的中心思想就是对数据文件的修改只能发生在这些修改已经到日志之后, 即先写日志再写数据^[28]。因为先写日志在写数据的特性, 所以即便出现了数据库崩溃的情况, 也可以通过预先写好的日志文件来进行数据库的恢复。

PgArch: 预写式日志归档进程; 它可以使数据库恢复到已记录在案的任何时间点上。

3.1.2 PostGIS 概述

虽然 **PostgreSQL** 能够支持空间数据的特性, 但它所提供的支持是远远不能满足 **GIS** 需求的。主要表现在: 没有复杂的空间类型; 不提供空间分析的功能; 没有投影变换。正因 **PostgreSQL** 存在着这些缺陷, 为了弥补这些缺陷, **PostGIS** 就诞生了。

PostGIS 是 **PostgreSQL** 的空间数据引擎, 它增强了空间数据库的存储管理能力。**PostGIS** 最大的特点就是对 **OpenGIS** 规范的完全支持, 它在空间数据上的管理能力, 就相当于 **Oracle** 的 **Spatial** 模块。**PostGIS** 提供如下空间信息服务功能: 空间对象、空间索引、空间操作函数和空间操作符。**PostGIS** 是由 **Refractions Research** 公司发起开发的。

3.2 PostGIS 数据模型

3.2.1 矢量数据模型

PostGIS 遵循 **OpenGIS** 规范的 **SFS** 模型(**Simple Feature for SQL Model**---简单要素的 **SQL** 模型)。图 3.1 就是 **OpenGIS** 的简单要素 **SQL** 模型。

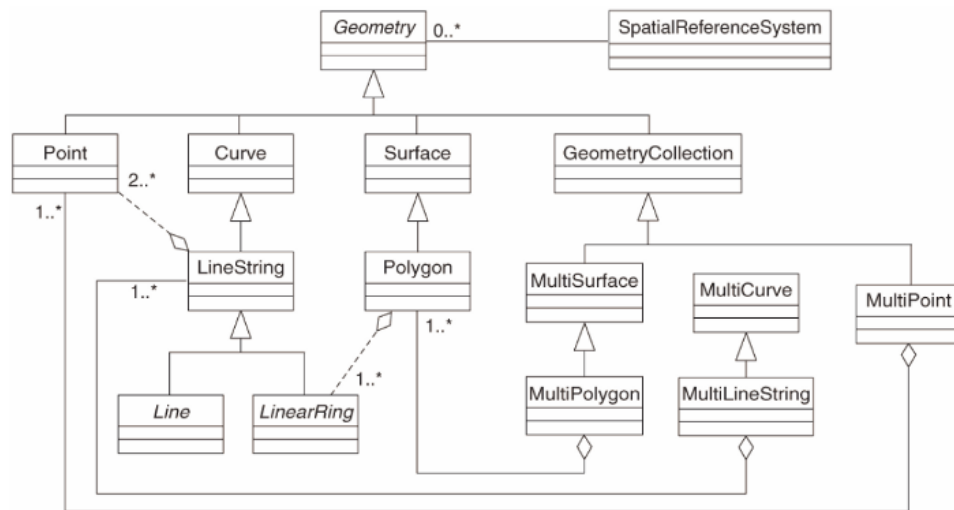


图 3.1 OpenGIS 的简单要素 SQL 模型

PostGIS 支持的空间数据类型(几何类型, Geometry Type)遵循 OGC 简单要素规范,同时在此基础上扩展了对 3D、2DM、3DM 坐标的支持,从而遵循 ISO/IEC SQL/MM 标准。PostGIS 支持主要几何类型包括:点(POINT)、线串(LINESTRING)、多边形(POLYGON)、多点(MULTIPOINT)、多线(MULTILINESTRING)、多多边形(MULTIPOLYGON)和集合对象集(GEOMETRYCOLLECTION)等。

PostGIS 支持的大部分几何类型是基于笛卡尔坐标系的。2D 坐标空间中的 Point 由 (X, Y) 定义, 3D 坐标空间中的 Point 由 (X, Y, Z) 定义, 2DM 空间中的 Point(一般用 PointM 几何类型予以区分)由 (X, Y, M) 定义, 3DM 空间中的 Point(一般用 PointMZ 几何类型予以区分)由 (X, Y, Z, M) 定义。

Linestring 至少由 2 个点来定义。与 Point 类似, Linestring 也有 4 种不同类型的坐标定义: 2D 中的 Linestring, 3D 中的 Linestring, 2DM 中的 Linestring, 3DM 中的 Linestring。

PostGIS 支持的其它几何类型也都支持 2D、3D、2DM、3DM 空间中的定义。

PostGIS 中几何对象的表达采用 EWKT 和 EWKB 格式, EWKT 和 EWKB 相比于 OGC WKT 和 WKB, 主要是扩展了 3D、2DM、3DM 和内嵌空间参考的支持。

以下列举了几个几何对象 EWKT 表达:

POINT(0 0 0): 3D 点

SRID=32632;POINT(0 0): 内嵌空间参考的点

POINTM(0 0 0): 带 M 值的点

POINT(0 0 0 0): 带 M 值的 3D 点

SRID=4326;MULTIPOINTM(0 0 0,1 2 1): 内嵌空间参考的带 M 值的多点

MULTILINESTRING((0 0 0,1 1 0,1 2 1),(2 3 1,3 2 1,5 4 1)): 3D 多线串

POLYGON((0 0 0,4 0 0,4 4 0,0 4 0,0 0 0),(1 1 0,2 1 0,2 2 0,1 2 0,1 1 0)): 3D 多边形

MULTIPOLYGON(((0 0 0,4 0 0,4 4 0,0 4 0,0 0 0),(1 1 0,2 1 0,2 2 0,1 2 0,1 1 0)),((-1 -1 0,-1 -2 0,-2 -2 0,-2 -1 0,-1 -1 0))) : 3D 多边形集合

GEOMETRYCOLLECTIONM(POINTM(2 3 9), LINESTRINGM(2 3 4, 3 4 5)): 2DM 几何集合

MULTICURVE((0 0, 5 5), CIRCULARSTRING(4 0, 4 4, 8 4)): 2D MULTICURVE

POLYHEDRALSURFACE(((0 0 0, 0 0 1, 0 1 1, 0 1 0, 0 0 0)), ((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)), ((1 1 0, 1 1 1, 1 0 1, 1 0 0, 1 1 0)), ((0 1 0, 0 1 1, 1 1 1, 1 1 0, 0 1 0)), ((0 0 1, 1 0 1, 1 1 1, 0 1 1, 0 0 1))): 3D 多面

TRIANGLE ((0 0, 0 9, 9 0, 0 0)): 2D 三角形

TIN(((0 0 0, 0 0 1, 0 1 0, 0 0 0)), ((0 0 0, 0 1 0, 1 1 0, 0 0 0))): 3D TIN

CIRCULARSTRING(0 0, 1 1, 1 0)

CIRCULARSTRING(0 0, 4 0, 4 4, 0 4, 0 0)

COMPOUNDCURVE(CIRCULARSTRING(0 0, 1 1, 1 0),(1 0, 0 1))

CURVEPOLYGON(CIRCULARSTRING(0 0, 4 0, 4 4, 0 4, 0 0),(1 1, 3 3, 3 1, 1 1))

MULTICURVE((0 0, 5 5),CIRCULARSTRING(4 0, 4 4, 8 4))

MULTISURFACE(CURVEPOLYGON(CIRCULARSTRING(0 0, 4 0, 4 4, 0 4, 0 0),(1 1, 3 3, 3 1, 1 1)),((10 10, 14 12, 11 10, 10 10)),((11 11, 11.5 11, 11 11.5, 11 11)))

EWKT、EWKB 格式输入、输出:

bytea EWKB = ST_AsEWKB(geometry);

text EWKT = ST_AsEWKT(geometry);

geometry = ST_GeomFromEWKB(bytea EWKB);

geometry = ST_GeomFromEWKT(text EWKT);

PostGIS 中有很多用于确定空间物体之间位置关系的空间谓词, 并且通过输出 true/false 来对空间对象的关系加以判断。PostGIS 中也存在一些处理空间数据的分析工具, 其中的 Union 主要是将多边形的边界进行合并, 生成一个新的多边形要素, 并且它的边界取的是两者中较大的。

在空间数据库中, 聚集函数是针对某一属性列进行操作的数据操作函数^[32]。就比如 Sum 和 Average 函数, 其中 Sum 是用来求某一列关系属性数据总和的函数^[33]; 而 Average 则是用于求某一列中所有关系属性数据平均值的函数。空间聚集函数在执行数据集合的操作时, 它针对的是空间数据而不是针对某一既定的属性列。例如 Extent 这个空间聚集函数, 它所返回的是要素中最大的外包矩形框。如“SELECT EXTENT(GEOM) FROM ROADS”这句 SQL 语句, 它的执行结果其实就是返回了 ROADS 数据表中最大的外包矩形框。

此外, PostGIS 还提供了地理类型(Geography Type), 主要是用来以地理坐标(常说的

大地坐标, 或者经纬度)表达地理要素。地理坐标属于球面(椭球面)坐标, 以度(degree)为单位来表示。

PostGIS 所提供的几何类型的基础是平面。而在进行几何图形的计算时, 可以使用笛卡尔数学公式来计算面积、距离、长度等。与此相反的, PostGIS 提供的另一类型地理类型, 它的基础则是一个球体(椭球体)。想要计算球体上两点之间的距离, 则需要更为复杂的数据公式来进行计算。因为, 地理类型需要更多的复杂数据基础作为理论, 所以它的功能函数与几何类型的相比, 还是比较少的。但 PostGIS 每时每刻都在发生着变化, 所以不久的将来一定会有越来越多的地理类型的功能函数出现于世。

目前, PostGIS 地理类型只支持 WGS84(SRID: 4326)的经纬度坐标。GEOS (Geometry Engine - Open Source)的所有功能函数都暂时还不支持这种数据类型。

PostGIS 地理类型现在仅支持最简单的要素, 包括点(Point), 线(LineString), 面(Polygon), 多点(MultiPoint), 多线(MultiLineString), 多面(MultiPolygon)以及混合数据类型(GeometryCollection)。

创建带有二维点数据的表可以如下例所示:

```
CREATE TABLE spheroid_points
(
  fid serial PRIMARY KEY,
  name VARCHAR(64),
  location geography(POINT, 4326)
);
```

location 字段是地理类型。它拥有两个参数, 一个是用来限制存储的图形类型和维数的; 另一个则是用来限制 SRID 的。目前的版本中, SRID 只能支持 4326 这个坐标。

3.2.2 栅格数据模型

在 PostGIS 中存储较大的栅格数据对象, 是通过一个新的数据类型来实现的。这种新型的数据类型是由 SRID(包裹矩形框)、类型以及一个字节的序列组成的。一般将数据页值的大小控制在(32*32)以下, 就能够实现对数据的快捷与随机的访问了。而在存储一般的图像时, 亦可以将图像切成像素为 32*32 像素大小的, 然后再将它们存储到空间数据库当中。

使用 raster2pgsql 进行栅格数据的导入。

raster2pgsql 是一个可加载栅格数据的可执行文件, 它将 GDAL 所支持的栅格格式数据转化为适合 PostGIS 的 SQL 栅格数据表。并且这个可执行文件还可以加载栅格文件夹以及创建栅格数据集。这个支持栅格数据类型的可执行文件依赖于对 GDAL 的编译, 这样才能获得 raster2pgsql 所支持的栅格类型数据的列表。

应用 PostGIS 所提供的函数进行栅格数据的创建。在很多时候，你需要在空间数据库中创建正确的栅格数据集以及栅格数据表。这个时候就可以应用 PostGIS 所提供的函数进行创建。下面是用 PostGIS 提供的函数创建了一个栅格数据表，用来存储栅格数据^[33]。

```
CREATE TABLE myrasters(rid serial primary key,rast raster);
```

当然，你还可以应用 ST_AddBand 或是 ST_MakeEmptyRaster 等函数进行栅格数据表的创建。待到你完成了栅格数据表的创建后，你就需要一个索引列表来提高你的查询效率。下面是针对栅格数据表的索引表的创建，如下所示：

```
CREATE INDEX myrasters_rast_st_convexhull_idx ON myrasters USING
gist(ST_ConvexHull(rast));
```

3.2.3 拓扑数据模型

PostGIS 中有许多函数是用来管理空间数据的空间关系的^[35]。例如，函数 ST_Area，它能够返回多边形或多面的面积。对于“几何”型区域是以 SRID 为单位，而对于“地理”区域来说则是以平方米为单位。而它所返回的值将存储在 ST_Surface 或是 ST_MultiSurface 之中。而 ST_MaxDistance 则是用来显示二维投影后两个最大图形之间的距离。

```
postgis=# SELECT ST_MaxDistance('POINT(0 0)::geometry','LINESTRING
(20,02)::geometry');
st_maxdistance
-----
2
-----
postgis=# SELECT ST_MaxDistance('POINT(0 0)::geometry','LINESTRING(22,
22)::geometry');
st_maxdistance
-----
2.31345678129845
-----
```

PostGIS 中也存在 9 交拓扑模型。如，ST_Contains，ST_Crosses，ST_Intersects，ST_Touches 等等。例如，当你考虑一个线性数据集表示道路网络时，作为一个地理信息系统的分析师，你的任务就是找出所有相互交叉的路段，这不是在一个点上而是在一条线上，所以对于突破规则的定义就显得尤为的重要。而在这种境况下，ST_Crosses 并不能发挥它的作用，因为它不是针对线性特性，它只能在有点交叉时才会返回 True。于是解决这种问题时，就需要应用两种函数相结合，首先先应用 ST_Intersection 进行路段空间相交的实际交点进行监测，然后再进行 ST_GeometryType 与 LINESTRING 的比较，最后就会返回正确的多点、多线、多面等之间的相交关系。

PostGIS 中 topology 模型通过类型 TopoGeometry 封装。在 PostGIS 中创建拓扑数据模型如下：

```
CREATE TYPE TopoGeometry AS(
    topology_id integer,
    layer_in integer,
    id integer,
```

PostGIS 中 TopoGeometry 函数实际上是看作为要素表(feature table)的一个列。真正的点、线、面要素的坐标数据并没在 Topo-Geometry 中，而是被存于了关系表---拓扑表(topology table)中。TopoGeometry 类型的对象在建立拓扑图层的组织、要素表与拓扑表之间的关系时，是通过这三者之间的关联信息表。如图 3.2 所示：

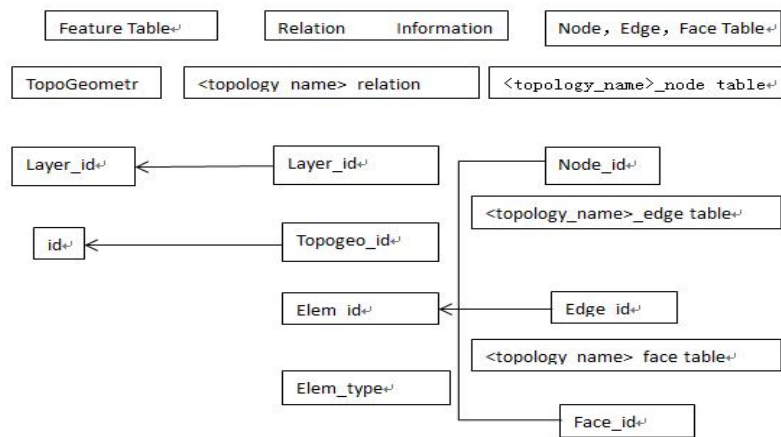


图 3.2 拓扑表与要素表的映射关系

在 PostGIS 的 topology 模式下定义了一个全局元数据表 topology(完全访问符为 topology.topology)。这个表是用来保存空间数据库中拓扑模型实例的 ID、命名、空间参考系统标识以及数据容错阈值等的。下图为 PostGIS/PostgreSQL 空间数据库的拓扑模型逻辑结构^[36]，如图 3.3 所示。

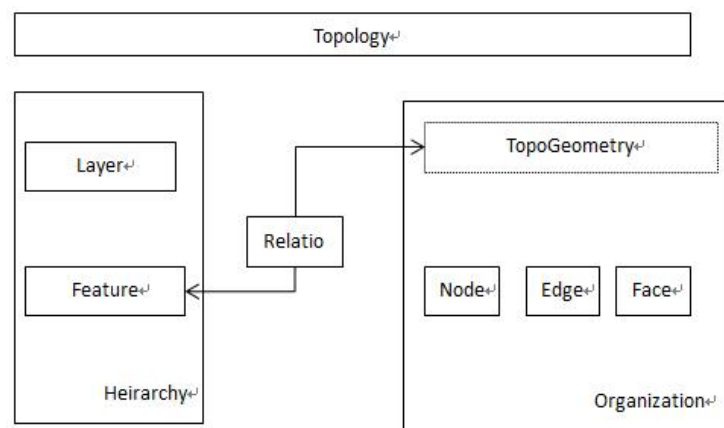


图 3.3 PostGIS/PostgreSQL 拓扑模型逻辑结构

每个拓扑模型的实例即为一个要素(Feature)的分层组织^[36]。而这些要素应当与 TopoGeometry 类型中的对象进行一一的相关联,而每个 TopoGeometry 类型里都有 Node、Edge、pylon 这三个基本要素构成。如多边形几何类型房屋,它必须由顶点(Node)、边界(Edge)、和所占据的面(Face)这三个要素来共同表达^[37]。而房屋与周围地物的拓扑关系可以通过三要素之间的关系来获得。

在 PostGIS 中定义了拓扑模型三要素在数据库中的映射关系:

Edge(边 ID[PRIMARY KEY], 起始节点[FOREIGN KEY], 终止节点[FOREIGN KEY], 下一条相邻左侧边[FOREIGN KEY], 下一条相邻右侧边[FOREIGN KEY], 左面[FOREIGN KEY], 右面[FOREIGN KEY], 几何坐标数据)。

Node(节点 ID[PRIMARY KEY], 所在面[FOREIGN KEY], 几何坐标数据)。

Face(面 ID[PRIMARY KEY], 最小外包矩形数据)。

可见,空间数据库中点、线、面三要素间的拓扑关系,能够表达出空间对象的具体坐标信息。在 OpenGIS 的 SFS 中的 Geometry 数据类型是进行几何数据的存储^[38]。这些数据类型可以用来建立空间索引或是空间查询与分析。PostGIS 有拓扑模型的继承特性,所以用户可以应用这一点来进行分析与运算。

3.2.4 长事务管理及线性参考

(1) PostGIS 中的长事务管理

PostGIS 中提供了针对长事务进行管理的函数。如, AddAuth, CheckAuth, DisableLongTranctions 以及 LockRow 和 UnlockRows。

AddAuth---在当前的事务中添加一个授权令牌;

将当前事务标识与授权时的令牌密钥添加到 temp_lock_have_table 的表中。具体实例如下所示:

```
SELECT LockRow ('road','353','babaoshan');
BEGIN TRANSACTION;
SELECT AddAuth('wudaokou');
UPDATE roda SET the_geom=ST_Translate(the_geom,2,2) WHERE
gid=353;
COMMIT;
```

CheckAuth---创建触发器来防止或是允许已经拥有授权令牌行的更新与删除;

DisableLongTranctions---禁用长事务支持,该函数它删除了长事务支持的元数据表并降低了所有触发器的 lock-checked 表;

EnableLongTransactions---允许使用长事务支持,该函数在创建所需的元数据表示时,需要先调用其他函数。所以,它也被称作二次调用。

LockRow---设置授权表特定行的锁;

UnlockRows---删除授权 ID 的所有被指定的锁，并返回这些锁的数量。

(2) PostGIS 中的线性参考

线性参考是指以线要素的位置为相对位置进行地理位置的存储方法。与传统的坐标相比，某些对象的位置沿着线性特征能够更准确的确定，并且它们的位置可以由一个已知的对象进行测量。例如，旅行的距离，它可以从已知的出发点开始计算。

在 PostGIS 中也有对线性参考的具体定义。PostGIS 中的线性参考是通过一系列的函数来展现的。如以下的几个函数所示：

ST_LineInterpolatePoint---沿着线返回一个内插的点。第二个参数是 0 和 1 表示了线串中的点须位于线长度的比例之间。如，在线串 A20%的位置内插一点(0.20)。

```
SELECT ST_AsEWKT(ST_Line_Interpolate_Point(the_line,0.5))
      FROM (SELECT ST_GeomFromEWKT('LINESTRING(123,456,678)')
            as the_line) As foo;
      st_asewkt
```

```
-----
POINT(3.5, 4.5, 5.5)
```

ST_AddMeasure---返回一个内插与几何元素起始点与终点之间的线性。如果几何元素没有进行尺寸的测量则应该对其进行补充；如果几何测量的维度过度的使用新值，则此函数只能支持线与多线。

ST_LocateAlong---此函数是返回一个派生的几何集合值，并审查它是否与指定的度量元素相匹配（此函数不支持多边形元素）；

ST_LocateBetween---返回在制定范围内与派生的几何集合值相匹配的元素。(此函数不支持多边形元素。)

ST_InterpolatePoint---返回与提供点相近点的几何测量维度的值。

3.3 重点流通企业空间数据库设计

本文采用 PostgreSQL 与 PostGIS 进行北京市石景山区重点流通企业空间数据库的设计。由于 PostgreSQL 的空间数据引擎 PostGIS 完全支持 OGC 标准，所以 OGC 中的数据模型 PostGIS 也完全能够支持。并且，PostGIS 中还有属于自己内部的空间数据模型。本文将以 OGC 的空间数据模型及 PostGIS 中的空间数据模型为理论基础，进行重点流通企业信息管理的空间数据库设计。

在进行空间数据库的设计之时，不但要将关系型数据库的理论规范化，还应当能够真实的反应现实世界对象之间的关系。尽量去除库中的重复与冗余的数据，并与流通企业的数据特点、用户需求相结合。应用 PostGIS 的空间数据模型建立适宜的要素类、对象类以及关系类等。具体的空间数据库设计应遵循以下几个步骤：需求分析、分层设计、

概念设计、逻辑结构设计以及完整性约束的设计^[40]。

3.3.1 需求分析设计

随着北京市的经济战略转型，石景山区同样面临着经济战略转型，面对诸多复杂经济问题的挑战，必须以科学发展观为指导，加强各部门的信息化建设，深化经济发展的前瞻性、综合性、系统性发展研究和决策。

对于重点的流通企业信息的管理是将不同业态、不同企业、不同行业的销售规模、运行质量、消费结构、网点布局等数据与地理空间数据建立相关的联系，并通过对它们的展示和运算分析方法等，直观的反映业态等的空间分布、业务要素数据与空间数据的对应关系、反应流通产业发展和消费变化的趋势，它能够帮助我们开展数据挖掘利用分析，了解经济、分布状况与街道或是社区等的关系，使用户能够更好的制定产业规划的政策。

3.3.2 分层设计

应北京市石景山区商委的项目需求，对重点流通企业信息管理的数据信息，将以图层的方式进行组织与管理。根据笔者前期调研的需求分析，重点流通企业管理系统的数据库中的数据主要是基础地形数据以及各流通企业的业务数据。

(1) 基础地理数据

目前，基础的地理数据已成为众多空间信息技术中最为普遍的数据信息了。因此，建立基础地理数据的空间库，可以避免不同部门需要使用时再次进行数据收集的复杂工作。北京市石景山区的基础地理数据有：区界、道路、水系、绿地、居民地、街道、单位、医院以及学校等，如表 3.1 所示。

表 3.1 基础地里数据分层表

图层名称	内容描述	类型
道路	区内各个主、次、支路	Polygon
绿地	区内所有绿地	Polygon
水系	区内所有湖池塘等	Polygon
街道	区内社区邻近街道	Line
单位	区内主要政府机关单位等	Polygon
医院	区内所有医院、卫生服务中心、卫生服务站等	Point
学校	区内所有小中大学	Point

续表 3.1 基础地里数据分层表

居民地	区内的所有居民用地	Polygon
区界	石景山区的区界	Line

(2) 重点流通企业数据

根据本课题的需求，将商委监管下的各个流通企业进行梳理，并建立相应的点图层。因商委下监管的流通企业众多，且由于篇幅的限制等原因，表 3.2 只是流通企业中的一部分。

表 3.2 流通企业数据分层表

图层名称	内容描述	类型
专卖店	各类型专营店	Point
4S 店	各类汽车 4S 店	Point
超市	区内所有小中大超市	Point
餐饮店	所有类型餐饮经营店（中、西、韩式等）	Point
百货店	所有百货经营店、购物中心	Point
蔬菜服务站	区内各社区蔬菜服务站（包括车载、与门店经营）	Point
社区商业	社区周边门店经营类	Point
加油站	区内所有加油站点	Point

3.3.3 概念结构设计

概念设计即对用户的需求进行了详细的分析后，抽象出一个不依赖空间数据库的存在而存在的结构模型设计。通过上述对用户的需求分析，得出了本课题中的实体，有水系、街道、居民地、单位、超市、加油站等。具体实体间的关系如 E-R 图 3.4 所示（超市为例）：

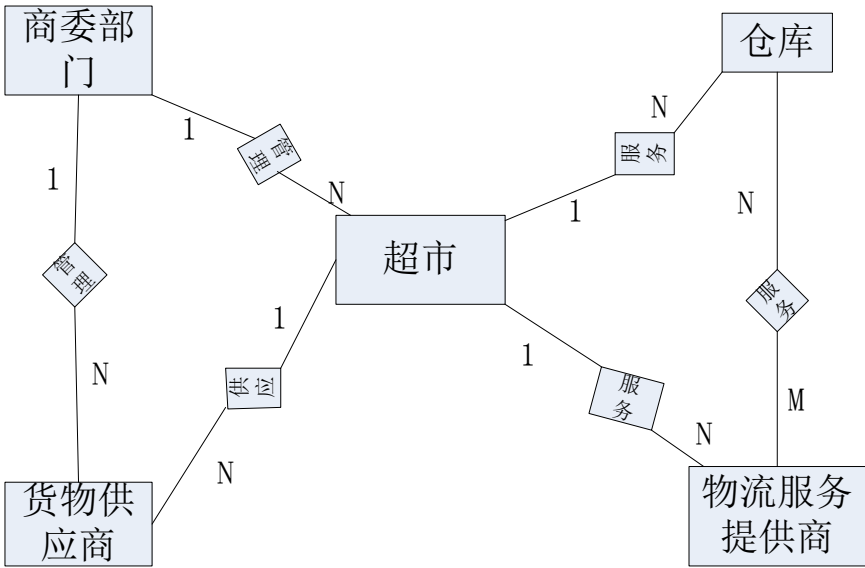


图 3.4 流通企业空间数据库 E-R 图

3.3.4 逻辑结构设计

逻辑设计是将概念设计中抽象出数据结构表达为具体的实体之间的关系。应用空间数据的模型，以层次类型对数据对象进行组织，并设计逻辑结构模型如图 3.5 所示。

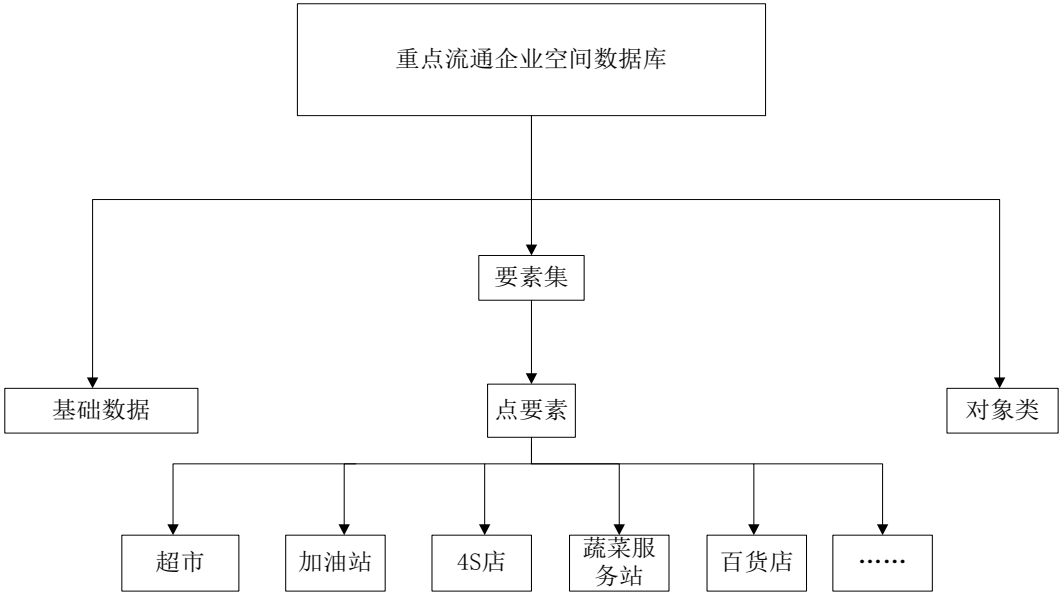


图 3.5 空间数据库层次结构图

在石景山区的重点流通企业信息管理系统中，共存在两个要素集：基础数据信息与流通企业的业务数据要素集。而在空间数据库中，统一要素集内的要素类的空间参考必须统一。并且在进行空间数据库的设计时，空间参考更是其设计中的重要组成部分^[41]。本文里的所有要素类都采用了 WGS 84 标准进行了坐标系统、空间域以及精度等的校验。

逻辑上来说，要素类是由表呈现的，每个表中不但要存储属性信息同时还会存储空

间信息。空间信息则被归类于要素类中进行存储，而属性信息则归于对象类中。至于关系类，则是为了说明要素类与对象类之间的关系^[41]。

根据对重点流通企业的需求分析与空间数据库的设计理论，本文的要素、对象类以及它们之间的关系类如下所示：

要素类：街道(线对象)；社区(面对象)；商委监管的流通企业均为点要素，例如，超市(点要素)、蔬菜服务站(点要素)、加油站(点要素)等。

对象类：商委部门、所属街道情况、所属社区情况、网点分布情况、销售量等。

关系类：由于流通企业包含许多的商业业态，所以这里仅以超市与蔬菜服务站为例，来说明两表之间的联系。

街道-超市关系：简单关系；社区-超市：简单关系；街道-蔬菜服务站：简单关系；社区-蔬菜服务站：简单关系；超市-商委部门：简单关系；超市-商委部门：简单关系；

业务数据表中存储了所有关于重点流通企业的数据信息，如商业业态、重点流通企业以及重点的限上/限下企业所必要的对象，因为它们并不是真正意义上的地理空间信息，有些对象是需要依赖空间要素层的对象才能够进行相对的定位，就比如本文中的企业或是业态信息，它们很多都是依赖于街道为基本单位，点击不同的街道查看这个街道中业态的分布，企业的销售规模以及运行质量等信息。由于业务数据的数据表数量众多，所以这里将以重点流通企业为例，来阐述业务数据表及一些重点字段的设计。

北京市石景山区重点流通企业空间数据库的名称为：SJS。此数据库中共有两个要素集，基础信息数据集、点要素集。其中包含的要素类有：百货店、超市、加油站、4S店、蔬菜服务站等，它们都是点对象。由于本文的空间数据库中除了基础信息数据集外，就是点要素集，所以在这里将选择 Vegetable Services 与 Supermarket 作为代表来进行属性数据结构的设计。如表 3.3 与 3.4 所示。

表 3.3 Vegetable Services 数据结构表

字段名称	数据类型	是否能为空	描述
Organization id	Int	否	组织机构代码（主键）
X	Int	否	X 坐标
Y	Int	否	Y 坐标
Road	Varchar（20）	否	所属街道
Community	Varchar（20）	否	所属社区
Date name	Varchar（20）	否	数据名称
locale	Varchar（20）	是	采集地点
Time	Date	是	采集时间
Purpose	Varchar（20）	是	采集目的

续表 3.3 Vegetable Services 数据结构表

PersonId	Int	是	采集人 ID
Path	Varchar (20)	是	存储路径
Equipment	Varchar (20)	是	设备
Class	Varchar (100)	否	是否为限上
Name	Varchar (20)	是	单位名称
Address	Varchar (20)	是	单位地址
Contact	Varchar (100)	是	负责人联系方式
Registration	Int	否	登记注册代码
Registrationclass	Varchar (100)	否	登记注册类别
Retail	Varchar (100)	是	销售情况
Manage	Varchar (100)	是	经营方式
Remark	Varchar (20)	是	备注

表 3.4 Supermarket 数据结构表

字段名称	数据类型	是否能为空	描述
Organization id	Int	否	组织机构代码（主键）
X	Int	否	X 坐标
Y	Int	否	Y 坐标
Road	Varchar (20)	否	所属街道
Community	Varchar (20)	否	所属社区
Date name	Varchar (20)	否	数据名称
locale	Varchar (20)	是	采集地点
Time	Date	是	采集时间
Purpose	Varchar (20)	是	采集目的
PersonId	Int	是	采集人 ID
Path	Varchar (20)	是	存储路径
Equipment	Varchar (20)	是	设备
Class	Varchar (100)	否	是否为限上
Name	Varchar (20)	是	单位名称
Address	Varchar (20)	是	单位地址
Contact	Varchar (100)	是	负责人联系方式
Registration	Int	否	登记注册代码

续表 3.4 Supermarket 数据结构表

Registrationclass	Varchar (100)	否	登记注册类别
Retail	Varchar (100)	是	销售情况
Manage	Varchar (100)	是	经营方式
Remark	Varchar (20)	是	备注

3.3.5 完整性约束设计

空间数据完整性约束指为了保护数据的正确性、有效性、相容性，并防止错误数据入库的一组规则^[42]。不过，实际上它的限制范围还是比较大，对于应用来说并不真正的适用。这就需要用户根据具体的应用来设计符合需求的空间数据完整性约束。与此同时，若是某些用户想要违反数据的这个约束，那么 PostgreSQL 将会报错，这种情况也同样适用于缺省值的境况。

约束是一个命名的规则，它通过 INSERT、UPDATE、DELETE 等对表的操作结果进行约束^[43]。当要进行 INSERT、UPDATE 等操作时，必须要满足约束规则，操作才能成功的完成。在 PostgreSQL 中存在着两种方式来定义完整性的约束，一是表约束另一个就是列/字段约束。

列/字段约束是属于部分约束，逻辑上而言，一旦创建了此种约束，就会自动成为表约束。下面列举了一些列约束：PRIMARY KEY、REFERENCES、UNIQUE、CHECK 以及 NOT NULL 等。

下面将根据北京市石景山区重点流通企业的空间库对 SJS_business 里的超市、加油站这两个业态进行具体的完整性约束设计。由于在上一个章节已经详细的介绍了关于完整性约束的基本理论知识，所以在此就不再赘述。

(1) NOT NULL 约束

NOT NULL 约束是指对某一列的数值不允许为空值^[44]。NOT NULL 只允许在列或字段上进行约束，不允许作为表的约束。

```
CREATE TABLE supermarket (
    did      DECIMAL(3) CONSTRAINT no_null NOT NULL,
    name     VARCHAR(40) NOT NULL
);
```

(2) UNIQUE (唯一)约束

UNIQUE 约束是指表中一组由一个或多个独立列组成的集合中只能包含一个唯一的数值^[45]。并且，一个字段或是列上已经有了 UNIQUE 的约束，就不能在含有 NOT NULL 的约束。

```
CREATE TABLE supermarket (
    did      DECIMAL(3),
```

```

    name    VARCHAR(40),
    UNIQUE(name)
);

```

(3) CHECK （检查）约束

CHECK 约束不但可以作为一个字段或是列的约束，它还能够作为表约束。它一般是以布尔表达式来进行表或是字段的约束。

```

CREATE TABLE supermarket (
    did      DECIMAL(3),
    name     VARCHAR(40)
    CONSTRAINT con1 CHECK (did > 100 AND name > '')
);

```

(4) PRIMARY KEY (主键)约束

PRIMARY KEY 列约束表明表中的一个列/字段只能包含唯一的（不重复），非空的数值^[46]。一个表只可以有一个主键约束。

```

CREATE TABLE supermarket (
    code_ID   NUMBER(5),
    name      VARCHAR(40),
    did       DECIMAL(3),
    kind      CHAR(10),
    sales     DECIMAL(10),
    CONSTRAINT code_name PRIMARY KEY(code_ID)
);

```

(5) REFERENCES 约束

REFERENCES （参考）约束声明了一个规则：一个字段的数值要与另外一个字段的数值做对比检查^[47]。

```

CREATE TABLE supermarket
(
    OBJECTID NUMBER NOT NULL ,
    CODE_ID  NUMBER,
    PRIMARY KEY (CODE_ID),
    FOREIGN KEY (Organization_ID)REFERENCES SJS_business (Organization_ID)
    ON DELETE CASCADE    /*级联删除 SJS_business 表中相应的元组*/
    ON UPDATE CASCADE    /*级联更新 SJS_business 表中相应的元组*/
);

```

(6) 空间语义完整性约束

空间语义完整性约束(Spatial Semantic Integrity Constraints)是指用于限制地理实体的空间布局、地理实体的空间属性及地理实体之间的空间关系。空间实体之间的空间关系一般是指下列关系：拓扑关系、方位关系和度量关系^[48]。因流通企业数据的特点，这里只针对拓扑语义的完整性约束进行设计。

拓扑语义完整性约束(Topological Semantic Integrity Constraints)用于约束两个几何形状之间应该满足某种空间拓扑关系，以确保空间数据符合拓扑语义。PostGIS 中提供了一系列的函数来对空间数据进行拓扑规则的检测。如下所述：

Getfaceedges_returntype---应用这个函数则会返回一个 ST_GetFaceEdges 的表，表是由一个序列号和一个边号组成的。

序列是一个整数：是指在 topology.topology 表中定义的拓扑架构和 SRID 定义的拓扑结构。边也是一个整数，是指边缘的标识符。

TopoGeometry---这个函数定义了如何校验复杂的拓扑关系的规则。

它是指一个拓扑几何形状中的特定拓扑结构层，具有特定的类型和特定的 ID。一个 TopoGeometry 的元素属性应该有：topology_id, layer_id, ID 为整数类型。

topology_id 是一个整数类型：是指在 topology.topology 表中定义的拓扑架构和 SRID 定义的拓扑结构。**layer_id** 是一个整数类型：是指该 TopoGeometry 所属的层表中的 layer_id。topology_id 与 layer_id 的组合为 topology.layers 表提供了唯一的参考。

validatetopology_returntype---它是一个复合类型函数，用来检查错误的消息，其中 id₁ 与 id₂ 表示错误的拓扑对象的 id。

当前的错误描述可能是：点重合，边端点相交，边的终点几何形状不匹配，边的起点几何形状不匹配，面内重叠，面在面上等。

AddTopoGeometryColumn---指增加了一个 topogeometry 列到本表中，然后注册这个新列作为图层 topology.layer 的新 layer_id。

在 PostGIS 中还有很多的拓扑函数，在这里就不再进行一一的列举了。结合重点流通企业的数据信息，进行重点流通企业的拓扑语义完整性约束设计。如，加油站不能建立在农林耕地的附近、不能建立在居民区的一千米以内；超市适宜建立在居民区、学校、医院等附近区域；每个街道都应存在蔬菜服务站点等。具体实现如下所示：

```
CREATE SCHEMA supermarket;
CREATE TABLE supermarket.parceles(gid serial,parcel_id varchar(20) PRIMARY
KEY,address text);
SELECT
topology.AddtopoGeomtrycolumn('supermarket_topo','supermarket','parceles','topo','
POINT');
CREATE SCHEMA ri;
CREATE TABLE ri.road(gid serial PRIMARY KEY,road_name text);
SELECT topology.AddTopoGeometryColumn('ri_topo','ri','road','topo','LINE');
CREATE SCHEMA shu;
CREATE TABLE shu_residential_area(gid serial PRIMARY KEY, residential area
_name text);
SELECT topology.ST_MumPoint('shu_topo','shu','topo','POLYGON');
```

3.3.6 触发器设计

一个触发器是一种声明，告诉数据库应该在执行特定的操作的时候执行特定的函数^[49]。触发器可以作用于表和视图。本文应用存储过程语言 PL/pgSQL 定义了一个触发器。下面是以 Stations(加油站)选址的规则进行一个触发器的设计。Stations(加油站)不能在居民区一千米以内建立，不能建立在农林耕地中。

```
CREATE TABLE stations (
    name text,
    address varchar(20),
    organizationID int,
);
CREATE FUNCTION e_stamp () RETURNS trigger AS $e_stamp$
BEGIN
    -- 检查是否给出了 name 和 address
    IF NEW.name ISNULL THEN
        RAISE EXCEPTION 'name cannot be null';
    END IF;
    IF NEW.address ISNULL THEN
        RAISE EXCEPTION '% cannot have null address', NEW.name;
    END IF;
    IF NEW.organizationID ISNULL THEN
        RAISE EXCEPTION '%cannot have null organization', NEW.address, NEW.name;
    END IF;
    IF NEW.address == community THEN
        RAISE EXCEPTION '% cannot have a negative address', NEW.name;
    END IF;
    IF NEW.address == farming land THEN
        RAISE EXCEPTION '% cannot have a negative address', NEW.name;
    END IF;
    RETURN NEW;
END;
$e_stamp$ LANGUAGE plpgsql;
CREATE TRIGGER e_stamp BEFORE SELECT ADDRESS ON SASTIONS
FOR EACH ROW EXECUTE PROCEDURE e_stamp();
```

第四章 重点流通企业信息管理系统的设计及实现

4.1 系统设计原则

对于系统设计，能够满足今后多层次的对于重点流通企业信息管理的需要，减少实现过程与设计当中可能出现部分的问题，保证系统开发的成果，应遵照以下的几个原则：

(1) 实用易用性

系统应在充分实现客户所提出的各项功能需求的基础上，包括本地区重点流通企业的各项信息查询与更新外，还应该着重考虑到系统的适用性问题，它的操作习惯应该是针对普通的用户，而非熟悉计算机的专业用户。

(2) 技术领先性

软件技术的发展是一个突飞猛进的过程，系统在开发的过程中应采用较为领先的技术，这样才能够使开发出的软件拥有诸多的优良特性，并且能够最大限度地提升用户的使用体验，以此来达到应有的使用效果和实际效益。

(3) 安全可靠

一个系统是否稳健，其在安全性能上的表现是十分重要的。由于现在大多数的客户端都会采取联网运行，这样就使其很容易遭受到网络病毒、木马等的危害。而在一个系统设计的过程中，不但会有为客户服务的客户端，还会有后端进行支持的数据库以及服务器等。所以本着对数据信息、用户信息以及数据库等的安全考虑，在进行系统设计之初就应当将安全性的设计纳入其中。

(4) 易于扩展性

任何软件系统都不是封闭的，它可以再某一特定的时间段内满足用户的需要，但这并不意味着它就可以这样一成不变。随着时间的推移，用户的需求在改变，数据库中的数据量也将变的越来越多。这样就要求设计人员在系统设计的初期，就应当考虑到以后软件的扩展与更新的可能性。因此在系统的实现过程中应该充分考虑到这些变化着的因素，留下开发接口，以便于日后进行扩展与维护。

(5) 经济省钱

这是本文研究的理念，由于商业软件越见强大，伴随而来的也是越来越高昂的价格。因此本系统在开源下的 QGIS 平台上进行二次开发，开发工具、数据库、数据处理工具、以及应用服务容器也全部都是开源软件。这样不但大大的减少了系统开发的成本，又不会涉及到任何版权问题，并且这些开源的软件功能也完全可以满足用户的需求，这就是本文最大的特点。

4.2 系统的架构

北京市石景山区的重点流通企业信息管理系统是一个 C/S 系统，其是针对某一特定的用户而进行开发。这个用户既可以在系统上进行空间信息与相关业务的浏览查询，又可以对这些数据进行修改更新，还可以对这些数据进行统计分析。其总体架构设计如图 4.1 所示：

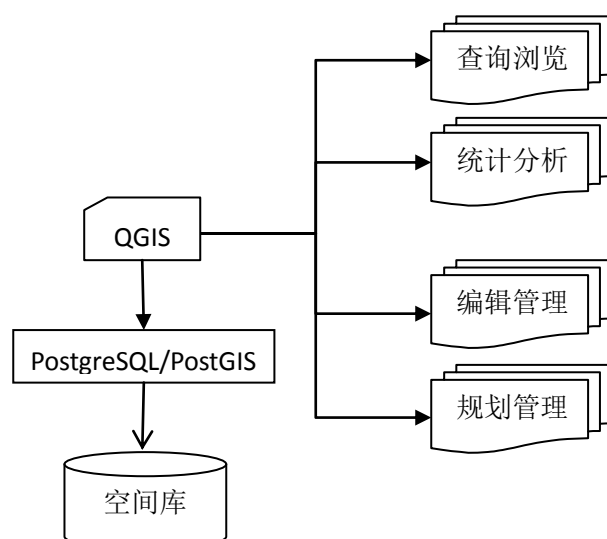


图 4.1 北京市石景山区的重点流通企业信息管理系统的总体架构图

4.3 系统应用逻辑设计

笔者首先针对国内目前已经正在运行的流通企业信息管理系统、商务直销行业管理信息系统等进行了比较深入的研究，并研究了传统应用商业软件开发的 C/S 系统，了解了它们的开发体系，数据库设计等，并以此为笔者的理论基础，为本文应用全开源 GIS 软件来实现系统的开发奠定了基础。

4.3.1 客户逻辑设计

客户逻辑就是用户通过这个逻辑进行系统的访问，一般都将它部署在用户的个人计算机上。本系统的用户界面简洁大方，并且十分易于操作。在用户使用本软件时，需要先用户的个人计算机上安装部署 QGIS 以及 QT，这样用户才可以使用本软件。

4.3.2 数据逻辑设计

数据逻辑是指系统分析人员对数据存储的观点，是对业务对象、业务对象的数据项以及业务对象之间关系的基本规划蓝图。它一般被部署在后端的数据库管理系统中^[50]。本系统通过 PostGIS 的内置函数等存储系统的业务数据。并通过 QGIS 的桌面应用系统

来访问 PostGIS 里的空间及属性数据，并对这些数据进行存取和操作。本文的数据主要包含空间数据与属性数据，支持的空间数据格式主要是 ESRI 的 shp 数据，当然也支持 AutoCAD 数据以及 XML 等格式的数据进行存储与操作。

4.3.3 系统功能设计

北京市石景山区重点流通企业信息管理系统除了具备对地图操作的基本 GIS 功能外，还包括了针对石景山区的经济中心地带一轴三心四街的查询、统计及分析功能以及具有特色的蔬菜服务网点分布的统计与分析功能。本系统一共有六个功能模块，对数据进行管理的数据管理模块；查看地图变化的视图功能模块；根据用户需求进行查询的查询模块；对系统数据进行编辑的编辑模块；对业务数据进行统计分析的统计分析模块以及对经济中心一轴三心四街的规划模块。它实现了对商业业态、重点限上/限下企业、蔬菜服务网点分布等的查询、统计分析等功能。还将不同年份的规划图进行叠加，以此来查看区内的规划情况。系统功能的总体设计如图 4.2 所示：

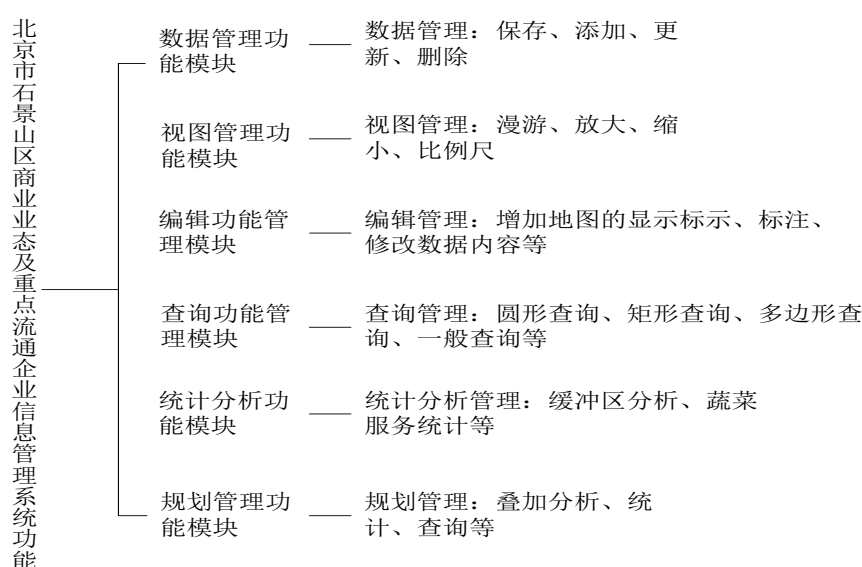


图 4.2 系统功能设计框架图

(1) 数据管理模块。主要是以实现数据的管理为主，包括添加、保存、删除、更新等数据上的操作，这样可以使客户十分便捷的处理数据。

(2) 视图功能模块。本模块主要是为了方便用户对地图的变化进行查看。主要有漫游、放大、缩小、鹰眼等功能组成。

(3) 编辑功能模块。即是以系统数据的编辑工作为主，例如增加地图的显示标示、标注、修改一些数据内容等。

(4) 查询功能模块。查询功能在 GIS 中是十分普遍也是十分重要的功能。用户可以根据的需求进行有目的的查询操作。在 GIS 中有两种查询方式，一个是图形查询，另一

个是属性查询，这两个查询功能是可以实现双向查询的。用户通过点击相应的查询按钮来进行多边形查询、一刻钟周边查询等，这些被查询的数据信息，将会被应用于统计与分析等工作，并为日后的数据对比做准备。

(5) 统计分析功能模块。统计分析模块是本系统的核心模块，主要是为商委对所管辖范围内的数据进行统计并给予分析，更是为日后石景山区的规划提供依据。本文的统计功能是针对商业业态，限上或是限下企业，蔬菜服务等进行统计分析。通过这些统计功能能够计算出某一个街道里，商业业态以及蔬菜服务的个数，并进行分析所选街道内业态或是蔬菜服务站的分布及展示出它的具体属性信息。

(6) 规划模块。本部分功能模块是应北京市石景山区商委所提出的特殊需求进行开发的。它是针对石景山区的经济中心地带即所谓的一轴三心四街进行的有针对性的查询、统计及分析。用户想借此来查看经济中心地带的发展，并为将来此地的发展规划提供有利的依据。此模块分为三个部分，一轴，三心，四街，用户可以分别对其进行查询，统计分析等操作，以此来获取经济中心地带的数据信息。用户还可选择叠加规划图来查看此部分的不同年限之间规划的差异。

4.4 空间数据库的环境配置与建立

4.4.1 环境的搭建与配置

(1) PostGIS 的安装与配置

PostGIS 是对象-关系型数据库 PostgreSQL 的空间数据引擎^[51]。在安装 PostGIS 前必须安装好 PostgreSQL，然后再在 Application Stack Builder 之中选择安装 PostGIS 组件。本文中用的是 PostgreSQL，Version 9.2 win 32。并 PostgreSQL 9.2 进行了安装，然后再将 PostGIS 组件勾选并安装，如图 4.3 所示：

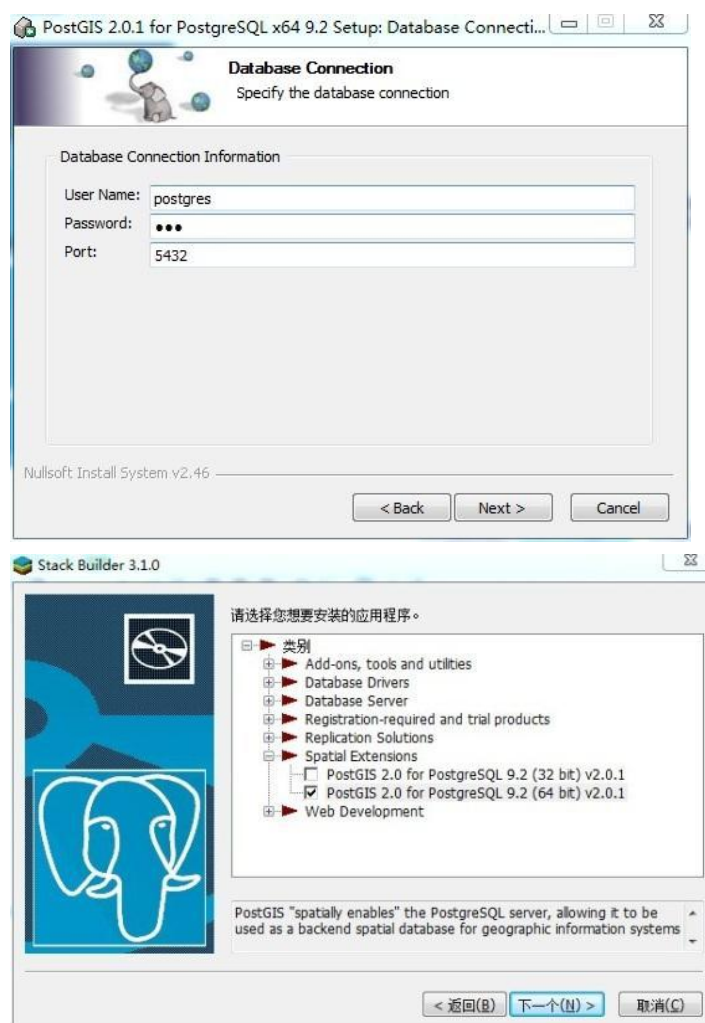


图 4.3 安装 PostGIS 组件

(2) QGIS 开发环境配置

QGIS 是一个自由的 GIS 桌面系统，它可以再许多的操作系统中运行。如 Linux、Unix、Mac OSX 和 Windows 等众多平台之上。如果，想要对 QGIS 进行扩展，则需要以插件的方式来完成。对于这些扩展 QGIS 的插件，需要使用 QT 开发包和 C++开发语言来进行插件的研发工作。

系统环境要求有以下几个部分，软件环境为 Windows、vs2010；硬件环境为 lenovo 笔记本；开发工具为 C++、QT 4.7.3、QGIS2.0 链接库等。

想要应用 QGIS 进行二次开发的工作，就必须要先对其的开发环境进行搭建与配置。第一，在 vs 中新建一个 QT Application 的项目之后，在进行 C++预处理器命令的配置。第二，进行链接库中 lib 文件与目录的常规配置。第三，再在附加包含目录中添加编译好的 QGIS 的 include 路径和 Osgeo4w 的 include 的路径；第四，将 QGIS 编译库中的 lib 文件路径与 Osgeo4w 的 lib 文件目录添加到上述的 lib 文件目录中。第五，将附加依赖项里的三个核心 lib 文件添加进去。具体配置如图 4.4 所以：

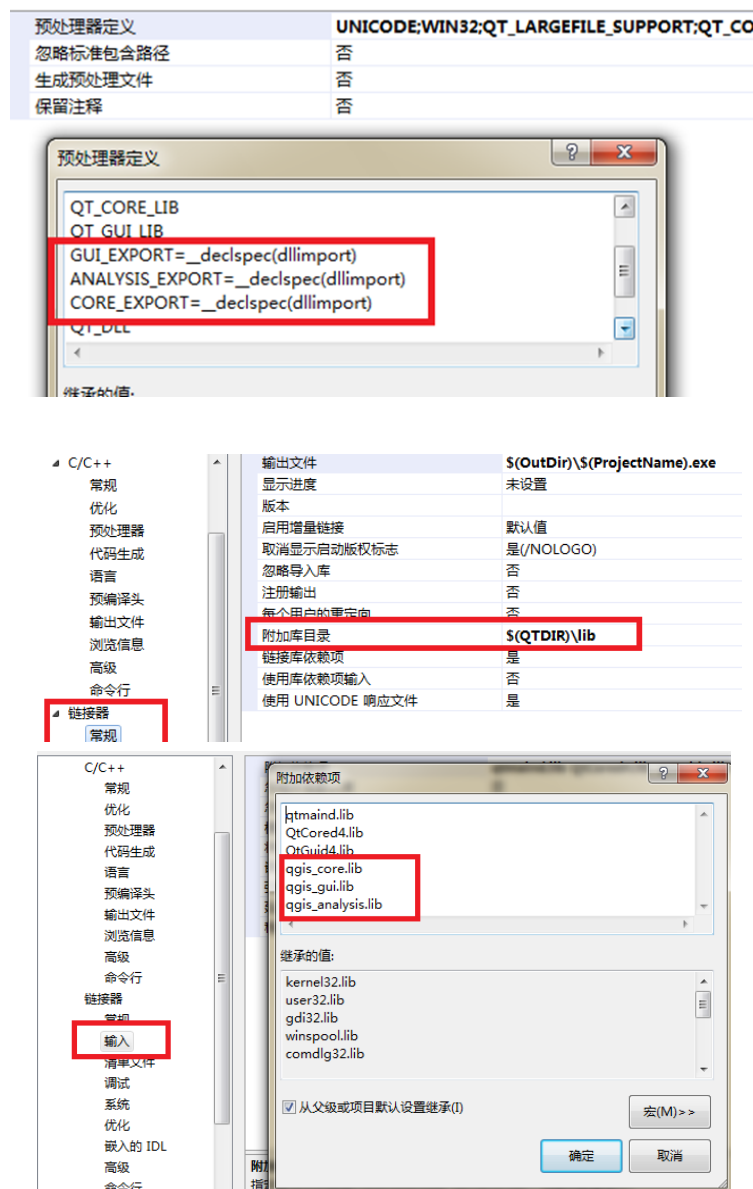


图 4.4 QGIS 开发环境的配置

在应用 QGIS 进行二次开发之时，如果对中文进行输出则必须通过代码的配置，修改主函数，才能时开发出的应用程序进行中文的输出。对于中文输出设置的代码如下：

修改 main.cpp 文件

```
#include "sjsqgis.h"
```

```
#include <QtGui/QApplication> int main(int argc, char *argv[])
```

```
{
```

```
    QgsApplication a(argc, argv, TRUE);
```

```
    sjsqgis w;
```

```
    w.show();
```

```

    return a.exec();
}

```

由于对 QGIS 的扩展项目大多数都是由国外的社区或是机构完成的，所以想要使用 QGIS 二次开发，并使其能够显示中文，需要通过一段代码来实现。支持中文显示的代码如下：

```

QgsApplication a(argc, argv, TRUE);
QTextCodec *codec = QTextCodec::codecForName("System");
QTextCodec::setCodecForCStrings(codec);
QTextCodec::setCodecForLocale(codec);
QTextCodec::setCodecForTr(codec);

sjsqgis w;
w.show();
return a.exec()

```

(3) 空间数据库的建立

想要在 PostgreSQL 中创建一个空间数据库，则需要一个内置模版来完成。这个内置的模版就是 `template_postgis`。它能够支持 PostgreSQL 的空间数据引擎 PostGIS 的所有函数或是操作符。这些函数或是操作符就是针对空间数据进行管理与处理的功能模块。在建立空间数据库的时候，也存在着两种方式，一种是以命令行进行创建，另一个则是使用 SQL 语句建立。具体空间库的创建如下所示：

以命令行的方式进行空间数据库的创建：

```
#createdb -T template_postgis business_spatial_db
```

以 SQL 语句的方式进行空间数据库的创建：

```
Postgres=# CREATE DATABASE business_spatial_db
```

```
TEMPLATE=template_postgis
```

1) 空间数据表的建立

数据表是由一组拥有相同命名字段的行组成的。并且每一个字段都有一类特定的数据类型。而 SQL 并没有对表中的行顺序进行设定，但每个字段在行里却有着固定的位置。当用户需要进行查询时，仍可以进行排序来达到自己获得的数据信息的目的。

空间数据表的创立如下所示：

```

CREATE TABLE road_Geom(
Road_ID      int 8,
Road_NAME    varchar(80)
);
SELECT AddGeometryColumn('public','road_Geom','gemo',356,'LINESTRING',2);

```

Varchar 是一个数据类型，它可以存储任意的字符串；(80)则是说明了这个字符串的长度最大为 80 个字符。int 即普通的整数类型；AddGeometryColumn 函数的功能是向表中增加几何字段。

2) 采用可视化工具建立数据库与数据表

PostgreSQL 提供了可视化的工具 pgAdminIII，使用它可以非常便捷的创建出带有 PostGIS 模版的空间数据库。具体操作如下：

链接 PostgreSQL9.2 以后，右键点击数据库选择新建数据库，在模版中选择 template_postgis 模版，再将名称、所有者以及变量、权限等进行填写与配置，这样 business_spatial_db 数据库就创建好了，图 4.5 则为创建的演示。

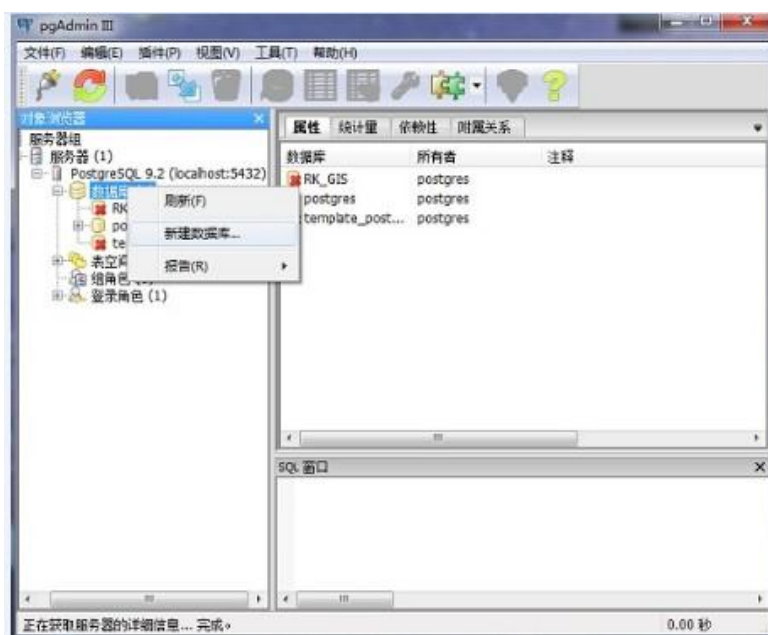


图 4.5 可视化工具创建新的数据库

4.4.2 空间数据入库

有两种方式能够将 shapefile 数据文件导入到 PostGIS 的数据库中。一种是应用智能化工具 PostGIS Shapefile Import/Export Manager 将 shapefile 文件存入数据库中；一种则是应用命令行将 shapefile 数据文件导入。本文采用了命令行的方式完成了 shapefile 数据文件的导入工作，如图 4.6 所示：

下面以命令行的方式进行了 shapefile 数据文件的入库：

E:

```
cd e:\SOFT\postgresql\bin
```

```
shp2pgsql.exe -s 4263 -W LATIN1 d:\baiduyundownload\business\餐饮业.shp 餐饮业>e:\business_spatial_db.sql
```

```
psql -d JCDL -f d:\baiduyundownload\business\餐饮业.shp 餐饮业
```

业>e:\business_spatial_db.sql postgres

数据导入的结果可以通过 pgAdminIII, 来进行查看, 查看的结果如图 4.6 所示。

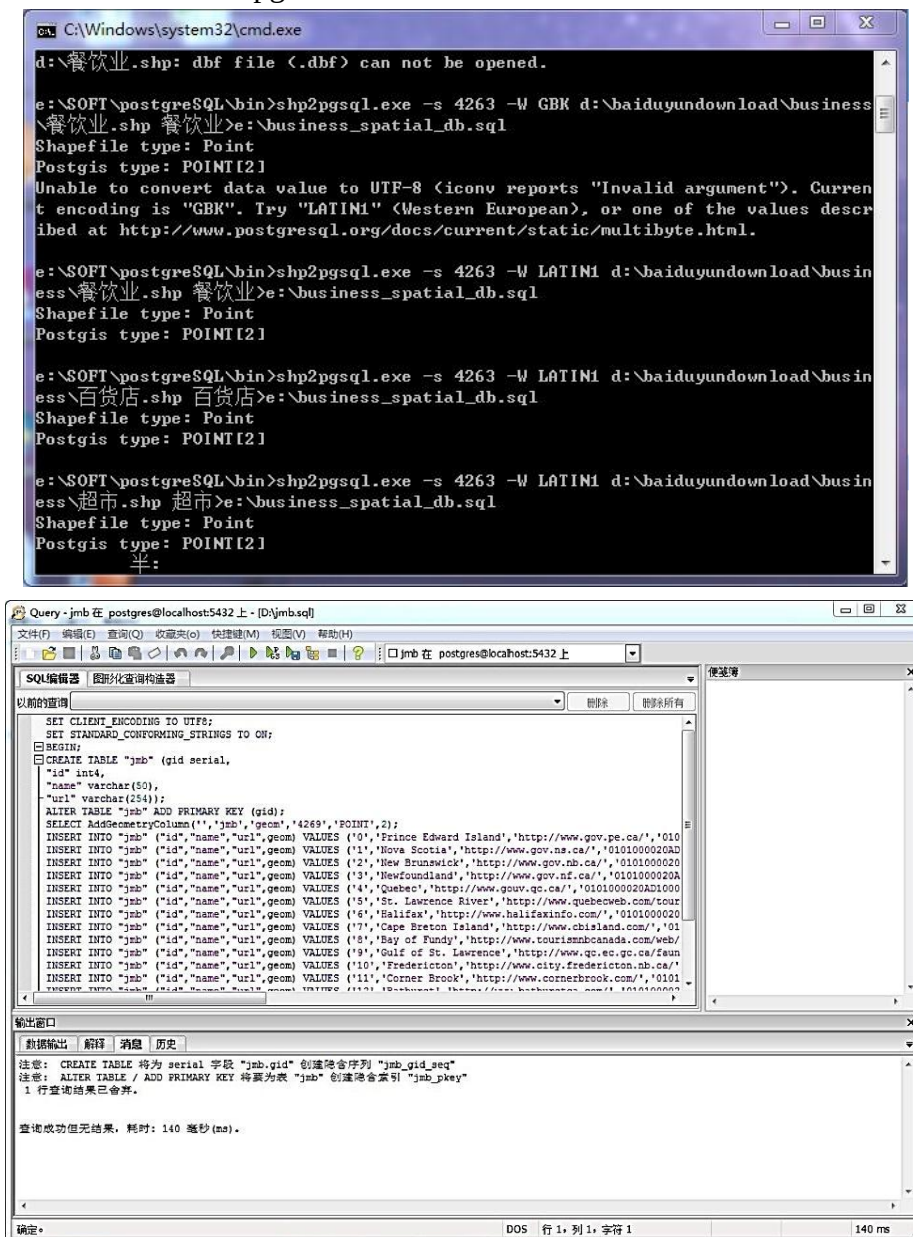


图 4.6 数据入库示意图

4.4.3 空间数据的浏览

PostGIS 的可视化管理工具 pgAdminIII 虽然可以管理数据库当中的空间数据,却不能很直观的反应出数据库中的属性数据,而 QGIS 可以完成此项工作。QGIS 既可以展现 PostgreSQL 数据库中的属性数据以及空间数据,同样可以展现 eVis 数据库中的数据,Oracle 空间图层数据以及 Oracle GeoRaster 数据以及 SQL anywhere 的图层数据。QGIS 当中内含的插件 SPIT,这个插件可以直接把 shapefile 格式的数据导入到已经建立好的 PostgreSQL 的数据库之中去。如图 4.7 所示。

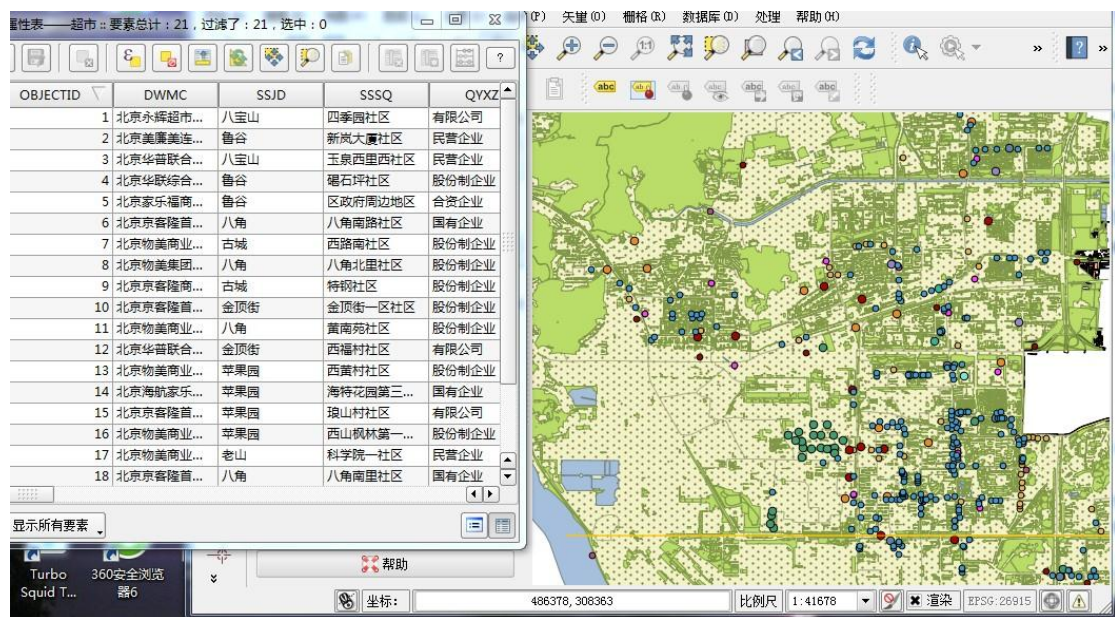


图 4.7 PostGIS 中的北京市石景山区数据展示图

4.5 重点流通企业信息管理系统的实现

基于 PostgreSQL 与 PostGIS 的商务委重点流通企业信息管理的数据分别为空间数据以及业务数据两种。这两种数据都将统一存储与 PostgreSQL 这个开源的数据库中。数据库的设计遵循 3NF 原则,对于每张表的每个数据项之间都消除了传递性依赖;并且遵循 3NF 设计的数据库,它的结构合理,不存在冗余的信息,能够很好的将空间数据与属性数据的关联表现出来。本文中,空间数据库在设计过程之中,使用了 PostgreSQL 这个全开源又强大的对象-关系型数据库并应用了其针对空间数据而发展的扩展模块 PostGIS 作为数据库的管理系统,使用开源的 GIS 软件 QGIS 作为本系统的二次开发平台,针对用户对于业态、限上/限下、流通企业等信息的管理,完成了用户提出的功能需求。

4.5.1 北京市石景山区概况

北京市石景山区是北京市西部的一个行政区,它也是北京市六个城区之一。它地处

北纬 39°53-39°59′，东经 116°07-116°14′，距天安门只有 16 千米，而且它还处在长安街的西向延长线上。位于北京市石景山区的西山风景区南麓和永定河冲积形成的扇面上，地势北高南低，略有起伏。石景山区的总占地面积为 86 平方千米，人口共有 61.6 万(2010 年)，下设了 10 个街道，气候条件是暖温带半湿润气候。石景山是因其地域上京都“第一仙山”的因素而得名。对石景山地区的地理位置和地域特点历史上描述是：“东临帝阙，西濒浑河”^[52]。

4.5.2 重点流通企业信息管理系统

随着近些年来北京市经济的迅猛发展，为响应北京市政府对于各区发展区内特色产业经济的政策，石景山区也不断的扩大招商引资的比例，大力发展区内特色产业经济。北京市石景山区政府决定采用信息化办公方式，并着力于推展区内各部门之间信息共享化，明确化，一体化。所以，石景山区商务委部门决定针对其监管下的重点流通企业的发展与规划等态势进行信息化管理。

因为石景山区商务委是石景山区政府下的一个部委，而它的重点流通企业信息管理系统只是一个小型的项目，所以本着节省成本的理念，本课题采用了全开源软件来完成这个项目。选取了较强大的开源型对象-关系数据库 PostgreSQL 及其的空间数据引擎 PostGIS 对项目当中的空间数据和属性数据进行存储与管理。而前端则应用 QGIS 进行展示，并应用 QGIS 平台进行二次开发，来完成客户提出的功能需求。

首先，以北京市石景山区的基础数据、空间数据和属性数据进行空间数据库的建立，并以此来展现商委监管下的各个商业业态，例如：专卖店、超级市场、大型综合超市、购物中心、蔬菜服务站等的空间、属性以及综合的地理信息^[53]。应用 PostGIS 有效的管理相关的数据，以便于满足商委以及其他部门对各类商业业态的信息化查询、统计、分析等要求。利用 QGIS 所提供以及本研究中扩展的空间分析功能对业态数据进行深入的挖掘与分析，以此来为石景山区经济发展当中的规划与发展提供支持。

(1) 系统界面

北京市石景山区重点流通企业信息管理系统，主要由以下几个功能组成：对于商业业态的统计与分析；限上或是限下企业网点分布的布局；针对流通企业的分析；对于商委部门职能规划地带的展示；便民的蔬菜服务站点的分布以及统计；一刻钟周边的定位查询；以及经济中心地带的特殊功能商业服务发展规划功能，它是针对石景山区经济中心地带一轴、三心、四街的统计、分析、以及规划展示的独立功能模块。以下 4.8 就是北京市石景山区重点流通企业信息管理系统的界面。

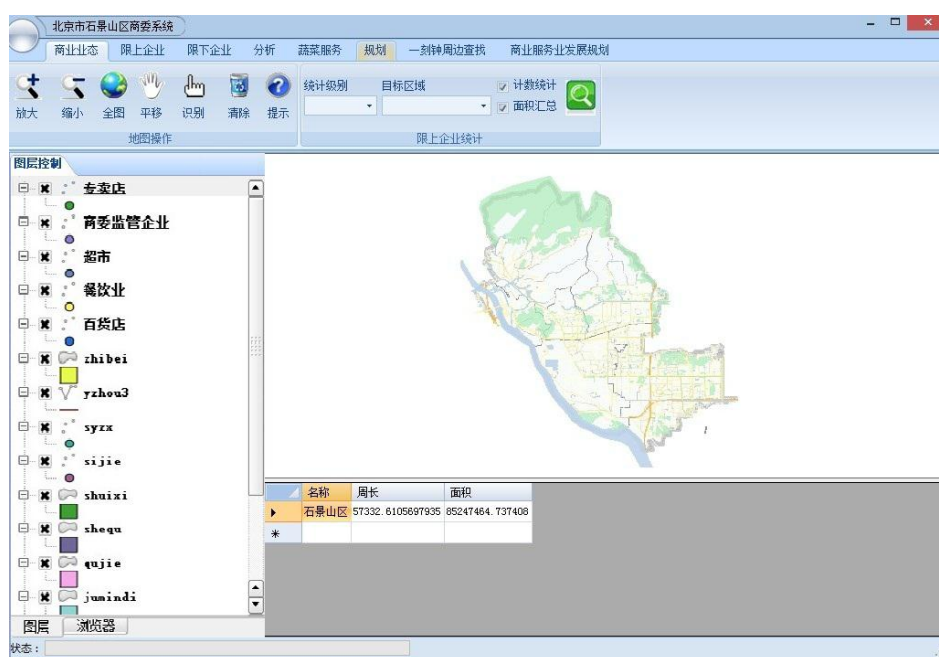


图 4.8 北京市石景山区重点流通企业信息管理系统

(2) 地图的基本操作

对地图的基本操作有放大、缩小、全图、平移等功能。它们可以使用户轻易的对地图进行一些列的操作，并方便用户快速的定位自己的感兴趣区，然后对这些感兴趣区进行浏览、查看等。以下便是地图基本操作的源码：

```
#include "sjsqgis.h"

sjsqgis:: sjsqgis (QWidget *parent, Qt::WFlags flags) : QMainWindow(parent,
flags)
{
    ui.setupUi(this);
    QString myPluginsDir = "c:/ProgramFiles/sjsqgis/plugins";
    QgsProviderRegistry::instance(myPluginsDir);
    //创建地图
    mpMapCanvas= new QgsMapCanvas(0,0);
    mpMapCanvas->freeze(false);
    mpMapCanvas->enableAntiAliasing(true);
    mpMapCanvas->setCanvasColor(QColor(255,255,255));
    mpMapCanvas->useImageToRender(false);
    mpMapCanvas->setVisible(true);
    mpMapCanvas->refresh();
    mpMapCanvas->show();
    mpMapCanvas->setFocus(); //布局窗口部件
    mpMapLayout = new QVBoxLayout();
    mpMapLayout->addWidget(mpMapCanvas);
    ui.frameMap->setLayout(mpMapLayout);
}
```

```

        setCentralWidget(ui.frameMap); //创建 Action 行为
        connect(ui.mpActionPan,SIGNAL(triggered()),this,SLOT(panMode()));
connect(ui.mpActionZoomIn,SIGNAL(triggered()),this,SLOT(zoomInMode()));
        connect(ui.mpActionZoomOut,SIGNAL(triggered()),this,SLOT(zoomOutMode()));
connect(ui.mpActionAddLayer,SIGNAL(triggered()),this,SLOT(addLayer()));
        //建立工具条
        mpMapToolBar=addToolBar(tr("File"));
        mpMapToolBar->addAction(ui.mpActionAddLayer);
        mpMapToolBar->addAction(ui.mpActionPan);
        mpMapToolBar->addAction(ui.mpActionZoomIn);
        mpMapToolBar->addAction(ui.mpActionZoomOut); //创建 maptool 功能
        mpPanTool= new QgsMapToolPan(mpMapCanvas);
        mpPanTool->setAction(ui.mpActionPan);
        mpZoomInTool = new QgsMapToolZoom(mpMapCanvas,FALSE);
        mpZoomInTool->setAction(ui.mpActionZoomIn);
        mpZoomOutTool = new QgsMapToolZoom(mpMapCanvas,TRUE);
        mpZoomOutTool->setAction(ui.mpActionZoomOut);
    }
    sjsqgis:: sjsqgis ()
    {
        delete mpZoomOutTool;
        delete mpZoomInTool;
        delete mpPanTool;
        delete mpMapToolBar;
        delete mpMapCanvas;
        delete mpMapLayout;
    }
    void sjsqgis::panMode()
    {
        mpMapCanvas->setMapTool(mpPanTool);
    }
    void sjsqgis::zoomInMode()
    {
        mpMapCanvas->setMapTool(mpZoomInTool);
    }
    void sjsqgis::zoomOutMode()
    {
        mpMapCanvas->setMapTool(mpZoomOutTool);
    }
    void sjsqgis::addLayer()
    {
        //读取矢量数据
        QString myLayerPath ="E:\\Qgis\\project\\sjsqgis\\data";

```

```

// 此处需要换成自己的矢量文件
QString myLayerBaseName = "test";
QString myPoviderName="ogr";
QList<QgsMapCanvasLayer> myLayerSet;
QgsVectorLayer * myLayer=new QgsVectorLayer
(myLayerPath,myLayerBaseName,myPoviderName);
if (myLayer->isValid())
{
    QgsSingleSymbolRenderer *myRenderer = new
QgsSingleSymbolRenderer(myLayer->geometryType());
    myLayer->setRenderer(myRenderer);
    QgsMapLayerRegistry::instance()->addMapLayer (myLayer,true);
//设置画布的 extent
    mpMapCanvas->setExtent(myLayer->extent());
    myLayerSet.append(QgsMapCanvasLayer(myLayer));
    mpMapCanvas->setLayerSet(myLayerSet);
}
else
{
    return;
}
}

```

(3) 识别与提示功能

系统的界面左侧是图层控制，界面上方是管理图层的工具，在这里点击识别或是提示，可以看到电子地图相应的动态变化。比如，当鼠标点击识别时，即会弹出识别的窗口，然后选择图层下拉框，选择要识别的图层，就会弹出属性窗口并显示该图层的属性信息。如图 4.9 所示。

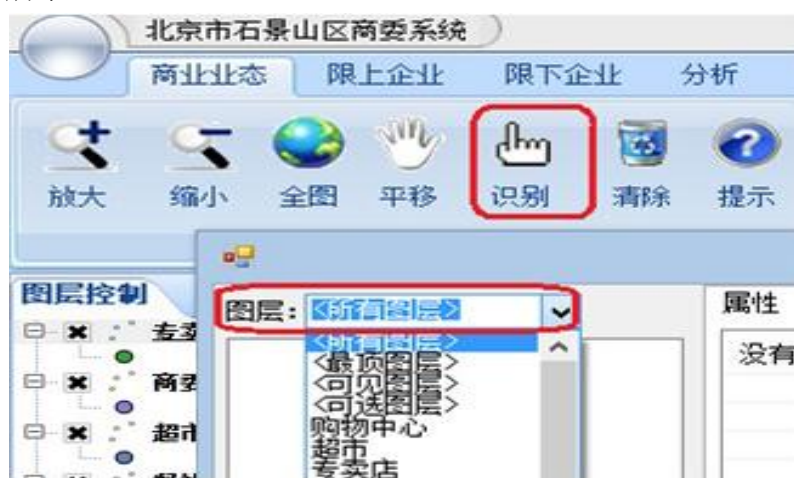


图 4.9 识别功能示意图

提示功能则是通过用户点击提示按键，然后鼠标滑动到图标的上方，就可以浏览选

中地物的名称。如图 4.10 所示。

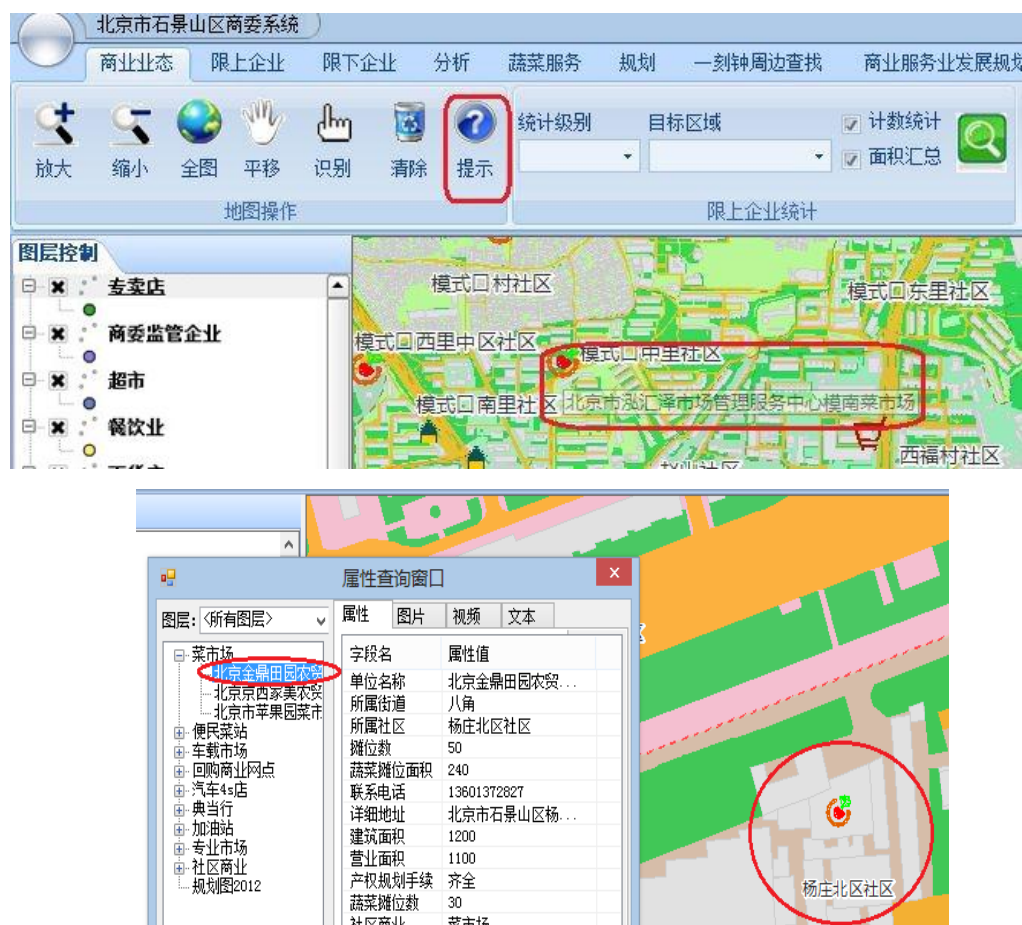


图 4.10 提示功能示意图

(4) 简单查询功能

本部分模块指在为用户提供获取其所需要的信息的一些手段，如属性查询、空间查询等。属性查询是指在地图上针对某一空间对象的属性数据进行查询，该数据可以是文本亦可以是数字等；空间查询是指在给定了某一个空间对象的属性值，然后根据这个具体的属性值来找出能够满足条件的空间对象。客户可以通过查询快速的找到他所要的兴趣点，尔后查看此兴趣点的相关空间信息，如地理位置，业态分布等，这样能够使用户全面、多方位的了解相关的信息。

单一查询：单一查询是指当用户单击一个点时，就能够获取一个地物的属性信息，例如百货店的详细属性信息。

多边形查询：多边形查询是指当用户利用鼠标在电子地图上画出一个不规则的多边形时，就能够获取在这个不规则多边形内的地物的属性信息，并以表的形式来显示查询对象的具体属性。如 4.11 所示。



图 4.11 多边形查询的示意图

一刻钟周边查询：一刻钟周边查询是指用户在电子地图上画一个圆，就能够获取与这个圆距离在一刻钟范围内的业态的属性信息，并将其以表的方式显示出来。如图 4.12 所示。



图 4.12 缓冲区分析示意图

(5) 分析与统计功能

空间分析是指对空间数据进行分析的有关技术的总称。空间分析以空间数据以及空间数据库为基础，以数理统计分析、代数运算以及几何逻辑运算等手段来解决地理空间中的实际问题。其目的就是为了提取空间信息中所隐含的信息，然后将这些隐含的信息进行传输，并以此来辅助 GIS 工作者进行决策。

本文的统计功能是针对商业业态，限上或是限下企业，蔬菜服务等进行分析。通过这些统计功能能够计算出某一个街道里，商业业态以及蔬菜服务的个数，并进行分析所选街道内业态或是蔬菜服务站的分布及展示出其的具体属性信息。如图 4.13、图 4.14 所示。



图 4.13 蔬菜服务站点分布统计示意图



图 4.14 商委监管企业统计示意图

清除功能则是在针对业态进行统计或者是进行规划分析，蔬菜服务分析时，对所选中的分析进行清除，以免对下一次的统计分析产生影响。如图 4.15 所示。

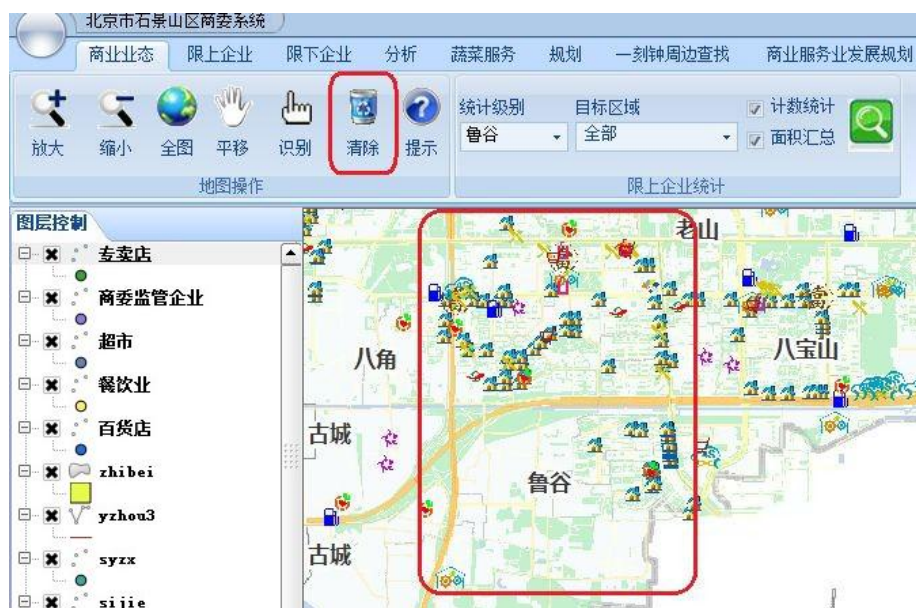


图 4.15 清除功能示意

(6) 商业服务业发展规划功能

商务服务发展规划功能是本文应北京市石景山区商委提出的特殊要求进行的功能模块开发。它是针对石景山区的经济中心地带即所谓的一轴三心四街进行的有针对性的查询、统计及分析。用户想借此来查看经济中心地带的发展，并为将来此地的发展规划提供有利的依据。

此模块分为三个部分，一轴，三心，四街，用户可以分别对其进行查询，统计分析等操作，以此来获取经济中心地带的信息。用户还可选择叠加规划图来查看此部分的不同年限之间规划的差异如图 4.16 所示：

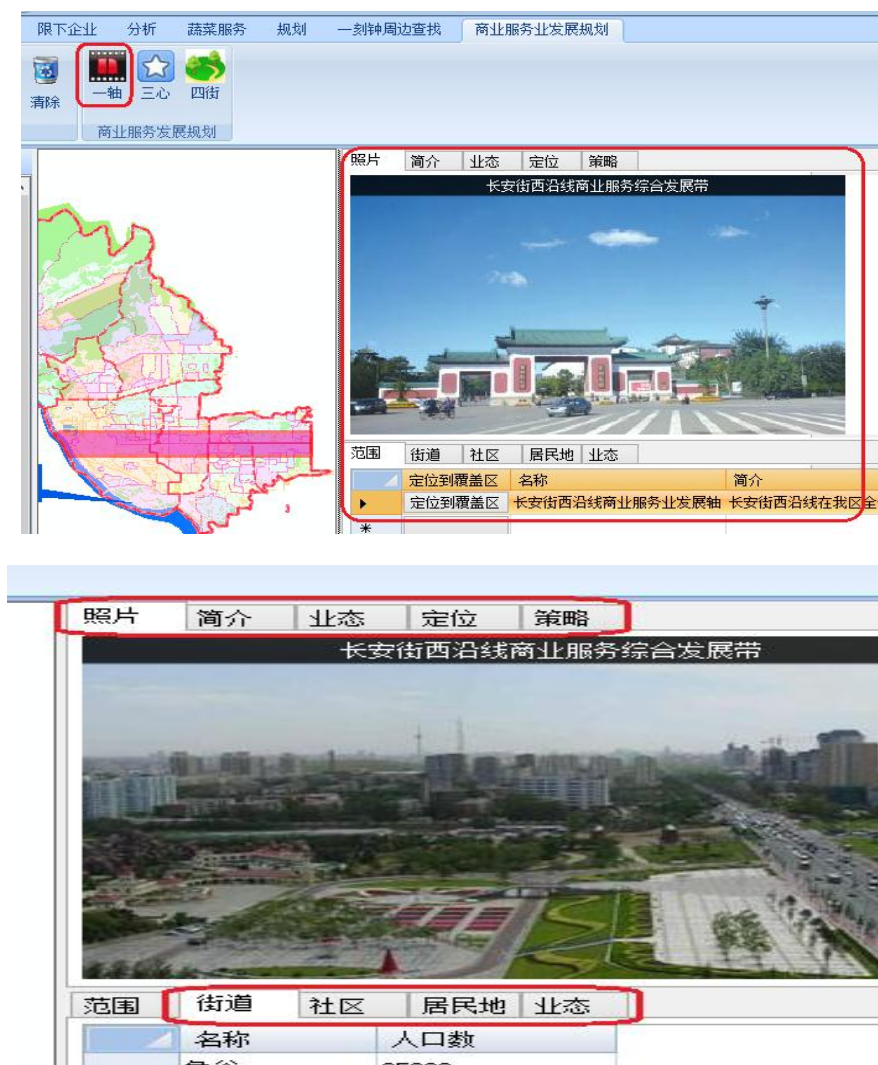
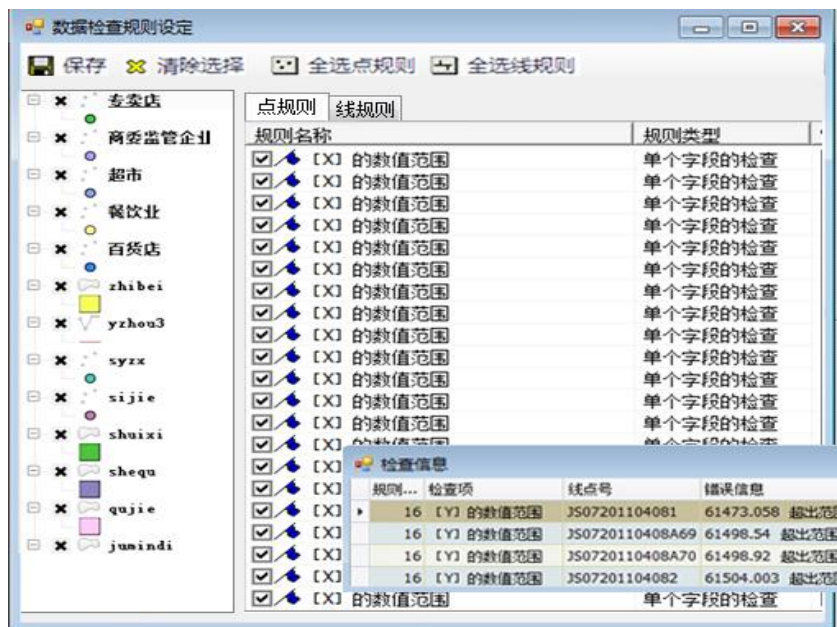


图 4.16 一轴的商业服务发展规划

(7) 数据检查功能

在 QGIS 当中点图层上的拓扑规则包括必须覆盖、必须有端点、必须在内部，比如在项目中选择了一个多边形的图层，而这点就必须在这个图层的内部否则 QGIS 会报错；不能够有重复，在 QGIS 中，若是一个点表示了两次或是更多就会发生错误；必须无无效的几何图形存在；不能有多部分的几何图形，若是有了多部分将会被 QGIS 写入到错误的字段中。在线图层上的拓扑规范，结束点必须要覆盖；不能够有重复，每当一行特性表示了两次甚至是两次以上 QGIS 将会报错；必须无伪节点的存在；不能有悬挂，因为这将会显示线的过度层而导致 QGIS 的出错；在面图层上的拓扑规则，必须要包含，即多边形图层必须要包含另一个图层中的至少一个点图层；不能有重复，就是说同一层不能够有相同的几何图形；不能有任何的间隙，在相邻的多边形中，它们之间是不允许有间隙的存在；必须没有无效的几何图形存在，如环形的多边形必须闭合，环形不但定义了内环同时应当在定义外环的界限等规则。最后就是不能够重叠，相邻的多边形之间不能够有重叠，否则 QGIS 会报错。



第五章 总结与展望

5.1 总结

随着计算机技术的不断发展,以计算机技术为支撑的众多信息技术也得到了长足的发展。而在这样的大背景下,不断发展的 Open-source 软件也渐渐成为了一种趋势。开源性软件正以其的低成本、小体积、较为强大的性能与资源共享等诸多的优点,慢慢赢得了众多用户的青睐。而在众多的开源性软件中,开源 DBMS 以及开源型 GIS 软件也赢得了越来越多目光。这也为 GIS 的发展带来了新方向,即开源型软件在 GIS 中的应用。

本课题结合了北京市石景山区重点流通企业信息管理系统的项目,进行了空间数据库具体设计以及前端客户端的应用开发。在总结了传统的商业数据库及 GIS 软件的优缺点,实现技术以及发展趋势之后,开始了本课题的研究工作。

首先,依据数据库、空间数据模型以及空间数据库的基本理论知识为基础,进行了基于 PostGIS 的北京市石景山区重点流通企业空间数据库的设计。选择了功能上较为强大的对象-关系型数据库 PostgreSQL 以及其的空间扩展模块 PostGIS,进行了空间数据库的需求分析、分层设计、概念设计、逻辑设计及完整性约束设计等工作。其次,依照上述的设计思路与逻辑,进行了北京市石景山区重点流通企业空间数据库的实现工作。选取了开源的桌面 GIS 软件 QGIS 作为客户端,对 PostGIS 中的空间数据进行展示。最后,根据北京市石景山区商委的具体需求,进行了北京市石景山区重点流通企业信息管理系统的开发工作。采用 C++ 语言对客户端 QGIS 进行了二次开发的工作,并实现了桌面 GIS 中一般的基础功能。还实现了商委对其监管下的重点流通企业进行可视化查询,统计分析,以及对经济中心地带的针对性统计、分析、查询等功能。

本课题研究出的成果,在北京市石景山区商务委的实际工作中得到了充分肯定。应北京市石景山商委的要求,在完成石景山区重点流通企业信息管理系统的整个过程中,不仅巩固和强化了笔者在攻读硕士研究生的前期所学到的专业知识和所培养的能力,还使我增长了大量的新知识以及培养了新能力。整个过程使笔者受益匪浅。

5.2 展望

由于笔者的经验尚浅,且精力与时间等诸多的原因,所以本课题还有地方并不十分的完善。第一,与用户交互方面上有待改进。为使用户能够获得更好的操作体验,还应

对系统的界面做的更加简洁与美观。第二,还应将空间数据库的设计更加的完善,尤其

是最为重要的完整性约束的设计上。本课题只是做了实体完整性、参考完整性，域完整性，触发器等约束的设计等。空间语义的完整性只做了拓扑语义的完整性约束，对于其他的空间语义完整性、时态完整性、组合语义完整性以及空间数据多版本等约束并没能完全的实现。第三，在系统的完整性方面上，出于对数据信息的保密措施，本系统只是做了 C/S 平台下的研发工作，并没有实现信息共享这一事项。所以在对于企业、业态等重点的数据信息上，还只能通过商委来进行更新与维护，并且当其他部门需要这些数据信息时，也只能到商委部门去调取，这一点上确实不是十分的便利。

本系统是应石景山区商委的要求，针对石景山区而设计开发的重点流通企业信息管理系统，具有一定的限制特性，但因采用全开源软件进行设计并实现的这套解决方案，仍可在其他部门甚至是其他区域的项目中进行应用的推广。因此，在往后的实际工作中，将针对政府各个部门的数据信息共享、空间数据库等相关技术进行深入的研究，为进一步降低中小型企业或是中小型项目当中的成本，提高信息化技术的水平，提高政府部门对其职能监管下的数据信息的掌控能力而努力。

参考文献

- [1] 李建峰,世纪新宠儿---开源 DBMS[J].程序员,2006,1:82.
- [2] 夏鹏万. PostGIS 开启开源空间数据库未来. 软件世界[J] 2006.10 : 52—54.
- [3] Shashi Shekhar, Sanjay Chawla 著. 谢青昆,马修军,杨冬青等译. 空间数据库[M]. 北京机械工业出版社, 2003.
- [4] song. Cambodia environment based on MapServer map system in Information and Service Science IEEE 2002,815-1-2356-,5/04.
- [5] Jaroslav.GRASS was adopted to realize the integrated application of a model of solar radiation for documents 2002 GIS'02 December 3-5,USA.
- [6] C.George 等.采用开源的 MapWindow 开发的联合国土壤和水资源评价工具[J].
- [7] Bas Van-meulebrouk 等.利用开源 GIS 软件开展了南非政府的 HIV/AIDS 管理信息系统的研究[J].
- [8] Alessandro Bezzi 等.采用了开源的 GRASS 在荷兰的 ITC 支持下开展了考古方面的模型建模与管理研究[J].
- [9] Lars Gunnar,Trond Andresen.进行了基于开源的 MapServer 软件在地区健康管理方面上的研究与开发实践[J].
- [10] Andrew J.基于卡特琳娜飓风地图的灾害下死亡率与位置关系的研究[J].
- [11] Autodesk 公司.支持很多研究机构开展基于开源 MapGuide 的网络空间信息服务方面的研究[N].
- [12] NASA.NASAWind 支持开源影响发布技术的研究[N].
- [13] 熊静,张箐. 基于 MapServer 的遥感影像发布系统的研究[J]. 遥感信息,2007,01:53-57+75.
- [14] 郑斌,唐旭.基于开源 GIS 的城市基准地价信息发布平台的设计与实现[J].国土资源科技管理,2006,5:12-18.
- [15] 圣荣,刘友兆.基于开源 MapServer 的网络空间数据共享系统研究[J].农业网络信息,2007,11:24-26
- [16] 张大鹏,张锦.基于开源 WebGIS 软件的 110 指挥中心警情分析系统[J].科技情报开发与经济,2008,11:160-162
- [17] 杨朝晖,郑文峰,李晓璐.基于开源 WebGIS 的网络房地产估价系统[J].软件导刊,2008,6:13-17.
- [18] 朱俊峰,赵俊三.基于开源平台的中小型 WebGIS 应用研究[J].地理空间信息 2008,6:123-127.
- [19] 黄冲,韩元杰.基于开源 WebGIS 的最短路径分析实现[J].现代电子技术,2007,16:21-25.
- [20] 冯宇,桂岚.基于开源 WebGIS 的干线公路网用地控制系统[J].2007,2:57-59.
- [21] 宋现锋,刘军志,吴建国.开源代码技术的 FLASH 地图实现方法---以 MapServer+Ming 为例[J].地球信息科学, 2006, 4:88-92.
- [22] 胡庆武,陈亚男,周洋,熊成利. 开源 GIS 进展及其典型应用研究[J]. 地理信息世界,2009,01:46-55.
- [23] 吕德奎,秦红现.开源版 MapGuide 及其应用研究[J].测绘通报,2008,4:102-105.
- [24] 张锐.基于 PostgreSQL 的空间数据库的实现[D].南京大学,2002.
- [25] 洪金益,银迎. 空间数据库中的拓扑数据模型研究[J]. 西部探矿工程,2007,03:242-245.
- [26] PostgresQL support. [http:// docs.ocf.berkeley.edu/wiki/PostgresQL](http://docs.ocf.berkeley.edu/wiki/PostgresQL).伯克利大学网站.
- [27] PostgreSQL Global Development Group. PostgreSQL Reference Manual [EB/OL].
<http://postgis.refractory.net/documentation/manual-1.3>.2008-3-10.
- [28] 芮建勋.PostGIS 发展历程(PostGIS 的专栏).
<http://blog.cdsn.net/zkhappyfol/archive/2006/08/21/1103271.aspx>2006.8.

- [29] PostGIS Website. <http://postgis.refractory.net/> (Accessed May 19,2011).
- [30] PostGIS Manual[EB/OL]. <http://postgis.refractory.net/docs/2006>, 2008-03-25.
- [31] Kiwon Lee Dept. of Information System Engineering Hansung University Seoul, Korea, Technical Architecture for Land Monitoring Portal using Google Maps API and Open Source GIS.
- [32] Mainak Bandyopadhyay, Maharana Pratap Singh GIS Cell, MNNIT Allahabad India Varun Singh Department of Civil Engineering MNNIT Allahabad, India, Integrated Visualization of Distributed Spatial Databases.
- [33] Shang Yan-Ling , Xu Xu-Dong."Effective load-balancing frame-work for distributed WebGIS" International Conference on New Trends in Information and Service Science IEEE 2009, 978-0-7695-3687-3/09.
- [34] 彭晓明. PostgreSQL 对象关系数据库开发[M].北京:人民邮电出版社,2001.3-7,20-21.
- [35] 曾侃.基于开源 DBMS PostgreSQL 的地理空间数据管理方法研究[D].浙江大学,2007.
- [36] 梁启靓.基于 Geoserver 的开源 WebGIS 开发与应用[D].长安大学,2010.
- [37] OSGeo.The Open Source Geospatial Foundation[EB/OL].<http://www.osgeo.org> 2011.02.20.
- [38] 李永先.城市道路管理空间数据库的设计与实现[D].云南大学,2012.
- [39] 唐经宇.基于 S-57 标准的电子海图数据库设计与程序库开发[D].广东工业大学,2012.
- [40] 王雪丽.基于 Java EE 的辽河口湿地地理信息管理系统设计[D].中国海洋大学,2013.
- [41] 张盼.基于 PostGIS 的海岸保护与利用规划空间数据库设计与实现[D].辽宁师范大学,2009.
- [42] 赵芳.基于开源 WebGIS 平台的地理信息系统应用研究[D].郑州大学,2010.
- [43] 宋海萍.基于开源 GIS 软件的矿山基础测绘空间数据管理和应用[D].太原理工大学,2011.
- [44] Franeis P. Donnelly. Evaluating open source GIS for libraries[J]. Library Hi Tech. 2010, 28(1):131-151.
- [45] Mohammad Akbari, Mohammad A. Rajabi , Ali Mousavi and Morteza Naghdi. Dept. of Surveying and Geomatics Eng., University of Tehran Tehran. Data Manipulation by PostGIS as a Free/Open Source Database for Web GIS Applications. 2012, 11.
- [46] P. Di Felice. Dipartimento di Ingegneria Industriale edell Informazione, Economia Università di L'Aquila (Italy). A Short Term Solution to Implement Applications about Moving Points on top of Existing DBMSs. 2013, 1.
- [47] 苏俊.基于开源 WebGIS 的郑州市水资源信息系统设计与实现[D].郑州大学,2011.
- [48] PostgreSQL 学习手册(数据表).
<http://www.cnblogs.com/stephen-liu74/archive/2011/12/16/2290803.html>
- [49] 杨德严.开源 GIS 开发与应用研究[D].昆明理工大学,2011.
- [50] QuantumGIS. Welcome to the QuantumGIS Project [EB/OL]. <http://www.qgis.org/>, 2011.03.08.
- [51] 付博,余志伟,孙成帅,伍世宾,田舫.基于 QGIS 平台的土壤侵蚀信息处理插件的开发[J].科技情报开发与经济, 2011, 22(6):23-27.
- [52] 武洪敬.石景山的魅力---从数据来看[J]. Census 专栏, 2012, 4(2):54-55.
- [53] 2012-2014 年度北京市商务部重点流通企业监测统计报表, 2014, 1.

致谢

光阴似箭，岁月如梭，转眼间人生最美好的学生时代即将画上圆满的句号。回首在母校的这七年里，拥有着太多的点点滴滴，这七年将会是我人生中最美好的回忆。

回首这三年间，经历了最初的研究生入学考试，再到进入梦寐以求的研究生生活，最后的毕业论文撰写，这期间得到太多的关怀、鼓励与帮助。于是，谨借此机会向研究生三年来给予我关心、帮助和鼓励的人们表示衷心的感谢！

首先要感谢我的导师兰小机教授。在得知我是跨专业的背景下，兰老师更是给予了我偌大的帮助。学业上的谆谆教诲，生活上的事无大小，科学研究上的严谨缜密态度等等。而在这最后的毕业论文撰写工作上，兰老师更是付出了巨大的精力并给予了我悉心的指导。从本论文的选题到方案的制定，再从资料的选取到论文的撰写，兰老师都给予了我很多的宝贵意见和建议。兰老师拥有敏锐的洞察力、渊博的知识和严谨的治学风范使我受益匪浅。在此，再次对尊敬的兰老师表示衷心的感谢和崇高的敬意。

感谢刘德儿老师。刘老师在学术研究上一丝不苟，每天和我们一起在实验室学习生活。他平易近人，对我们提出的问题总是很耐心的解答。在本论文研究过程中，刘老师给了我很多宝贵的意见和建议，在这里，对刘老师表示衷心的感谢。

感谢建测学院的所有老师在我研究生学习中的悉心教导及传授给我宝贵的知识，这一切是我的学位论文得以完成的坚实基础，并为我今后进一步提高专业技术水平建立了深厚的理论基础。

感谢实验室的所有同学，我们一起学习一起生活共同进步。感谢彭成、张恩、袁显贵、贺书、于海霞、孙迪、杨玉霞在生活中对我的照顾，感谢师弟刘义海、李翔、兰磊，感谢我的室友曾洁、李琳和已经毕业的苑海涛你们对我的无限包容，感谢所有在我论文写作期间对我帮助过的人。

最重要的要感谢我的爸爸妈妈，是你们给了我生命，给了我优越的生活条件，可以安心学习不必担心繁杂的事情。感谢亲人和朋友对我的关心和照顾，使我能一心一意的投入到学习和工作当中，并不断的给予我鼓励和支持，使我能够顺利完成学业。

对所有关心、帮助过我的可爱的人们说声谢谢。

攻读学位期间的研究成果

个人简历

姓名：林媛媛 性别：女 民族：汉 籍贯：北京 出生年月：1988 年 7 月 政治面貌：中共党员
学士学位：江西理工大学应用科学学院 2007 年 9 月-2011 年 7 月 网络工程 工学学士
硕士学位：江西理工大学 2011 年 9 月-2014 年 7 月 地图学与地理信息系统 理学硕士

发表论文

1. 林媛媛, 林川, 何德. 浅谈大数据时代下的 GIS 发展[J].江西测绘, 2013, 97(3):15-16.

参与项目

1. 北京市石景山区商委重点流通企业数据信息管理系统;
2. 北京市石景山区电子政务空间数据库共享平台信息系统;
3. 江西省赣州市旅游信息管理系统;

志如高遠 青春夢



硕士学位论文
Thesis for Master's Degree